

NEV

Algorithm Specifications and Supporting Documentation (version 1.1)

Jiang Zhang¹, Di Yan¹, Shuqin Fan¹, Debiao He², Yu Yu^{3,5}, Runqing Xu²,
Jiangxia Ge⁴, Yuanmi Chen⁵, Weijia Wang⁶, Cong Peng², Kang Yang¹, Hui
Guo¹, Hui Zhu⁷, Ruikang Liu⁷, and Jing Pan⁷

¹ State Key Laboratory of Cryptology, Beijing, China

² Wuhan University, Wuhan, China

³ Shanghai Jiao Tong University, Shanghai, China

⁴ China Telecom Quantum Group Ltd., Hefei, China

⁵ East China Normal University, Shanghai, China

⁶ Shandong University, Qingdao, China

⁷ Xidian University, Xi'an, China

June 28, 2026

Table of Contents

1	Introduction	1
1.1	Feature Statements	1
1.2	Overview of Design Approach	3
2	Preliminaries and Syntax	4
2.1	Notations	4
2.2	Syntax of Public-Key Encryption	5
2.3	Syntax of Key Encapsulation Mechanism	7
2.4	Hardness Assumptions	8
3	Specification of the NEV Encryption Family	11
3.1	Randomized Plaintext Encoding and Decoding	11
3.2	Description of NEV-PKE-OW	13
3.3	Description of NEV-KEM-CPA	16
3.4	Description of NEV-PKE-CPA	16
3.5	Description of NEV-KEM-CCA	17
3.6	Description of NEV-PKE-CCA	17
4	Concrete Parameters	18
5	Implementation Details	19
5.1	Partial NTT Multiplication	19
5.2	Partial NTT Inversion	21
5.3	Symmetric Primitives	21
5.4	Multi-target Countermeasure	22
5.5	Encoding of R_q Elements	22
5.6	Sampling Algorithms	25
5.7	The NEV-PKE-OW Component Scheme	27
5.8	The NEV-KEM-CPA Key-Encapsulation Mechanism	29
5.9	The NEV-PKE-CPA Public-Key Encryption	30
5.10	The NEV-KEM-CCA Key-Encapsulation Mechanism	30
5.11	The NEV-PKE-CCA Public-Key Encryption	31
6	Performance and Comparison	32
6.1	Performance on x86	33
6.2	Performance on ARM	34
7	Provable Security	37
7.1	Security Proof of NEV-PKE-OW	37
7.2	Security Proof of NEV-KEM-CPA	38
7.3	Security Proof of NEV-PKE-CPA	42
7.4	Security Proof of NEV-KEM-CCA	44
7.5	Security Proof of NEV-PKE-CCA	46
8	Known Attacks	47
8.1	Attacks against LWE	47
8.2	Solving the DNTRU $_{n,q,\chi_f,\chi_g,v}$ problem	53
8.3	Solving the sspRLWE $_{n,q,\chi_r,B_{\eta},\bar{v}}$ problem	53
8.4	Concrete Security Estimates Against Lattice Attacks	53

8.5	Attacks Exploiting Decryption Failures	54
8.6	Attacks against Symmetric Primitives	57
9	Side-Channel Vulnerabilities and Countermeasures	58
9.1	Security Analysis	59
9.2	A Discussion on Masking Countermeasures	60
A	Appendix	68
A.1	Proof of Theorem 7	68
A.2	Proof of Theorem 8	72
A.3	Proof of Theorem 9	85
A.4	Proof of Theorem 10	89

1 Introduction

1.1 Feature Statements

This document presents a family of PKE/KEM schemes, including NEV-PKE-OW, NEV-KEM-CPA, NEV-PKE-CPA, NEV-KEM-CCA and NEV-PKE-CCA (see Table 1), where NEV-PKE-OW is exactly the PKE scheme NEV' proposed at Asiacrypt 2023 [67]. All other schemes are derived from NEV-PKE-OW with well-established generic transformations [11,33]. The relations among those schemes are also depicted in Fig. 1.

Table 1: The descriptions of the schemes in this document

Schemes	Description
NEV-PKE-OW	Sec.3.2: the one-way secure PKE NEV' in [67]
NEV-KEM-CPA	Sec.3.3: a CPA secure KEM NEV' in the random oracle model
NEV-PKE-CPA	Sec.3.4: a CPA secure PKE supporting arbitrary message length
NEV-KEM-CCA	Sec.3.5: a CCA secure KEM from NEV-PKE-OW using FO transformation
NEV-PKE-CCA	Sec.3.6: a CCA secure PKE supporting arbitrary message length

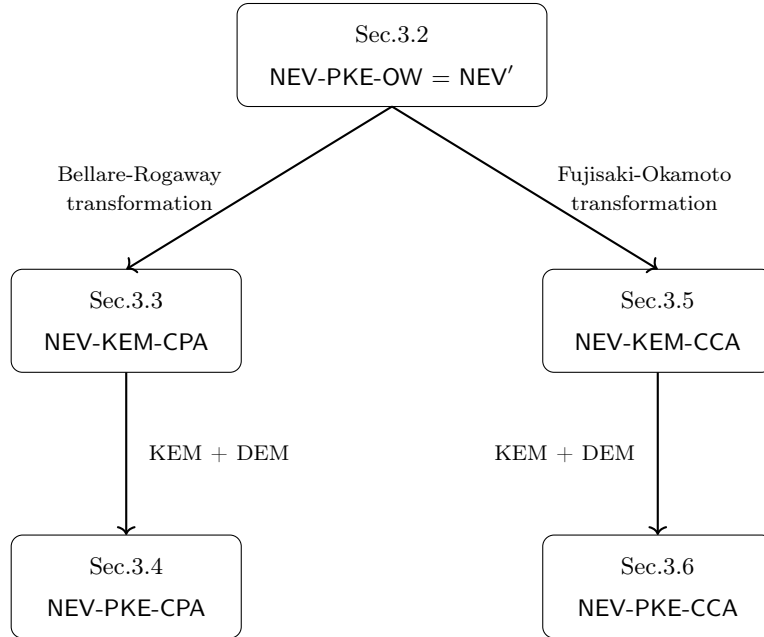


Fig. 1: Relations among schemes in the NEV family

As shown in [67], the PKE scheme NEV-PKE-OW is provably one-way secure under the NTRU and Subset-Sum Parity Ring Learning with Error (sspRLWE) assumptions in the standard model, where sspRLWE is an intermediate assumption which is essentially as hard as the standard RLWE assumption under appropriate choices of parameters. By relying on the random oracle (RO) heuristic [11], NEV-PKE-OW can be naturally transformed into a CPA-secure KEM

NEV-KEM-CPA or a CPA-secure PKE NEV-PKE-CPA. We can also obtain a CCA-secure KEM NEV-KEM-CCA or a CCA-secure PKE NEV-PKE-CCA by applying the Fujisaki-Okamoto (FO) transformation to NEV-PKE-OW in a standard way [33]. The followings summarize several advantages of the NEV schemes:

- **Novel and provably secure design:** NEV builds on the long-standing NTRU encryption [41] with several key enhancements that improve both security and performance. These include a well-studied NTT-friendly ring and a novel plaintext encoding. Formal security proofs are provided under the NTRU and sspRLWE assumptions in both the random oracle model (ROM) and the quantum ROM (QROM). Notably, sspRLWE is provably at least as hard as the standard RLWE assumption. Moreover, NEV-KEM-CCA and NEV-PKE-CCA achieve CCA security in the QROM without relying on the implicit-rejection strategy of [66]—a technique commonly used in other post-quantum KEMs.
- **No independence assumptions:** Unlike many existing schemes that rely on a problematic assumption in the analysis of decryption failures, NEV deliberately avoids error-correcting mechanism whose analysis would depend on the independence of coefficient failures—an assumption rigorously disproven both theoretically and empirically [29]. Instead, NEV adopts a decryption failure analysis based on the numeric computation of a worst-case union bound, which provides a provably sound upper bound on the failure probability. Consequently, this approach enables more robust parameter selection and substantially enhances the scheme’s reliability in real-world deployments.
- **Robustness against decryption failure attacks:** In contrast to many other NTRU-like schemes, NEV XORs the message with random bits, which decouples the decryption failure probability from the message distribution [27]. In addition, our simple yet robust method for analyzing decryption failures (see above) enables a thorough investigation of potential attacks exploiting such failures. Our analysis shows that an adversary cannot craft a malicious message or search for “bad ciphertexts” to induce even a single decryption failure at a cost below the claimed security levels. These results strongly support our concrete parameter choices, which guarantee a sufficiently low decryption failure rate for each target security level in active attack scenarios.
- **Conservative security estimates:** Our concrete security analysis adopts a conservative approach, relying primarily on the Core-SVP model introduced in [5]—a standard and widely referenced methodology for estimating the hardness of lattice problems. However, recognizing recent progress in hybrid attacks on LWE, as discussed in [17], we also account for the MATZOV algorithm [50], which represents a significant improvement in attacks against certain LWE parameter regimes. By including this algorithm in our estimates, we ensure that our security claims remain valid under the latest cryptanalytic understanding, particularly for long-term security requirements.
- **Easy to implement securely:** NEV is intentionally designed to avoid complex, leakage-prone operations such as higher-order error-correcting codes and floating-/fixed-point arithmetic. Instead, it unifies all parameter sets under the same modulus and fixed NTT, ensuring a clean and side-channel-friendly

arithmetic foundation. Additionally, unlike KEM schemes that rely on implicit rejection (e.g., ML-KEM), **NEV-KEM-CCA** does not need to protect ciphertext comparison results under masking. This key difference greatly simplifies the masking strategy, reduces implementation overhead, and removes a common source of side-channel vulnerability.

- **Competitive performance:** Even under several key conservative design principles—such as protection against side-channel leakage and the use of conservative security parameters mentioned above—**NEV** achieves highly competitive performance. The public key and ciphertext each consist of a single ring element, requiring less than 700 bytes for 128-bit security. This compact size is achieved without resorting to compression techniques that could raise patent concerns or rely on LWR-like assumptions. Moreover, for 128-bit classical security, the total time for key generation, encryption, and decryption is under 0.03 ms in a reference implementation and below 0.01 ms with AVX optimizations.
- **Patent relevance:** A key design goal of **NEV** is to avoid techniques that carry significant patent risks. These include foundational methods that were prominent in the NIST PQC competition, such as the framework for GAM/LPR-like encryption and rounding-like or reconciliation mechanisms for ciphertext compression. It is worth noting that **NEV** differs significantly from NTRU, and most of the relevant patents surrounding NTRU have now expired. In addition, we have filed two Chinese patent applications for **NEV**: CN202310188848.0, filed in early 2023, on the application of vector encoding/decoding to NTRU-like schemes; and CN201910340234.3, filed in early 2019, on the application of asymmetric RLWE. To ensure freedom to operate, we have submitted the necessary statements of free use as required.

1.2 Overview of Design Approach

Our algorithms are based on the PKE scheme called **NEV'** at Asiacrypt 2023 [67], which is basically a variant of the NTRU encryption using a special polynomial plaintext modulus that allows vector decoding. Specifically, let n be a power of 2, and q be a prime. Let $R = \mathbb{Z}[x]/(x^n + 1)$ and $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Let $v = (1 - x^{n/k}) \in R_q^*$ be a ring element, whose inverse is $v^{-1} = \frac{q+1}{2}\bar{v} = \frac{q+1}{2}(1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*$. Let $\chi_f, \chi_g, \chi_r, \chi_e$ be four distributions over R . Let $H : \{0, 1\}^k \rightarrow \{0, 1\}^k$ be a hash function modeled as a random oracle. The public key h and the ciphertext c of **NEV'** have forms of

$$h = vg/(vf' + 1) = g/(f' + v^{-1}), \quad c = hr + e,$$

where $g \leftarrow \chi_g, f' \leftarrow \chi_f, r \leftarrow \chi_r, e \leftarrow \chi_e$. Note that by setting $v = p$, the above construction is essentially the same as the original NTRU encryption. For decryption, we first compute $u = (f' + v^{-1})c = gr + f'e + v^{-1}e$. Let $\bar{v} = 1 + x^{n/k} + \dots + x^{(k-1)n/k}$, $e_0 = \bar{v}e \bmod 2$, and $2e_1 = \bar{v}e - e_0$, we have

$$v^{-1} = \frac{q+1}{2}\bar{v}, \quad v^{-1}e = e_1 + \frac{q+1}{2}e_0 \in R_q, \quad \text{and } u = gr + f'e + e_1 + \frac{q+1}{2}e_0 \in R_q.$$

Let $\text{Ext} : R_2 \rightarrow \{0, 1\}^{n/k}$ be a function that returns the first n/k coefficients of e_0 . By the nice property of $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ and the choice of $\bar{v}$ (and v^{-1}), it

is easy to check that e_0 is essentially a polynomial with coefficients consisting of k copies of $\text{Ext}(e_0)$. This fact allows us to use a coefficient vector of u in k dimension to decode each single coefficient of $\text{Ext}(e_0)$. Moreover, if $\chi_e = B_\eta$ is a center binomial distribution with parameter η , then there exist an efficient randomized algorithm $\text{Pt2noise}(\cdot, \eta)$ such that (1) sampling an element $e \leftarrow B_\eta$ can be equivalently done by first sampling $M \leftarrow \{0, 1\}^{n/k}$ and then outputting $e = \text{Pt2noise}(M, \eta)$; (2) for any $e = \text{Pt2noise}(M, \eta)$, we always have $M = \text{Ext}(\bar{v}e \bmod 2)$. This allows us to show that the above scheme is indeed a one-way secure PKE scheme under the sub-set sum parity RLWE (sspRLWE) problem with binomial error distribution. For convenience, we rename the PKE scheme using Pt2noise to encode the message as **NEV-PKE-OW**, which is exactly the PKE scheme **NEV'** in [67]. The public key h and the ciphertext c of **NEV'** that encrypts message $M \in \{0, 1\}^\kappa$ have forms of

$$h = vg/(vf' + 1) = g/(f' + v^{-1}), \quad c = hr + e,$$

where $g \leftarrow \chi_g, f' \leftarrow \chi_{f'}, r \leftarrow \chi_r, e \leftarrow \text{Pt2noise}(M, \eta)$.

Then, we can directly obtain a CPA-secure KEM **NEV-KEM-CPA** in the random oracle model. Concretely, let $H : \{0, 1\}^{n/k} \rightarrow \{0, 1\}^\kappa$ be a hash function modeled as a random oracle. The public key h and the ciphertext c of **NEV-KEM-CPA** are exactly the same as **NEV'**:

$$h = g/(f' + v^{-1}), \quad c = hr + e, \quad K = H(M)$$

where $g \leftarrow \chi_g, f' \leftarrow \chi_{f'}, r \leftarrow \chi_r, M \leftarrow \{0, 1\}^{n/k}, e \leftarrow \text{Pt2noise}(M, \eta)$. By the property of B_η and Pt2noise , the above ciphertext can be equivalently generated as follows:

$$h = g/(f' + v^{-1}), \quad c = hr + e, \quad K = H(\text{Ext}(e_0))$$

where $g \leftarrow \chi_g, f' \leftarrow \chi_{f'}, r \leftarrow \chi_r, e \leftarrow B_\eta$ and $e_0 = \bar{v}e \bmod 2$.

In order to obtain a IND-CPA secure PKE scheme, we follow the strategy of [11, 28] which allows to obtain a worst-case decryption failure independent from the choices of the message. Formally, let $H : \{0, 1\}^k \rightarrow \{0, 1\}^\ell$ be a hash function modeled as a random oracle, we have the following PKE scheme **NEV-PKE-CPA** with message space $\{0, 1\}^\ell$, where the public key h and the ciphertext $c = (c_1, c_2)$ have the following form:

$$h = g/(f' + v^{-1}), \quad c_1 = hr + e, \quad c_2 = H(\text{Ext}(e_0)) \oplus m,$$

where $g \leftarrow \chi_g, f' \leftarrow \chi_{f'}, r \leftarrow \chi_r, e \leftarrow \chi_e$ and $e_0 = \bar{v}e \bmod 2, m \in \{0, 1\}^\ell$.

Finally, by applying the Fujisaki-Okamoto (FO) transformation to **NEV-PKE-OW** in a standard way, we can also obtain an IND-CCA secure KEM **NEV-KEM-CCA** and PKE **NEV-PKE-CCA**.

2 Preliminaries and Syntax

2.1 Notations

Let n be a power of 2, and q a prime. We denote by R the ring $R = \mathbb{Z}[X]/(X^n + 1)$ and by R_q the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. The regular font letters (e.g., a, b)

represent elements in R or R_q (including elements in $\mathbb{Z}$ or $\mathbb{Z}_q$), and bold lower-case letters (e.g., $\mathbf{a}$, $\mathbf{b}$) denote vectors of R or $\mathbb{Z}$ elements. For a positive integer $\ell \in \mathbb{Z}$, by $[\ell]$ we denote the set $\{0, \dots, \ell - 1\}$. By $r' = r \bmod^\pm q$ we denote the unique element in the range $[-\frac{q-1}{2}, \frac{q-1}{2}]$ such that $r' = r \bmod q$. For an element $w \in \mathbb{Z}_q$, by $\|w\|_\infty$ we denote $|w \bmod^\pm q|$. The ℓ_∞ and ℓ_2 norms of a ring element $w \in R_q$ is defined as that of its coefficient vector $\mathbf{w} \in \mathbb{Z}_q^n$.

By $x \leftarrow \mathcal{D}$ we denote sampling x according to a distribution $\mathcal{D}$ and by $\mathcal{U}(S)$ we denote the uniform distribution over a finite set S . When we write that sampling a polynomial $g \leftarrow \mathcal{D}$ from a distribution $\mathcal{D}$ over $\mathbb{Z}$, we mean that sampling each coefficient of g from $\mathcal{D}$ individually. We use $\log_b$ to denote the logarithm function in base b (e.g., 2 or natural constant e) and $\log$ to represent $\log_e$. We say that a function $f : \mathbb{N} \rightarrow [0, 1]$ is *negligible*, if for every positive c and all sufficiently large κ it holds that $f(\kappa) < 1/\kappa^c$. We denote by $\text{negl} : \mathbb{N} \rightarrow [0, 1]$ an (unspecified) negligible function.

Binomial Distribution. The centered binomial distribution B_η with some positive $\eta \in \mathbb{Z}$ is defined as follows:

$$B_\eta = \left\{ \sum_{i=0}^{\eta-1} (a_i - b_i) : (a_0, \dots, a_{\eta-1}, b_0, \dots, b_{\eta-1}) \leftarrow \{0, 1\}^{2\eta} \right\}$$

Ternary Distribution. The ternary distribution $\mathcal{T}_\sigma$ with some positive real $\sigma \in (0, 1/2)$ denotes the distribution of sampling a variable $x \in \{-1, 0, 1\}$ with $\Pr[x = 1] = \Pr[x = -1] = \sigma$, and $\Pr[x = 0] = 1 - 2\sigma$. By this notation, we have $\mathcal{T}_{1/3} = \mathcal{U}(\{-1, 0, 1\})$ is the uniform ternary distribution, and $\mathcal{T}_{1/4} = B_1$ is the centered binomial distribution with $\eta = 1$.

Gaussian Distribution. The Gaussian function $\rho_{s,\mathbf{c}}(\mathbf{x})$ over $\mathbb{R}^m$ centered at $\mathbf{c} \in \mathbb{R}^m$ with parameter $s > 0$ is defined as $\rho_{s,\mathbf{c}}(\mathbf{x}) = \exp(-\pi \|\mathbf{x} - \mathbf{c}\|^2 / s^2)$. For lattice $\Lambda \subseteq \mathbb{R}^m$, let $\rho_{s,\mathbf{c}}(\Lambda) = \sum_{\mathbf{x} \in \Lambda} \rho_{s,\mathbf{c}}(\mathbf{x})$, and define the discrete Gaussian distribution over Λ as $D_{\Lambda,s,\mathbf{c}}(\mathbf{y}) = \frac{\rho_{s,\mathbf{c}}(\mathbf{y})}{\rho_{s,\mathbf{c}}(\Lambda)}$, where $\mathbf{y} \in \Lambda$. We omit the subscript $\mathbf{c}$ in the above notations if $\mathbf{c} = \mathbf{0}$.

Lemma 1 ([10,47]). *For any real $s, t > 0$, $c \geq 1$, $C = c \cdot \exp(\frac{1-c^2}{2}) < 1$, integer $m > 0$, and any $\mathbf{y} \in \mathbb{R}^m$ we have that $\Pr_{\mathbf{x} \leftarrow D_{\mathbb{Z}^m,s}}[\|\mathbf{x}\|_\infty > t \cdot s] \leq 2e^{-\pi t^2}$.*

Lemma 2 (Special case of [58, Theorem 3.1]). *Let $\mathbf{V} \in \mathbb{Z}^{n \times n}$ be the multiplication matrix corresponding to the ring element $v^{-1} = 1 - x^{n/k} \in R_q$. Let $\alpha, \beta, \gamma > 0$ be reals such that $\alpha \geq \omega(\sqrt{\log n})$, $\gamma = \sqrt{\alpha^2 + 2\beta^2}$ and $\alpha\beta/\gamma > \omega(\sqrt{\log n})$. Consider the following probabilistic experiment:*

Choose $\mathbf{x}_2 \leftarrow D_{\mathbb{Z}^n,\beta}$, then choose $\mathbf{x}_1 \leftarrow \mathbf{V}\mathbf{x}_2 + D_{\mathbb{Z}^n,\alpha}$.

Then, the marginal distribution of $\mathbf{x}_1$ is statistically close to $D_{\mathbb{Z}^n,\gamma}$.

2.2 Syntax of Public-Key Encryption

A public-key encryption (PKE) Π_{PKE} with plaintext space $\mathcal{M}$ consists of three PPT algorithms (KeyGen, Enc, Dec):

- **KeyGen**(1^κ): given a security parameter κ as input, output a pair of public and secret keys (pk, sk) , denoted as $(pk, sk) = \text{KeyGen}(1^\kappa)$.
- **Enc**($pk, M; r$): given the public key pk , a plaintext $M \in \mathcal{M}$ and a randomness r (which might be an empty string) as inputs, output a ciphertext C , denoted as $C = \text{Enc}(pk, M; r)$ or $C = \text{Enc}(pk, M)$ in brief.
- **Dec**(sk, C): given the secret key sk and a ciphertext C as inputs, output a plaintext M' (which might be $\perp$), denoted as $M' = \text{Dec}(sk, C)$.

On the notion of correctness, we recall the definition in [28], which focuses on worst-case correctness across all possible message choices. This definition is key to defending against attacks that exploit decryption failures.

Definition 1 (δ -correct [28]). We say that a PKE scheme Π_{PKE} with message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is δ -correct or has decryption failure rate δ if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}} \Pr [\text{Dec}(sk, c) \neq m : c = \text{Enc}(pk, m; r), r \leftarrow_{\$} \mathcal{R}] \right] \leq \delta.$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$.

Definition 2 (γ -spread [28]). We say that a PKE scheme Π_{PKE} with message space $\mathcal{M}$, ciphertext space $\mathcal{C}$, and randomness space $\mathcal{R}$ is (weakly) γ -spread if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}, c \in \mathcal{C}} \Pr[c = \text{Enc}(pk, m; r) : r \leftarrow_{\$} \mathcal{R}] \right] \leq 2^{-\gamma},$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$.

For our interest, we recall the standard security notions for PKEs, which is modeled by games between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ in Fig. 2.

OW-CPA Games:	IND-CPA/CCA Games:
<pre> 1 $(pk, sk) = \Pi_{\text{PKE}}.\text{KeyGen}(1^\kappa)$; 2 $M^* \leftarrow \mathcal{M}$; 3 $C^* = \Pi_{\text{PKE}}.\text{Enc}(pk, M^*)$; 4 $M' = \mathcal{A}(pk, C^*)$; 5 return $M' = M^*$; </pre>	<pre> 1 $(pk, sk) = \Pi_{\text{PKE}}.\text{KeyGen}(1^\kappa)$; 2 $(M_0, M_1, st) = \mathcal{A}_1^{\mathcal{O}(\cdot)}(pk)$; 3 $\mu^* \leftarrow \{0, 1\}$; 4 $C^* = \Pi_{\text{PKE}}.\text{Enc}(pk, M_{\mu^*})$; 5 $\mu' = \mathcal{A}_2^{\mathcal{O}(\cdot)}(C^*, st)$; 6 return $\mu' = \mu^*$; </pre>
Oracle $\mathcal{O}_{\text{Dec}}(C)$:	
<pre> 1 if $C = C^*$ then 2 return $\perp$; 3 end 4 $M = \Pi_{\text{PKE}}.\text{Dec}(sk, C)$; 5 return M; </pre>	

Fig. 2: Security Games for PKEs

Definition 3 (OW-CPA PKE). We say that a PKE scheme Π_{PKE} is OW-CPA secure if for any PPT adversary $\mathcal{A}$, its advantage

$$\text{Adv}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ow-cpa}}(\kappa) = \Pr[M' = M^*]$$

in the OW-CPA security game in Fig. 2 is negligible in security parameter κ .

Definition 4 (IND-CPA PKE). We say that a PKE scheme Π_{PKE} is IND-CPA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage

$$\text{Adv}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ind-cpa}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the IND-CPA security game with $\mathcal{O}(\cdot) = \perp$ in Fig. 2 is negligible in security parameter κ .

Definition 5 (IND-CCA PKE). We say that a PKE scheme Π_{PKE} is IND-CCA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage

$$\text{Adv}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ind-cca}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the IND-CCA security game with $\mathcal{O}(\cdot) = \mathcal{O}_{\text{Dec}}(\cdot)$ in Fig. 2 is negligible in security parameter κ .

2.3 Syntax of Key Encapsulation Mechanism

A key encapsulation mechanism (KEM) Π_{KEM} with session key space $\mathcal{K}$ consists of three PPT algorithms (KeyGen, Encap, Decap):

- **KeyGen**(1^κ): given a security parameter κ as input, output a pair of public and secret keys (pk, sk) , denoted as $(pk, sk) = \text{KeyGen}(1^\kappa)$.
- **Encap**($pk; r$): given the public key pk , and a randomness r as inputs, output a ciphertext C and a session key $K \in \mathcal{K}$, denoted as $(C, K) = \text{Encap}(pk; r)$, or $(C, K) = \text{Encap}(pk)$ in brief.
- **Decap**(sk, C): given a secret key sk and a ciphertext C as inputs, output a key K' (which might be a failure symbol $\perp$), denoted as $K' = \text{Decap}(sk, C)$.

We now recall the chosen-ciphertext security for KEMs from [22], which is modeled by the game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ in Fig. 3.

Definition 6 (IND-CPA KEM). We say that a KEM scheme Π_{KEM} is IND-CPA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage

$$\text{Adv}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ind-cpa}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the security game with $\mathcal{O}(\cdot) = \perp$ in Fig. 3 is negligible in security parameter κ .

Definition 7 (IND-CCA KEM). We say that a KEM scheme Π_{KEM} is IND-CCA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage

$$\text{Adv}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ind-cca}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the security game with $\mathcal{O}(\cdot) = \mathcal{O}_{\text{Dec}}(\cdot)$ in Fig. 3 is negligible in security parameter κ .

KEM Security Games:	Oracle $\mathcal{O}_{\text{Dec}}(C)$:
<pre> 1 $(pk, sk) = \Pi_{\text{KEM}}.\text{KeyGen}(1^\kappa)$; 2 $\mu^* \leftarrow \{0, 1\}$; 3 $(C^*, K_0) = \Pi_{\text{KEM}}.\text{Encap}(pk)$; 4 $K_1 \leftarrow \mathcal{K}$; 5 $\mu' = \mathcal{A}^{\mathcal{O}(\cdot)}(pk, C^*, K_{\mu^*})$; 6 return $\mu' = \mu^*$; </pre>	<pre> 1 if $C = C^*$ then 2 return $\perp$; 3 end 4 $K = \text{Decap}(sk, C)$; 5 return K; </pre>

Fig. 3: Game for IND-CCA Security of KEMs

2.4 Hardness Assumptions

Let n be a power of 2, and q a prime. Let $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Let R_q^* denote all invertible ring elements in R_q . Let $\chi_f, \chi_g, \chi_r, \chi_e$ be four probability distributions over R . Let $v \in R_q^*$ be a publicly known small ring element.

The NTRU Assumption. The computational NTRU problem $\text{NTRU}_{n,q,\chi_f,\chi_g,v}$ asks an algorithm, given $h = g/f \in R_q$ as input, to output f' , where $f' \leftarrow \chi_f, g \leftarrow \chi_g$ and $f = vf' + 1 \in R_q^*$. The decisional NTRU problem $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ asks an algorithm to distinguish the following two distributions:

$$\{h = g/f \mid f' \leftarrow \chi_f, g \leftarrow \chi_g, \text{ and } f = vf' + 1, g \in R_q^*\} \text{ and } \{u \mid u \leftarrow R_q^*\}.$$

The computational (resp., decisional) NTRU assumption says that it is hard for any PPT algorithms to solve $\text{NTRU}_{n,q,\chi_f,\chi_g,v}$ (resp., $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$) with non-negligible advantage over a random guess.

Remark 1. The above definition generalizes the common NTRU assumption with $v = p \in R_q^*$ for some integer p (e.g., $p = 3$ in [41,42,63,19]) with a publicly known ring element $v \in R_q^*$. We note that this generalization is mild up to the choices of the secret key distribution χ_f , because $\text{NTRU}_{n,q,\chi_f,\chi_g,v}$ is essentially equivalent to the standard NTRU problem $\text{NTRU}_{n,q,\chi'_f,\chi_g,p}$ with $\chi'_f = p^{-1}v\chi_f$ (or $\chi_f = pv^{-1}\chi'_f$). In fact, the polynomial $v = x + 2$ was recommended by the authors of the original NTRU cryptosystem as early as 2000 [42], and was investigated in [9,53,43,34,40,63].

Since its introduction [41], the NTRU problem has been studied more than 25 years, and there is no significant algorithmic progress. The decisional NTRU (DNTRU) assumption over the cyclotomic ring $R = \mathbb{Z}_q[x]/(x^n + 1)$, which is also known as the decisional small polynomial ratio (DSPR) assumption, has been extensively used and investigated in [63,48,25,31,15,59]. Notably, Stehlé and Steinfeld [63] showed that the DNTRU assumption indeed holds unconditionally if χ_f, χ_g are discrete Gaussian distributions of standard deviation $\sigma = \omega(n\sqrt{q})$ (We note that their proof mainly focuses on the special case $v = 3$, but it essentially applies to any invertible $v \in R_q^*$). For small secret distributions, a variant of the NTRU problem over $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ is also shown to be at least as hard as the worst-case approximate SVP problem on ideal lattices [59].

The RLWE Assumption. The computational RLWE problem $\text{RLWE}_{n,q,\chi_r,\chi_e}$ asks an algorithm, given a polynomial number of samples from the distribution

$\{(a, b = ar + e) \mid a \leftarrow R_q, e \leftarrow \chi_e\}$ as inputs, to output the secret $r \in R_q$, where $r \leftarrow \chi_r$. The decisional RLWE problem $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ asks an algorithm, given a polynomial number of samples to distinguish the following two distributions:

$$\{(a, b = ar + e) \mid a \leftarrow R_q, e \leftarrow \chi_e\} \text{ and } \{(a, u) \mid a \leftarrow R_q, u \leftarrow R_q\}.$$

The computational (resp., decisional) RLWE assumption says that it is hard for any PPT algorithms to solve $\text{RLWE}_{n,q,\chi_r,\chi_e}$ (resp., $\text{DRLWE}_{n,q,\chi_r,\chi_e}$) with non-negligible advantage over a random guess.

As an extension of the LWE problem [61], the RLWE problem was first considered in [64,49], and was provably as hard as some hard lattice problems such as the Shortest Vectors Problem (SVP) on ideal lattices.

The Subset-Sum Parity RLWE Assumption. We introduce a variant of the RLWE problem which we call subset-sum parity RLWE (sspRLWE) problem. Formally, the sspRLWE problem $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ asks an algorithm, given an RLWE instance $(a, b = ar + e) \in R_q$ as input, to output $ve \bmod 2 \in R_2$ for some fixed ring element $v \in R_2$. This name comes from the fact that for $R = \mathbb{Z}[X]/(x^n+1)$, the i -th coefficient of $ve \bmod 2 \in R_2$ is essentially equal to the parity of the subset sum $\sum_{v_j=1} e_{(i-j) \bmod n}$ of the coefficient vector $\mathbf{e} = (e_0, \dots, e_{n-1})$ of $e \in R_q$. The sspRLWE assumption says that it is hard for any PPT algorithms to solve $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ with non-negligible advantage over a random guess according to the distribution $\chi' = v\chi_e \bmod 2$.

Remark 2. Our sspRLWE problem can be seen as a generalization of the RLWE2 problem [28] from a special choice of $v = 1$ to a general chosen $v \in R_2$. On the first hand, the $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ problem is not harder than the corresponding RLWE problem $\text{RLWE}_{n,q,\chi_r,\chi_e}$. On the other hand, if the $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ is hard, it seems that a RLWE sample $(a, b = ar + e)$ essentially hides all the information about e , and that the best way for a PPT algorithm to solve the sspRLWE problem is to make a random guess on $ve \bmod 2$ according to the distribution $\chi' = v\chi_e \bmod 2$. Moreover, the problem of reducing $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ to $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ can be seen as the problem of solving $\text{DRLWE}_{n,q,\chi_r,\chi_e}$ with modular hints $ve \bmod 2$, and an efficient algorithm to solve $\text{sspRLWE}_{n,q,\chi_r,\chi_e,v}$ directly leads to a more efficient algorithm to solve $\text{RLWE}_{n,q,\chi_r,\chi_e}$ according to the study in [23,51,16].

For our purpose, we are particularly interested in the problem $\text{sspRLWE}_{n,q,\chi_r,\chi_e,\bar{v}}$ satisfying the following two conditions:

- $\bar{v} = 1 + x^{n/k} + x^{2n/k} + \dots + x^{(k-1)n/k} \in R_2$ for some $k|n$;
- χ_e is the center binomial distribution.

In the following, we first show that for discrete Gaussian noise distributions, the $\text{sspRLWE}_{n,q,\chi_r,D_{\mathbb{Z}^n,\gamma},\bar{v}}$ problem is at least as hard as the decisional RLWE problem $\text{DRLWE}_{n,q,\chi_r,D_{\mathbb{Z}^n,\beta}}$ for sufficiently large parameters $\gamma > \beta$, which can be extended to binomial distributions by a standard argument using Rényi divergence [8].

Theorem 1. *Let n, q, k, χ_r and $\bar{v}$ be defined as above. Let α, β, γ be three positive reals satisfying $\alpha \geq \omega(\sqrt{\log n})$, $\gamma = \sqrt{\alpha^2 + 2\beta^2}$, $\alpha\beta/\gamma \geq \omega(\sqrt{\log n})$ and $\gamma\sqrt{n} < q/2$. Let $D_{\mathbb{Z}^n, \beta}, D_{\mathbb{Z}^n, \gamma}$ be two discrete Gaussian distributions with parameter β and γ , respectively. If there is a PPT algorithm $\mathcal{A}$ solving the $\text{sspRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \gamma}, \bar{v}}$ problem (with probability negligibly close to 1), then there is another PPT algorithm $\mathcal{B}$ solving the $\text{DRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \beta}}$ problem.*

Proof. It is sufficient to give the description of $\mathcal{B}$. Formally, let $v^{-1} = 1 - x^{n/k}$, and it is easy to check that $v^{-1}\bar{v} = 2 \in R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Now, given a DRLWE tuple $(a, b) \in R_q \times R_q$ as input, $\mathcal{B}$ first randomly chooses a polynomial $e' \in R_q$ from the distribution $D_{\mathbb{Z}^n, \alpha}$, and sets $(a', b') = (v^{-1}a, v^{-1}b + e') \in R_q \times R_q$. Then, it runs algorithm $\mathcal{A}$ with input (a', b') , and obtains $w \in R_2$ from $\mathcal{A}$. Finally, $\mathcal{B}$ returns 1 if $w = \bar{v}e' \bmod 2$, otherwise returns 0.

We now analyze the behavior of algorithm $\mathcal{B}$. First, if $(a, b = ar + e)$ is a real $\text{DRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \beta}}$ tuple, then we have that the coefficients of e are chosen from $D_{\mathbb{Z}^n, \beta}$. By Lemma 2, we have that the distribution of $\hat{e} = v^{-1}e + e'$ is statistically close to $D_{\mathbb{Z}^n, \gamma}$. Since $\gamma\sqrt{n} < q/2$, we have that $\|\hat{e}\|_\infty < q/2$ with probability negligibly close to 1 by Lemma 1, which means that $\hat{e} \bmod q = \hat{e}$ holds with probability negligibly close to 1. Thus, the distribution of $(a' = v^{-1}a, b' = v^{-1}ar + \hat{e}) \in R_q \times R_q$ is statistically close to an $\text{sspRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \gamma}, \bar{v}}$ tuple. Using the fact that $v^{-1}\bar{v} = 2 \in R_q$ and $w = \bar{v}\hat{e} = \bar{v}e' \bmod 2$, we have that $\mathcal{B}$ will return 1 with probability negligibly close to 1. Second, if (a, b) is randomly chosen from $R_q \times R_q$, we have that $(a' = v^{-1}a, b' = v^{-1}b + e')$ is also randomly distributed over $R_q \times R_q$. This means that the probability for any $\mathcal{A}$ to output $w \in R_2$ such that $w = \bar{v}e' \bmod 2$ is negligible in n/k by our choice of $e' \leftarrow D_{\mathbb{Z}^n, \alpha}$ with $\alpha \geq \omega(\sqrt{\log n})$. In all, we have shown that $\mathcal{B}$ is a valid distinguisher for $\text{DRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \beta}}$ problem. This completes the proof.

Remark 3. The above proof is essentially obtained by replacing the pair $(a', b') = (2a, 2b + e')$ in the original proof of [67] with $(a', b') = (v^{-1}a, v^{-1}b + e')$. This change allows to obtain a slightly tighter proof. Concretely, by letting $\beta = \alpha$ we have that $\gamma = \sqrt{\alpha^2 + 2\beta^2}$ is about $\sqrt{3}$ times larger than β , this is much tighter than reductions exist from RLWE to a shortest vector problem and thus gives a strong theoretical connection between the hardness of sspRLWE and DRLWE . In fact, as discussed in [27] for the concrete hardness of the RLWE2 problem, we also believe that based on the state of the art of how RLWE problems are solved, the constant factor $\sqrt{3}$ is an artificial outcome of the proof, and the $\text{sspRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \gamma}, \bar{v}}$ problem with $\gamma = \beta$ is already as hard as the $\text{DRLWE}_{n, q, \chi_r, D_{\mathbb{Z}^n, \beta}}$ problem. Intuitively, if the DRLWE problem $\text{DRLWE}_{n, q, \chi_r, \chi_e}$ is hard, then for any PPT sspRLWE solver $\mathcal{A}$, the pair $(a, b = ar + e) \in R_q \times R_q$ is already indistinguishable from a uniformly random tuple over $R_q \times R_q$, which indicates that $\mathcal{A}$ should not have noticeable advantage over a trivial solver just making a random guess on $ve \bmod 2$ according to the distribution $\chi' = v\chi_e \bmod 2$.

3 Specification of the NEV Encryption Family

In this section, we first give a provably one-way secure PKE scheme **NEV-PKE-OW** from the decisional NTRU and sspRLWE assumptions, which can be easily transformed into a CPA-secure KEM by using a hash function modeled as a random oracle. Then, we transform **NEV-PKE-OW** into a CCA-secure KEM scheme called **NEV-KEM-CCA** by using the Fujisaki-Okamoto (FO) transformation [33]. We also present a CPA-secure PKE **NEV-PKE-CPA** and a CCA-secure PKE **NEV-PKE-CCA** by applying a standard “KEM + DEM” paradigm to **NEV-PKE-OW** together with a one-time pad encryption. We begin by describing our vector decoding technique.

3.1 Randomized Plaintext Encoding and Decoding

Let n be a power of 2, and q be a prime. Let $R = \mathbb{Z}[x]/(x^n + 1)$ and $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Let $\mathcal{M} = \{0, 1\}^{n/k}$ be the plaintext space. Let $v = (1 - x^{n/k}) \in R_q^*$ be a ring element, whose inverse is $v^{-1} = \frac{q+1}{2}(1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*$. Let B_η be the binomial distribution with parameter $\eta \in \mathbb{Z}$. We define a pair of encoding and decoding algorithm (**Pt2noise**, **Noise2Pt**) as follows:

- **Pt2noise**(M, η) : given a plaintext $M \in \{0, 1\}^{n/k}$ and an integer η as inputs, first randomly choose $s \leftarrow \{0, 1\}^{2n\eta - n/k}$, and parse $s = (s_0, \dots, s_{2k\eta-2})$ as $(2k\eta - 1)$ blocks of n/k bits (i.e., $s_i \in \{0, 1\}^{n/k}$ for all $i \in [2k\eta - 1]$). Then, set $s_{2k\eta-1} = M \oplus (\oplus_{i=0}^{2k\eta-2} s_i) \in \{0, 1\}^{n/k}$, arrange the bit string $(s_0, \dots, s_{2k\eta-1}) \in \{0, 1\}^{2n\eta}$ as a bit array with 2η rows and n columns, and use the 2η bits in the i -th column as the randomness to sample the i -th coefficient of a polynomial $m \in R_q$ from B_η , as depicted in Fig. 4. Finally, return $m = m_0 + m_1x + \dots + m_{n-1}x^{n-1} \in R_q$, where

$$m_{in/k+j} = \sum_{t=0}^{\eta-1} (s_{2i\eta+t,j} - s_{2i\eta+\eta+t,j}) \text{ for } i \in [k], j \in [n/k].$$

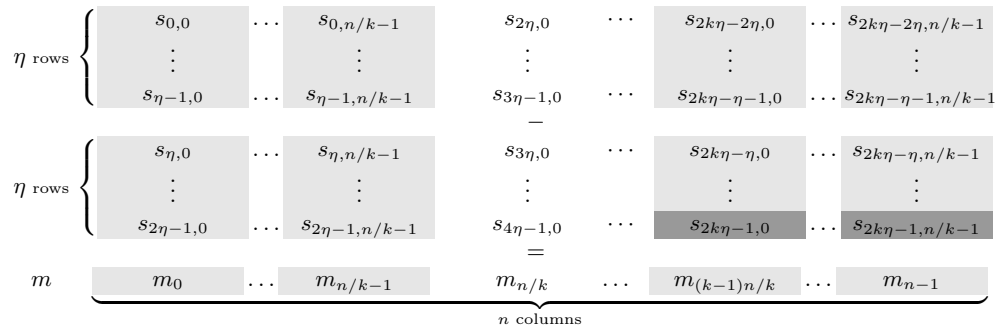


Fig. 4: The bit array for randomized encoding of a plaintext

- **Noise2Pt**(w) : given a polynomial $w = w_0 + w_1x + \dots + w_{n-1}x^{n-1} \in R_q$ as input, first compute $\tilde{w}_i = w_i - \frac{q+1}{2} \bmod^\pm q$ for all $i \in [n]$. Then, compute

$t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i|$ for all $j \in [\ell]$. Finally, set

$$M_j = \begin{cases} 1, & \text{if } t_j < \frac{k \cdot (q-1)}{4}; \\ 0, & \text{otherwise,} \end{cases}$$

and return $M = (M_0, \dots, M_{\ell-1}) \in \{0, 1\}^\ell$.

We have the following lemma for the above two algorithms.

Lemma 3. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v \in R_q^*$ be defined as above. If M is uniformly chosen from $\{0, 1\}^{n/k}$, then the coefficient distribution of $m = \text{Pt2noise}(M, \eta)$ is identical to the binomial distribution B_η . Moreover, if $k\eta < \frac{q}{2}$, then for any $m = \text{Pt2noise}(M, \eta)$ and any polynomial $e = e_0 + e_1x + \dots + e_{n-1}x^{n-1} \in R_q$ satisfying the following condition*

$$\left(\sum_{i=j \bmod n/k} |e_i \bmod^\pm q| \right) < \frac{k \cdot (q-1)}{4} - k \frac{k\eta + 1}{2} \text{ for } i \in [n] \text{ and } j \in [n/k] \quad (1)$$

we always have $\text{Noise2Pt}(v^{-1}m + e) = M$.

Proof. The first claim directly follows from the fact that $(s_0, \dots, s_{2k\eta-2})$ are uniformly chosen from $\{0, 1\}^{2n\eta-n/k}$, and given $(s_0, \dots, s_{2k\eta-2})$, $s_{2k\eta-1}$ is also uniformly distributed over $\{0, 1\}^{n/k}$.

Let $\bar{m}$ be a polynomial whose first n/k coefficients are $M = (M_0, \dots, M_{n/k-1}) \in \{0, 1\}^{n/k}$. We now show that $v^{-1}m = v^{-1}\bar{m} + e' \in R_q$ for some $\|e'\|_\infty \leq \frac{k\eta+1}{2}$. Formally, let $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k})$, and $u = u_0 + u_1x + \dots + u_{n-1}x^{n-1} = \bar{v}m$, we have that $u_{in/k+j} = \sum_{t \leq i} m_{tn/k+j} - \sum_{t > i} m_{tn/k+j}$ for all $i \in [k], j \in [n/k]$. By the assumption that $k\eta < \frac{q}{2}$, we have that $u_{in/k+j} \in [-\frac{q-1}{2}, \frac{q-1}{2}]$ for all $i \in [k], j \in [n/k]$. Moreover, using a routine calculation one can check that $u_{in/k+j} = \sum_{t=0}^{k-1} m_{tn/k+j} = M_j \bmod 2$ for all $i \in [k], j \in [n/k]$ by the definition of m , and that there exists a polynomial e' such that $u = 2e' + \bar{v}\bar{m}$ and $\|e'\|_\infty \leq \frac{k\eta+1}{2}$ by the definition of $\bar{m}$. We immediately have $w = v^{-1}m + e = v^{-1}\bar{m} + e' + e$ using the the fact that $v^{-1} = \frac{q+1}{2}\bar{v}$.

Now, we continue the proof by considering the value of each $M_j \in \{0, 1\}$ for $j \in [n/k]$:

- $M_j = 1$: we have that $w_i = \frac{q+1}{2} + e'_i + e_i$ for all $i = j \bmod n/k$, and that $\tilde{w}_i = w_i - \frac{q+1}{2} = e'_i + e_i \bmod^\pm q$. This means that

$$t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i| = \sum_{i=j \bmod n/k} |e'_i + e_i \bmod^\pm q| < \frac{k \cdot (q-1)}{4},$$

where the inequality holds because $\sum_{i=j \bmod n/k} |e_i \bmod^\pm q| < \frac{k \cdot (q-1)}{4} - k \frac{k\eta+1}{2}$ by assumption and $\|e'\|_\infty \leq \frac{k\eta+1}{2}$ as shown above. This shows that $\text{Noise2Pt}(w)$ will output $M_j = 1$;

- $M_j = 0$: we have that $w_i = e'_i + e_i$ for all $i = j \bmod n/k$, and that $\tilde{w}_i = w_i - \frac{q+1}{2} = e'_i + e_i - \frac{q+1}{2} \bmod^\pm q$. Since we have either $e'_i + e_i = |e'_i + e_i \bmod^\pm q|$ or $e'_i + e_i = q - |e'_i + e_i \bmod^\pm q|$, it is easy to check that $|\tilde{w}_i| \geq \frac{q-1}{2} - |e'_i + e_i \bmod^\pm q|$. This means that

$$t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i| \geq \sum_{i=j \bmod n/k} \left(\frac{q-1}{2} - |e'_i + e_i \bmod^\pm q| \right) > \frac{k \cdot (q-1)}{4},$$

and that $\text{Noise2Pt}(w)$ will output $M_j = 0$. This completes the proof.

Remark 4. Since $v^{-1}m + e = v^{-1}\bar{m} + e + e'$, the condition (1) in Lemma 3 can actually be relaxed to the following condition:

$$\left(\sum_{i=j \bmod n/k} |e_i + e'_i \bmod^\pm q| \right) < \frac{k \cdot (q-1)}{4} \text{ for } i \in [n] \text{ and } j \in [n/k]. \quad (2)$$

3.2 Description of NEV-PKE-OW

Let $n, q, k, \eta \in \mathbb{Z}$ and $v \in R_q^*$ be defined as above. Let χ_f, χ_g, χ_r be three distributions over R . We now give our PKE scheme NEV-PKE-OW, which consists of the following three algorithms (KeyGen, Enc, Dec):

- **NEV-PKE-OW.KeyGen**(κ): given the security parameter κ as inputs, randomly choose $f' \leftarrow \chi_f$ and $g \leftarrow \chi_g$ such that $f = f' + v^{-1}, g \in R_q^*$ are invertible. Then, return the public key and secret key pair $(pk, sk) = (h = g/f, f) \in R_q \times R_q$.
- **NEV-PKE-OW.Enc**(pk, M): given the public key $pk = h \in R_q$ and a plaintext $M \in \{0, 1\}^{n/k}$ as inputs, sample $r \leftarrow \chi_r$ and $m \leftarrow \text{Pt2noise}(M, \eta) \in R_q$. Then, compute and return the ciphertext $c = hr + m$.

In this document, we will frequently use the notation

$$c = \text{NEV-PKE-OW.Enc}(pk, M; \rho),$$

which means that we run the algorithm $\text{NEV-PKE-OW.Enc}(pk, M)$ by using $\rho \in \{0, 1\}^\kappa$ as the random seed to a secure pseudorandom generator (PRG) to generate the random coins to sample $r \leftarrow \chi_r$ and $m \leftarrow \text{Pt2noise}(M, \eta) \in R_q$.

- **NEV-PKE-OW.Dec**(sk, C): given the secret key $sk = f = f' + v^{-1} \in R_q^*$ and a ciphertext $c \in R_q$ as inputs, compute $u = fc$, and $M' = \text{Noise2Pt}(u)$. Finally, return the plaintext $M' \in \{0, 1\}^{n/k}$.

Correctness and Decryption failures. Let $\bar{m}$ be a polynomial whose first n/k coefficients are $M = (M_0, \dots, M_{n/k-1}) \in \{0, 1\}^{n/k}$. Then, we have $v^{-1}m = v^{-1}\bar{m} + e' \in R_q$ for some $\|e'\|_\infty \leq \frac{k\eta+1}{2}$. Since we have the following decryption formula

$$w = fc = gr + (f' + v^{-1})m = gr + f'm + v^{-1}m = \underbrace{gr + f'm + e'}_{=e''} + v^{-1}\bar{m},$$

the decryption is correct as long as we set the parameters such that $e'' = (e''_0, \dots, e''_{n-1})$ satisfies

$$\left(\sum_{i=j \bmod n/k} |e''_i \bmod^{\pm} q| \right) < \frac{k \cdot (q-1)}{4} \text{ for all } i \in [n] \text{ and } j \in [n/k].$$

by the condition (2) in Lemma 3.

For our concrete choices of parameters, we numerically compute a union bound on the probability of decryption failures by using an approach adapted from [5,13]. Formally, we are more interested in the case of k, n being powers of 2 and χ_f, χ_g, χ_r being centrally symmetric distributions (namely, $-\chi = \chi$). By $S = \mathbb{Z}[Y]/(Y^k + 1)$ we denote the $2k$ -th cyclotomic ring, having rank k over $\mathbb{Z}$. Since $k|n$, we can view S as a subring of $R = \mathbb{Z}[X]/(X^n + 1)$ by identifying $Y = X^{n/k}$. In particular, an element $a \in R$ can be identified to a vector $(\hat{a}_0, \dots, \hat{a}_{n/k-1}) \in S^{n/k}$ such that

$$a(X) = \hat{a}_0(X^{n/k}) + X\hat{a}_1(X^{n/k}) + \dots + X^{n/k-1}\hat{a}_{n/k-1}(X^{n/k}).$$

The coefficients of $\hat{a}_j \in S$ exactly consists of $a_j, a_{j+n/k}, \dots, a_{j+(k-1)n/k}$ of $a \in R$. Using the above notations, the condition for correct decryption can be restated as

$$\|\hat{e}''_j\|_1 < \frac{k \cdot (q-1)}{4} \text{ for all } j \in [n/k].$$

Furthermore, let $y \in S$ be a polynomial ranges over all polynomials with ± 1 coefficients. Then, we have $\|\hat{e}''_j\|_1 = \max_y \langle \hat{e}''_j, y \rangle$ and

$$\Pr \left[\|\hat{e}''_j\|_1 \geq \frac{k \cdot (q-1)}{4} \right] = \Pr \left[\max_y \langle \hat{e}''_j, y \rangle \geq \frac{k \cdot (q-1)}{4} \right].$$

By using a union bound, we have

$$\Pr \left[\|\hat{e}''_j\|_1 \geq \frac{k \cdot (q-1)}{4} \right] \leq \sum_y \Pr \left[\langle \hat{e}''_j, y \rangle \geq \frac{k \cdot (q-1)}{4} \right].$$

Since $e'' = gr + f'e + e'$, we have

$$\hat{e}''_j = \sum_{i \in [n/k]} Y^{z_j} (\hat{g}_i \hat{r}_{(j-i) \bmod n/k} + \hat{f}'_i \hat{e}_{(j-i) \bmod n/k}) + \hat{e}'_j, \quad (3)$$

where the powers z_j are integers. Since all $\hat{g}_i, \hat{f}'_i, \hat{r}_i, \hat{e}_i$ have coefficients independently chosen from centrally symmetric distribution (which are invariant under multiplication by Y^{z_i} and by -1), we can numerically compute the probability

$$\begin{aligned} p_j &= \sum_y \Pr \left[\langle \hat{e}''_j, y \rangle \geq \frac{k \cdot (q-1)}{4} \right] \\ &= \sum_y \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}'_i \hat{e}_i) + \hat{e}'_j, y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right] \end{aligned} \quad (4)$$

by randomly choosing $\hat{g}_i \leftarrow \chi_g, \hat{f}_i' \leftarrow \chi_f, \hat{r}_i \leftarrow \chi_r, \hat{e}_i \leftarrow B_\eta$. To address worst-case correctness across all possible messages (Definition 1), our numerical computation actually considers a distribution for $\hat{e}_i = (\hat{e}_{i,0}, \hat{e}_{i,1}, \hat{e}_{i,2}, \hat{e}_{i,3})$, where the first three coefficients are independently drawn from B_η . The last coefficient $\hat{e}_{i,3}$, as defined by our message encoding algorithm $\text{Pt2noise}(M, \eta)$ (see Fig. 4), is drawn from either

$$\{e - (e \bmod 2) : e \leftarrow B_{\eta-1} + \mathcal{U}(\{0, 1\})\} \quad \text{or} \quad (5)$$

$$\{e - ((e + 1) \bmod 2) : e \leftarrow B_{\eta-1} + \mathcal{U}(\{0, 1\})\},$$

depending on the message bit $M_i \in \{0, 1\}$ and the first three coefficients, with $B_0 = 0$. Since the two distributions have the same zero expectation (and, except for $\eta = 1$, the same variance $\eta/2$), the estimated failure probability obtained by our numerical computation is essentially independent of the message bits and matches the case $\hat{e}_i \leftarrow B_\eta$ —which is equivalent to assuming M_i is uniformly random. This implies that our scheme is indeed robust against decryption failure attacks that use maliciously crafted messages.

Finally, by taking a union bound we can obtain an upper bound on the probability of decryption failures

$$1 - \Pr \left[\|\hat{e}_j''\|_1 < \frac{k \cdot (q - 1)}{4} \text{ for all } j \in [n/k] \right] \leq n/k \cdot p_j. \quad (6)$$

It is worth to note that the above analysis does not rely on extra assumption on the coefficients distribution of the final error term e'' , and gives a very conservative upper bound on the probability of decryption failures. In particular, it does not need the identically and independently distributed assumption on the coefficients of e'' , which is usually used to simplify the analysis of lattice encryption schemes but was disproved both theoretically and practically in [30]. In fact, our numerical computation shows that, under the independence assumption, even for our simple vector decoding that relies solely on the correlation among four coefficients, the estimated failure probability can be reduced by a factor of up to 2^{16} for our concrete choice of parameters given in Sec. 4.

Remark 5 (Supporting Ciphertext Compression). Since $h \in R_q^*$, the ciphertext in $\text{NEV-PKE-OW.Enc}(pk, M)$ can be equivalently written as $c' = h^{-1}m + r$. In this form, decryption proceeds by first multiplying c' by g , yielding the same decryption formula as before. This conceptual change does not affect the security or correctness analysis. A convenient feature of this formulation is that it naturally supports existing rounding-like compression techniques for ciphertexts. Such techniques essentially introduce an additional error term e' into the ciphertext, which can be directly absorbed into r , i.e., $c' = h^{-1}m + r + e' = h^{-1}m + r'$. This property also extends to other schemes in the NEV family.

However, rounding-like techniques may introduce significant uncertainties into rigorous security analysis (e.g., rounding-based assumptions [55,56]) and correctness analysis (e.g., heuristic independence assumptions [13]). They may also raise patent issues. Therefore, we believe that the non-compression version is a better

choice for standardization. Nevertheless, to illustrate the efficiency gains achievable for NEV under these techniques, we also provide concrete parameters for the compressed variant based on rounding-like assumptions (namely, sspRLWR), denoted as **NEV-C1***, **NEV-C2***, and **NEV-C3*** (see Sec. 4). These are intended for those who are comfortable with the aforementioned weaknesses.

3.3 Description of **NEV-KEM-CPA**

Let $n, q, k, \eta \in \mathbb{Z}$ and $v \in R_q^*$ be defined as above. Let χ_f, χ_g, χ_r be three distributions over R . Let $H : \{0, 1\}^{n/k} \rightarrow \{0, 1\}^\kappa$ be a hash function modeled as a random oracle. We now give our KEM scheme **NEV-KEM-CPA**, which consists of the following three algorithms **NEV-KEM-CPA** = (KeyGen, Encap, Decap):

- **NEV-KEM-CPA.KeyGen**(κ): given the security parameter κ as inputs, randomly choose $f' \leftarrow \chi_f$ and $g \leftarrow \chi_g$ such that $f = f' + v^{-1}, g \in R_q^*$ are invertible. Then, return the public key and secret key pair $(pk, sk) = (h = g/f, f) \in R_q \times R_q$.
- **NEV-KEM-CPA.Encap**(pk): given the public key $pk = h \in R_q$ as input, sample $r \leftarrow \chi_r$, a plaintext $M \leftarrow \{0, 1\}^{n/k}$ and $m \leftarrow \text{Pt2noise}(M, \eta) \in R_q$. Then, compute $c = hr + m$ and $K = H(M)$. Finally, return the ciphertext and session key pair (c, K) .
- **NEV-KEM-CPA.Decap**(sk, C): given the secret key $sk = f = f' + v^{-1} \in R_q^*$ and a ciphertext $c \in R_q$ as inputs, compute $w = fc$, and $M' = \text{Noise2Pt}(w)$. Finally, return the session key $K = H(M')$.

Remark 6. The encryption algorithm can be simplified as follows: first, directly sample a noise vector $m \leftarrow B_\eta$; then, derive a pre-session key $M = \bar{K}$ from the first n/k coefficients of $\bar{v}m \bmod 2$. By Lemma 3, this procedure is equivalent to first choosing a prekey $M = \bar{K} \in \{0, 1\}^{n/k}$ uniformly at random, and then computing $m = \text{Pt2noise}(M, \eta)$.

3.4 Description of **NEV-PKE-CPA**

Let **NEV-PKE-OW** = (KeyGen, Enc, Dec) be defined in the above subsection. Let $H : \{0, 1\}^{n/k} \rightarrow \{0, 1\}^\ell$ be a hash function modeled as a random oracle. We now transform **NEV-PKE-OW** into an IND-CPA secure PKE **NEV-PKE-CPA** = (KeyGen, Enc, Dec) following a standard “KEM + DEM” paradigm.

- **NEV-PKE-CPA.KeyGen**(κ): given the security parameter κ as input, randomly choose $f' \leftarrow \chi_f$ and $g \leftarrow \chi_g$ such that $f = f' + v^{-1}, g \in R_q^*$ are invertible. Then, return the public key and secret key pair $(pk, sk) = (h = g/f, f) \in R_q \times R_q$.
- **NEV-PKE-CPA.Enc**(pk, μ): given the public key $pk = h \in R_q$ and a plaintext $\mu \in \{0, 1\}^\ell$ as inputs, sample $r \leftarrow \chi_r$, $M \leftarrow \{0, 1\}^{n/k}$ and $m \leftarrow \text{Pt2noise}(M, \eta) \in R_q$. Then, compute $c = hr + m$ and $c_2 = H(M) \oplus \mu$. Finally, return the ciphertext $c = (c_1, c_2)$.
- **NEV-PKE-CPA.Dec**(sk, C): given the secret key $sk = f = f' + v^{-1} \in R_q^*$ and a ciphertext $c \in R_q$ as inputs, compute $u = fc$, and $M' = \text{Noise2Pt}(u)$. Finally, return the plaintext $\mu = H(M') \oplus c_2 \in \{0, 1\}^\ell$.

3.5 Description of NEV-KEM-CCA

Let $\text{NEV-PKE-OW} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be defined in the above subsection. Let $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$, $H_2 : \{0, 1\}^{n/k+\kappa} \rightarrow \{0, 1\}^\kappa \times \{0, 1\}^\kappa$ be two hash functions, which will be modeled as random oracles. We now transform NEV-PKE-OW into an IND-CCA secure KEM $\text{NEV-KEM-CCA} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ following the generic FO-transformation.

- $\text{NEV-KEM-CCA.KeyGen}(\kappa)$: given the security parameter κ as input, compute $(pk', sk') = \text{NEV-PKE-OW.KeyGen}(1^\kappa)$. Then, return the public key $pk = pk'$, and secret key $sk = (sk', pk, H_1(pk))$.
- $\text{NEV-KEM-CCA.Encap}(pk)$: given the public key pk as input, randomly choose $M \leftarrow \{0, 1\}^{n/k}$, and compute

$$(K, \rho) = H_2(M, H_1(pk)), \quad c = \text{NEV-PKE-OW.Enc}(pk, M; \rho).$$

Then, return the ciphertext and session key pair (c, K) .

- $\text{NEV-KEM-CCA.Decap}(sk, c)$: given the secret key $sk = (sk', pk, H_1(pk))$ and a ciphertext c as inputs, compute $M' = \text{NEV-PKE-OW.Dec}(sk', c)$, and $(K', \rho') = H_2(M', H_1(pk))$, $c' = \text{NEV-PKE-OW.Enc}(pk, M'; \rho')$. If $c' = c$, return K' , otherwise, return $\perp$.

Since NEV-KEM-CCA is obtained by a standard application of the FO transformation (with explicit rejection) to NEV-PKE-OW , the correctness of NEV-KEM-CCA directly follows from that of NEV-PKE-OW .

3.6 Description of NEV-PKE-CCA

Let $\text{NEV-PKE-OW} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be defined in the above subsection. Let $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$, $H_2 : \{0, 1\}^{n/k+\kappa+\ell} \rightarrow \{0, 1\}^\kappa$ and $H_3 : \{0, 1\}^{n/k} \rightarrow \{0, 1\}^\ell$ be three hash functions, which will be modeled as random oracles in the security proof. We now transform NEV-PKE-OW into a IND-CCA secure PKE $\text{NEV-PKE-CCA} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ following the generic FO-transformation.

- $\text{NEV-PKE-CCA.KeyGen}(\kappa)$: given the security parameter κ as input, compute $(pk', sk') = \text{NEV-PKE-OW.KeyGen}(1^\kappa)$. Then, return the public key $pk = pk'$, and secret key $sk = (sk', pk, H_1(pk))$.
- $\text{NEV-PKE-CCA.Enc}(pk, \mu)$: given the public key pk and a message $\mu \in \{0, 1\}^\ell$ as inputs, randomly choose $M \leftarrow \{0, 1\}^{n/k}$, and compute

$$d = H_3(M) \oplus \mu, \quad \rho = H_2(M, H_1(pk), d), \quad \text{and } c = \text{NEV-PKE-OW.Enc}(pk, M; \rho).$$

Then, return the ciphertext (c, d) .

- $\text{NEV-PKE-CCA.Dec}(sk, c, d)$: given the secret key $sk = (sk', pk, H_1(pk))$ and a ciphertext $c = (c, d)$ as inputs, compute $M = \text{NEV-PKE-OW.Dec}(sk', c)$, $\rho' = H_2(M, H_1(pk), d)$, and $c' = \text{NEV-PKE-OW.Enc}(pk, M; \rho')$. If $c' = c$, return $d \oplus H_3(M)$, otherwise, return $\perp$.

4 Concrete Parameters

In Table 2, we select nine parameter sets, divided into three groups of three: “NEV-C-1/2/3*” (compact), “NEV-R-1/2/3” (recommended) and “NEV-D-1/2/3” (DFR-oriented), each targeted $\geq 128/256/512$ classical (and $\geq 80/128/256$ quantum) bits security. The 4th and 5th columns give the distributions used for key generation and encryption. For the parameter sets NEV-C-1/2/3*, we incorporate the conceptual modification described in Remark 5: the ciphertext is compressed from R_{769} to R_{256} via

$$c' = \lfloor h^{-1}m \cdot \frac{256}{769} \rfloor_{256}.$$

The resulting distribution $\mathcal{T}_{1/3}^*$, obtained by rounding from $\mathbb{Z}_{769}$ to $\mathbb{Z}_{256}$, closely approximates $\mathcal{T}_{1/3}$. This compression reduces the ciphertext size by roughly 16.7%, but at the cost of introducing a rounding-based hardness assumption and requiring an implicit rejection strategy for the security proof. As an alternative, one may forgo compression entirely and instead set $c' = h^{-1}m + r$ with $r \leftarrow \mathcal{T}_{1/3}$, thus relying on more standard hardness assumptions for both security and correctness. The uncompressed variants, denoted “NEV-C-1/2/3”, differ from the compressed “NEV-C-1/2/3*” sets only in using $\mathcal{T}_{1/3}$ instead of the rounded $\mathcal{T}_{1/3}^*$. Their ciphertext size matches the public key size, and their estimated security and decryption failure rates remain essentially the same as those of the compressed versions.

The 6th column gives the corresponding sizes of public key (PK) and ciphertext (CT) in bytes. For PKEs NEV-PKE-CPA and NEV-PKE-CCA, the figures corresponding to CT only means the ciphertext overhead, and the real ciphertext sizes should add the length of the plaintext. The 7th column presents the decryption failure probability, which is computed by using a python script following the analysis given in Sec 3.2.⁸ We set our decryption failure probabilities $< 2^{-128}$ with some margin so that it is infeasible to obtain a single decryption failure using at most 2^{80} decryption queries (see the directional failure boosting attacks [24] and Sec. 8.5). The 8-10th columns give the concrete security estimates for the hardness of NTRU and RLWE using the core-SVP model [5], where C/Q denotes the lowest classical/quantum security bits. In the last column, we also present the estimated security bits in a (refined) MATZOV model (see MATZOV[†] in Sec. 8.1) recently shown to have lowered the concrete security estimates of Kyber below the NIST requirements [17]. Given the recent progress of dual-hybrid LWE attacks, we believe it is prudent to consider the (refined) MATZOV model in concrete security estimates as also commented in [37]. Table 2 shows that the MATZOV hybrid dual attack largely determines the hardness of the RLWE problem. Although this type of dual attack is not directly applicable to NTRU problems, we believe that selecting NTRU parameters solely based on this assumption would not be a conservative approach. Therefore, we recommend using the NEV-R-1/2/3 parameter

⁸ For the NEV-C-1/2/3 parameter sets, both the security analysis and the decryption failure computation rely on the same heuristic assumption commonly adopted in rounding-based schemes like Kyber [13]: namely, that each coefficient of the rounding error is randomly and independently distributed. This assumption is distinct from the independence-of-coefficient-failures heuristic, which assumes that decryption failures are identically and independently distributed across coefficients—an assumption that has been disproven both theoretically and empirically [29]. In contrast, the security and correctness analyses for all other parameter sets are completely heuristic-free.

sets to ensure that NTRU remains hard even if some variant of the MATZOV hybrid dual attack were to affect the NTRU setting.

Table 2: Concrete Parameters Sets for NEV

Parameters	n	q	Key Dist.	Enc Dist.	Sizes	Dec.	NTRU Hardness	RLWE Hardness (C,Q)		
			(χ_f, χ_g)	(χ_r, χ_e)	(PK, CT)	Failure	Primal (C,Q)	Primal	Dual	MATZOV [†]
NEV-C1*	512	769	(B_1, B_2)	$(\mathcal{T}_{1/3}^*, B_2)$	(615, 512)	2^{-160}	(132, 116)	(135, 118)	(133, 117)	(129, 103)
NEV-R1		1409	(B_3, B_3)	(B_3, B_3)	(672, 672)	2^{-133}	(135, 119)	(135, 119)	(133, 117)	(134, 107)
NEV-D1		3329	(B_7, B_7)	(B_7, B_7)	(768, 768)	2^{-136}	(135, 118)	(135, 118)	(133, 117)	(134, 111)
NEV-C2*	1024	769	$(B_1, \mathcal{T}_{1/8})$	$(\mathcal{T}_{1/3}^*, B_2)$	(1229, 1024)	2^{-162}	(266, 234)	(300, 264)	(297, 261)	(268, 217)
NEV-R2		1409	(B_2, B_2)	(B_2, B_2)	(1344, 1344)	2^{-158}	(284, 250)	(284, 250)	(282, 248)	(261, 211)
NEV-D2		3329	(B_4, B_4)	(B_4, B_4)	(1536, 1536)	2^{-217}	(280, 246)	(280, 246)	(277, 244)	(264, 218)
NEV-C3*	2048	769	$(\mathcal{T}_{1/8}, \mathcal{T}_{1/8})$	$(\mathcal{T}_{1/3}^*, B_1)$	(2458, 2048)	2^{-163}	(546, 480)	(610, 537)	(607, 534)	(521, 410)
NEV-R3		1409	$(\mathcal{T}_{1/3}, \mathcal{T}_{1/3})$	$(\mathcal{T}_{1/3}, B_2)$	(2688, 2688)	2^{-147}	(583, 513)	(596, 525)	(592, 521)	(526, 424)
NEV-D3		3329	(B_2, B_3)	(B_2, B_3)	(3072, 3072)	2^{-301}	(572, 503)	(572, 503)	(568, 499)	(516, 420)

5 Implementation Details

We have implemented our schemes on both x86 and ARM platforms. For each, we provide a reference C implementation and a vector-optimized version (AVX2 for x86, NEON for ARM). The following details focus on the implementation choices that most affect performance, specifically for our recommended parameter sets (NEV-R-1/2/3) on x86.

5.1 Partial NTT Multiplication

One costly arithmetic operation in NEV is to do polynomial multiplication in R_q . Since the use of small modulus, we cannot apply full NTT multiplications in $R_q = \mathbb{Z}[x]/(x^n + 1)$. But because $q - 1 \bmod 128 = 1$ for $q = 1409$, we can still speedup polynomial multiplications by first splitting the polynomials in R_q to a set of sub-polynomials in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$ and then realize a single polynomial multiplication in R_q by using a number of polynomial multiplications in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$, which in turn can be done efficiently using full NTT multiplications.⁹ Taking $n = 512$ as an example, by letting $y = x^4$ we can split any two polynomials $a, b \in R_q = \mathbb{Z}[x]/(x^{512} + 1)$ as follows:

$$\begin{aligned} a(x) &= a_0(y) + xa_1(y) + x^2a_2(y) + x^3a_3(y) + x^4a_4(y) + x^5a_5(y) + x^6a_6(y) + x^7a_7(y), \\ b(x) &= b_0(y) + xb_1(y) + x^2b_2(y) + x^3b_3(y) + x^4b_4(y) + x^5b_5(y) + x^6b_6(y) + x^7b_7(y), \end{aligned}$$

where all the a_i 's and b_i 's are polynomials in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$. Since multiplications between a_i 's and b_j 's can be done using full NTT multiplications in R'_q , we can realize the multiplication between $a(x)$ and $b(x)$ by roughly using 64 NTT

⁹ For $q \in \{769, 3329\}$, we have $q \bmod 256 = 1$, and the ring $R'_q = \mathbb{Z}_q[y]/(y^{128} + 1)$ can be used to speed up multiplications.

multiplications in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$. We can save more NTT multiplications in R'_q by using the Karatsuba method as observed in [69].

In the following, we fix $q = 1409$ to describe the implementation details. To facilitate the above polynomial multiplications, we directly represent each polynomial in $R_q = \mathbb{Z}[x]/(x^n + 1)$ by simply concatenating the coefficient vectors of its split sub-polynomials, which are almost for free when all the coefficients are identically chosen from the same distribution. Moreover, we will keep the split sub-polynomials for the public key, secret key and ciphertext in their NTT forms to save some forward and inverse NTT operations in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$.

Let $\zeta \in \mathbb{Z}_q$ is a primitive 128-th root of unity modulo $q = 1409$. Define $\text{BitRev}_6(k)$ to be the integer represented by bit-reversing the unsigned 6-bit value that corresponds to the input integer $k \in \{0, \dots, 63\}$. Specifically, if $k = k_0 + 2k_1 + 4k_2 + \dots + 32k_5$ with $k_i \in \{0, 1\}$, then $\text{BitRev}_6(k) = k_5 + 2k_4 + 4k_3 + \dots + 32k_0$. The NTT and NTT^{-1} algorithms are defined in Algorithm 1 and 2.

Algorithm 1: NTT(f)

Input: array $f \in \mathbb{Z}_q^n$
Output: array $\hat{f} \in \mathbb{Z}_q^n$

```

1  $\hat{f} := f$ 
2 for  $i := 0; i < n; i := i + 64$  do
3    $k := 0$ 
4   for  $len := 32; len > 1; len := len/2$  do
5     for  $start := 0; start < 64; start := start + len$  do
6        $zeta := \zeta^{\text{BitRev}_6(k)} \bmod q$ 
7        $k := k + 1$ 
8       for  $j := start; j < start + len; j ++$  do
9          $t := zeta \cdot \hat{f}[i + j + len]$ 
10         $\hat{f}[i + j + len] := \hat{f}[i + j] - t$ 
11         $\hat{f}[i + j] := \hat{f}[i + j] + t$ 
12      end
13    end
14  end
15 end
16 return  $\hat{f}$ 

```

Algorithm 2: $\text{NTT}^{-1}(\hat{f})$

Input: array $\hat{f} \in \mathbb{Z}_q^n$
Output: array $f \in \mathbb{Z}_q^n$

```

1  $f := \hat{f}$ 
2 for  $i := 0; i < n; i := i + 64$  do
3    $k := 63$ 
4   for  $len := 1; len \leq 64; len := 2 \cdot len$  do
5     for  $start := 0; start < 64; start := start + len$  do
6        $zeta := \zeta^{\text{BitRev}_6(k)} \bmod q$ 
7        $k := k - 1$ 
8       for  $j := start; j < start + len; j ++$  do
9          $t := f[i + j]$ 
10         $f[i + j] := t + f[i + j + len]$ 
11         $f[i + j + len] := zeta \cdot (t - f[i + j + len])$ 
12      end
13    end
14  end
15 end
16  $f := f \cdot 1387 \bmod q$ ; // multiply every entry by  $1387 \equiv 64^{-1} \bmod q$ 
17 return  $f$ 

```

5.2 Partial NTT Inversion

The other costly arithmetic operation is to do polynomial inversion in R_q to generate the public key. Note that if we can do full NTT multiplications in R_q , this operation can be simply done by using n inversions in $\mathbb{Z}_q$ using the NTT representation. Fortunately, we can still speedup this operation by making full use of partial NTT multiplications given above as shown in [32]. Specifically, given a polynomial $f \in R_q = \mathbb{Z}_q[x]/(x^n + 1)$, by letting $z = x^2$ we can first use Karatsuba with an even/odd split to obtain two sub-polynomials in $\hat{R}_q = \mathbb{Z}_q[z]/(z^{n/2} + 1)$:

$$f(x) = f_0(z) + xf_1(z).$$

Then, the inversion of f in R_q can be done using one polynomial multiplication in R_q and one polynomial inversion in $\hat{R}_q$ because

$$\frac{1}{f(x)} = \frac{f_0(z) - xf_1(z)}{(f_0(z) + xf_1(z))(f_0(z) - xf_1(z))} = \frac{f_0(z) - xf_1(z)}{f_0^2(z) - zf_1^2(z)}.$$

By repeating this process, we can finally reduce the inversion of f to a few polynomial multiplications in R_q and a single polynomial inversion in $R'_q = \mathbb{Z}_q[y]/(y^{64} + 1)$, which in turn can be done using 64 inversions in $\mathbb{Z}_q$.

5.3 Symmetric Primitives

In our default implementations, we use SHA3 for both the hash function and the pseudorandom generator (PRG), as in the NIST ML-KEM standard. Given

an input bit string N and an output length d , the function $\text{KECCAK}[c](N, d)$ is defined as [54], we define four hash functions as follows:

$$\begin{aligned}\text{Hash-128}(M) &= \text{KECCAK}[256](M\|01, 128); \\ \text{Hash-256}(M) &= \text{KECCAK}[512](M\|01, 256) = \text{SHA3-256}(M); \\ \text{Hash-512}(M) &= \text{KECCAK}[1024](M\|01, 512) = \text{SHA3-512}(M). \\ \text{Hash-1024}(M) &= \text{KECCAK}[1024](M\|01, 1024).\end{aligned}$$

Then, we define three key-derivation functions as follows:

$$\begin{aligned}\text{XOF-128}(M, d) &= \text{KECCAK}[256](M\|1111, 128) = \text{SHAKE128}(M, 128); \\ \text{XOF-256}(M, d) &= \text{KECCAK}[512](M\|1111, 256) = \text{SHAKE256}(M, 256); \\ \text{XOF-512}(M, d) &= \text{KECCAK}[1024](M\|1111, 512).\end{aligned}$$

The instantiating of PRG, H_1 , H_2 , and H_3 is listed in Table 3.

Table 3: Instantiating of PRG

	$\text{PRG}(M, d)$	$H_1(M)$	$H_2(M)$	$H_3(M)$
NEV-512	$\text{XOF-128}(M, d)$	$\text{Hash-128}(M)$	$\text{Hash-256}(M)$	$\text{XOF-128}(M, 128)$
NEV-1024	$\text{XOF-256}(M, d)$	$\text{Hash-256}(M)$	$\text{Hash-512}(M)$	$\text{XOF-256}(M, 256)$
NEV-2048	$\text{XOF-512}(M, d)$	$\text{Hash-512}(M)$	$\text{Hash-1024}(M)$	$\text{XOF-512}(M, 512)$

5.4 Multi-target Countermeasure

For the OW-CPA encryption, we follow the strategy of the NIST ML-KEM standard to hash the public key into a prekey $\bar{K}$ and the random coins ρ , aiming at improving the security against multi-target attacks. Since the performance of symmetric primitives is a major bottleneck of our schemes, this countermeasures will significantly reduce the performance.

5.5 Encoding of R_q Elements

We use the function Encode_{16} (Algorithm 7) to store 16 elements in R_q to the byte form. In particular, we encode coefficients by groups of 16 in 168 bits: each 2 coefficients are encoded into a 21-bit integer which bases $q = 1409$ (For $q = 769$, we can pack 5 coefficients in $\mathbb{Z}_{769}$ into a 48-bit integer [67]). For $n = 512$, this will lead to a reduction of 32 bytes in storing a polynomial in R_q . The decoding function Decode_{16} in Algorithm 8 uses a fact that $\lfloor (1524119 * x) / 2^{31} \rfloor = \lfloor x / 1409 \rfloor$ for any x in 2^{21} to recover the coefficients. For any n coefficients, we pack them to bytes in a usual method as shown in Algorithm 9 and 10. Algorithms 3-6 are intermediate procedures for converting between bit strings, byte strings, and integers.

Algorithm 3: BitsToBytes(y)

Input: a bit string y of length α .

Output: a byte string z of length $\lceil \alpha/8 \rceil$.

```

1  $z \in \mathcal{B}^{\lceil \alpha/8 \rceil} := 0^{\lceil \alpha/8 \rceil};$ 
2 for  $i$  from 0 to  $\alpha - 1$  do
3    $z \lfloor i/8 \rfloor := z \lfloor i/8 \rfloor + y[i] \cdot 2^{i \bmod 8};$ 
4 end
5 return  $z$ ;

```

Algorithm 4: BytesToBits(z)

Input: a byte string z of length α .

Output: a bit string y of length 8α .

```

1 for  $i$  from 0 to  $\alpha - 1$  do
2   for  $j$  from 0 to 7 do
3      $y[8i + j] := z[i] \bmod 2;$ 
4      $z[i] := \lfloor z[i]/2 \rfloor;$ 
5   end
6 end
7 return  $y$ ;

```

Algorithm 5: BitsToInteger(y, α)

Input: a positive integer α and a bit string y of length α .

Output: a nonnegative integer x .

```

1  $x := 0;$ 
2 for  $i$  from 1 to  $\alpha$  do
3    $x := 2x + y[\alpha - i];$ 
4 end
5 return  $x$ ;

```

Algorithm 6: IntegerToBits(x, α)

Input: a nonnegative integer x and a positive integer α .

Output: a bit string y of length α .

```

1 for  $i$  from 0 to  $\alpha - 1$  do
2    $y[i] := x \bmod 2;$ 
3    $x := \lfloor x/2 \rfloor;$ 
4 end
5 return  $y$ ;

```

Algorithm 7: $\text{Encode}_{16}(a_0, a_1, \dots, a_{15})$

Input: integer $a_0, a_1, \dots, a_{15} \in \mathbb{Z}_q$

Output: bit array $b \in \{0, 1\}^{168}$

```

1 for  $i$  from 0 to 7 do
2    $t_i \in \mathbb{Z} := a_{2i+1}q + a_{2i}$ ;
3 end
4  $b := \text{IntegerToBits}(t_0, 21) || \dots || \text{IntegerToBits}(t_7, 21)$ ;
5 return  $b$ ;

```

Algorithm 8: $\text{Decode}_{16}(B)$

Input: byte array $B \in \mathcal{B}^{21}$

Output: integer array $(a_0, a_1, \dots, a_{15}) \in \mathbb{Z}_q^{16}$

```

1  $t_0 \in \mathbb{Z} := (B[2] \bmod 32) \cdot 2^{16} + B[1] \cdot 2^8 + B[0]$ ;
2  $t_1 \in \mathbb{Z} := (B[5] \bmod 4) \cdot 2^{19} + B[4] \cdot 2^{11} + B[3] \cdot 2^3 + \lfloor B[2]/32 \rfloor$ ;
3  $t_2 \in \mathbb{Z} := (B[7] \bmod 128) \cdot 2^{14} + B[6] \cdot 2^6 + \lfloor B[5]/4 \rfloor$ ;
4  $t_3 \in \mathbb{Z} := (B[10] \bmod 16) \cdot 2^{17} + B[9] \cdot 2^9 + B[8] \cdot 2 + \lfloor B[7]/128 \rfloor$ ;
5  $t_4 \in \mathbb{Z} := (B[13] \bmod 2) \cdot 2^{20} + B[12] \cdot 2^{12} + B[11] \cdot 2^4 + \lfloor B[10]/16 \rfloor$ ;
6  $t_5 \in \mathbb{Z} := (B[15] \bmod 64) \cdot 2^{15} + B[14] \cdot 2^7 + \lfloor B[13]/2 \rfloor$ ;
7  $t_6 \in \mathbb{Z} := (B[18] \bmod 8) \cdot 2^{18} + B[17] \cdot 2^{10} + B[16] \cdot 2^2 + \lfloor B[15]/64 \rfloor$ ;
8  $t_7 \in \mathbb{Z} := B[20] \cdot 2^{13} + B[19] \cdot 2^5 + \lfloor B[18]/8 \rfloor$ ;
9 for  $i$  from 0 to 7 do
10    $a_{2i+1} := \lfloor t_i \cdot 1524119/2^{31} \rfloor$ ;
11    $a_{2i} := r - a_{2i+1} \cdot 1409$ ;
12 end
13 return  $(a_0, a_1, \dots, a_{15})$ ;

```

Algorithm 9: $\text{PolyToBytes}(a)$

Input: polynomial $a \in R_q$

Output: integer array $B \in \mathcal{B}^{21 \cdot \lfloor n/16 \rfloor + \lceil 11 \cdot (n \bmod 16)/8 \rceil}$

```

1  $i := 0$ ;
2 for  $i < \lfloor n/6 \rfloor$ ;  $i := i + +$  do
3    $b = b || \text{Encode}_{16}(a_{16i}, a_{16i+1}, \dots, a_{16i+15})$ ;
4 end
5 for  $j := 0$ ;  $j < n \bmod 16$ ;  $j := j + +$  do
6    $b = b || \text{IntegerToBits}(a_{16i+j}, 11)$ ;
7 end
8  $B := \text{BitsToBytes}(b)$ ;
9 return  $B$ ;

```

Algorithm 10: PolyFromBytes(a)

Input: integer array $B \in \mathcal{B}^{21 \cdot \lfloor n/16 \rfloor + \lceil 11 \cdot (n \bmod 16)/8 \rceil}$

Output: polynomial $a \in R_q$

```

1  $i := 0$ ;
2 for  $i < \lfloor n/16 \rfloor$ ;  $i := i + 1$  do
3    $(a_{16i}, \dots, a_{16i+15}) := \text{Decode}_{16}(B[21i], \dots, B[21i+20]);$ 
4 end
5  $z := \text{BytesToBits}(B[21i], \dots, B[21i + \lceil 11 \cdot (n \bmod 16)/8 \rceil - 1]);$ 
6  $(a_{16i}, \dots, a_{n-1}) := \text{BitsToInteger}(z, 11);$ 
7 return  $B$ ;
```

5.6 Sampling Algorithms

The algorithms of NEV require two sampling subroutines Binomial Distribution (Algorithm 11) and Ternary Distribution (Algorithm 12-13). Both of these algorithms can be used to convert a stream of uniformly random bytes into a sample from some desired distribution. These algorithms will be invoked with a stream of pseudorandom bytes as the input. Algorithm 14 is used to determine which distribution is adopted according to Table 2.

Algorithm 11: CBD $_{\eta}(B)$

Input: byte array $B \in \mathcal{B}^{\lceil n\eta/4 \rceil}$.

Output: array $f \in \mathbb{Z}_q^n$.

```

1  $b := \text{BytesToBits}_q(B)$ ;
2 for  $i := 0$ ;  $i < n$ ;  $i + 1$  do
3    $x := \sum_{j=0}^{\eta-1} b[2i\eta + j]$ ;
4    $y := \sum_{j=0}^{\eta-1} b[2i\eta + \eta + j]$ ;
5    $f[i] := (x - y) \bmod q$ ;
6 end
7 return  $f$ ;
```

Algorithm 12: Ternary_{1/8}(ρ, N)

Input: byte array $B \in \mathbb{B}^{\lceil n/4 \rceil}$.

Output: array $f \in \mathbb{Z}_q^n$.

```

1  $B := \text{PRG}(\rho || N, \lceil n/4 \rceil + \lceil n/8 \rceil);$ 
2  $f := \text{CBD}_1(B[0], \dots, B[\lceil n/4 \rceil - 1]);$ 
3 for  $i := 0; i < \lceil n/8 \rceil; i++$  do
4    $a := B[\lceil n/4 \rceil + i];$ 
5   for  $i := 0; i < \lceil n/8 \rceil; i++$  do
6      $b := \text{AND}(a \gg j, 1);$ 
7      $f[8i + j] := \text{AND}(f[8i + j], -b);$ 
8   end
9 end
10 return  $f;$ 

```

Algorithm 13: Ternary_{1/3}(ρ, N)

Input: byte array B .

Output: array $f \in \mathbb{Z}_q^n$.

```

1  $\text{ctx} := \text{PRG.Init}();$ 
2  $\text{ctx} := \text{PRG.Absorb}(\text{ctx}, \rho || N);$ 
3 while  $j < n$  do
4    $z := \text{PRG.Squeeze}(\text{ctx}, 1);$ 
5    $r := \text{BytesToInteger}(z, 1);$ 
6   if  $r < 243$  then
7     for  $i := 0; i < 5$  and  $j < n; i++$  do
8        $a := \lfloor r/3 \rfloor;$ 
9        $f[j] := 1 - (r - 3a);$ 
10       $r := a;$ 
11       $j := j + 1;$ 
12    end
13  end
14 end
15 return  $f;$ 

```

Algorithm 14: $\text{SamplePoly}_\chi(\rho, N)$

Input: byte array B .
Output: array $f \in \mathbb{Z}_q^n$.
1 **if** $\chi = B_\eta$ **then**
2 $B := \text{PRG}(\rho || N, \lceil n\eta/4 \rceil)$;
3 $f := \text{CBD}_\eta(B)$;
4 **end**
5 **else if** $\chi = \mathcal{T}_{1/8}$ **then**
6 $f := \text{Ternary}_{1/8}(\rho, N)$;
7 **end**
8 **else if** $\chi = \mathcal{T}_{1/3}$ **then**
9 $f := \text{Ternary}_{1/3}(\rho, N)$;
10 **end**
11 **return** f ;

5.7 The NEV-PKE-OW Component Scheme

This section describes the component scheme NEV-PKE-OW. It is an instantiation of the scheme described in Sec. 3.2. It serves as a collection of subroutines for use in the algorithms of the schemes NEV-KEM-CCA and NEV-PKE-CCA, as described in Sec. 5.10 and Sec 5.11. NEV-PKE-OW consists of four algorithms: NEV-PKE-OW.KeyGen (Algorithm 15), NEV-PKE-OW.Enc_internal (Algorithm 16), NEV-PKE-OW.Enc (Algorithm 17), and NEV-PKE-OW.Dec (Algorithm 18).

Algorithm 15: NEV-PKE-OW.KeyGen()

Output: public key pk , secret key sk .

```

1  $rnd \leftarrow \mathcal{B}^{\kappa/8}$ ;
2  $\sigma := H_1(rnd)$ ;
3  $N := 0$ ;
4  $g := \text{SamplePoly}_{\chi_g}(\sigma, N)$ ;
5  $N := N + 1$ ;
6 if  $g$  is not invertible then
7   go back to step 4;
8 end
9  $f' := \text{SamplePoly}_{\chi_f}(\sigma, N)$ ;
10  $N := N + 1$ ;
11  $f := f' + v^{-1}$ ;
12 if  $f$  is not invertible then
13   go back to step 9;
14 end
15  $\hat{g} := \text{NTT}(g)$ ;
16  $\hat{f} := \text{NTT}(f)$ ;
17  $\hat{f}^{-1} := \text{Inverse}(\hat{f})$ ;
18  $\hat{h} := \hat{g} \circ \hat{f}^{-1}$ ;
19  $pk := \text{PolyToBytes}(\hat{h})$ ;
20  $sk := \text{PolyToBytes}(\hat{f})$ ;
21 return  $(pk, sk)$ ;
```

Algorithm 16: NEV-PKE-OW.Enc_internal(pk, M, ρ)

Input : the public key pk , the message M , the seed ρ .
Output: the ciphertext c .

```

1  $\hat{h} := \text{PolyFromBytes}(pk)$ ;
2  $r := \text{SamplePoly}_{\chi_r}(\rho, 0)$ ;
3  $m := \text{Pt2noise}(M, \eta, \rho, 1)$ ;
4  $\hat{r} := \text{NTT}(r)$ ;
5  $\hat{m} := \text{NTT}(m)$ ;
6  $\hat{v} := \hat{h} \circ \hat{r} + \hat{m}$ ;
7  $c := \text{PolyToBytes}(\hat{v})$ ;
8 return  $c$ ;
```

Algorithm 17: NEV-PKE-OW.Enc(pk, M)

Input : the public key pk , the message M .**Output:** the ciphertext c .

- 1 $\rho \leftarrow \mathcal{B}^{\kappa/8};$
 - 2 $c := \text{NEV-PKE-OW.Enc_internal}(pk, M, \rho);$
 - 3 **return** $c;$
-

Algorithm 18: NEV-PKE-OW.Dec(sk, c)

Input : the ciphertext c , the private key sk .**Output:** the message M' .

- 1 $\hat{f} := \text{PolyFromBytes}(sk);$
 - 2 $\hat{v} := \text{PolyFromBytes}(c);$
 - 3 $\hat{u} = \hat{f} \circ \hat{v};$
 - 4 $u = \text{NTT}^{-1}(\hat{u});$
 - 5 $M' = \text{Noise2Pt}(u);$
 - 6 **return** $M';$
-

5.8 The NEV-KEM-CPA Key-Encapsulation Mechanism

This section describes the Key-Encapsulation Mechanism NEV-KEM-CPA. It is an instantiation of the scheme described in Sec. 3.3. NEV-PKE-OW consists of three algorithms: NEV-KEM-CPA.KeyGen (Algorithm 19), NEV-KEM-CPA.Encap (Algorithm 20), and NEV-KEM-CPA.Decap (Algorithm 21).

Algorithm 19: NEV-KEM-CPA.KeyGen()

Output: public key pk , secret key sk

- 1 $(pk, sk) := \text{NEV-PKE-OW.KeyGen}();$
 - 2 **return** $(pk, sk);$
-

Algorithm 20: NEV-KEM-CPA.Encap(pk)

Input : the public key pk .**Output:** the ciphertext c , the shared secret key K .

- 1 $M := \{0, 1\}^{n/k};$
 - 2 $K := H_3(M);$
 - 3 $c := \text{NEV-PKE-OW.Enc}(pk, M);$
 - 4 **return** $(c, K);$
-

Algorithm 21: NEV-KEM-CPA.Decap(sk, c)

Input : the ciphertext c , the private key sk .

Output: the message M' .

- 1 $M' := \text{NEV-PKE-OW.Dec}(sk, c);$
 - 2 $K := H_3(M');$
 - 3 **return** $K;$
-

5.9 The NEV-PKE-CPA Public-Key Encryption

This section describes the Public-Key Encryption scheme NEV-PKE-CPA. It is an instantiation of the scheme described in Sec. 3.4. NEV-PKE-OW consists of three algorithms: NEV-PKE-CPA.KeyGen (Algorithm 22), NEV-PKE-CPA.Enc (Algorithm 23), and NEV-PKE-CPA.Dec (Algorithm 24).

Algorithm 22: NEV-PKE-CPA.KeyGen()

Output: public key pk , secret key sk

- 1 $(pk, sk) := \text{NEV-PKE-OW.KeyGen}();$
 - 2 **return** $(pk, sk);$
-

Algorithm 23: NEV-PKE-CPA.Enc(pk, μ)

Input : the public key pk , the message μ .

Output: the ciphertext c .

- 1 $(c_1, K) := \text{NEV-KEM-CPA.Encap}(pk);$
 - 2 $c_2 := \mu \oplus K;$
 - 3 **return** $(c_1, c_2);$
-

Algorithm 24: NEV-PKE-CPA.Dec(sk, c)

Input : the ciphertext c , the private key sk .

Output: the message μ' .

- 1 $(c_1, c_2) := c;$
 - 2 $K' := \text{NEV-KEM-CPA.Decap}(sk, c_1);$
 - 3 $\mu' := c_2 \oplus K';$
 - 4 **return** $\mu';$
-

5.10 The NEV-KEM-CCA Key-Encapsulation Mechanism

This section describes the Key-Encapsulation Mechanism NEV-KEM-CCA. It is an instantiation of the scheme described in Sec. 3.5. NEV-KEM-CCA consists of three algorithms: NEV-KEM-CCA.KeyGen (Algorithm 25), NEV-KEM-CCA.Encap (Algorithm 26), and NEV-KEM-CCA.Decap (Algorithm 27).

Algorithm 25: NEV-KEM-CCA.KeyGen()

Output: public key pk , secret key sk

- 1 $(pk', sk') := \text{NEV-PKE-OW.KeyGen}();$
- 2 $pk := pk';$
- 3 $sk := (sk', pk, H_1(pk));$
- 4 **return** $(pk, sk);$

Algorithm 26: NEV-KEM-CCA.Encap(pk)

Input : the public key pk .
Output: the ciphertext c , the shared secret key K .

- 1 $M \leftarrow \{0, 1\}^{n/k};$
- 2 $(K, \rho) := H_2(M || H_1(pk));$
- 3 $c := \text{NEV-PKE-OW.Enc_internal}(pk, M, \rho);$
- 4 **return** $(c, K);$

Algorithm 27: NEV-KEM-CCA.Decap(sk, c)

Input : the ciphertext c , the private key sk .
Output: the message M' .

- 1 $(sk', pk, H_1(pk)) := sk;$
- 2 $M' := \text{NEV-PKE-OW.Dec}(sk', c);$
- 3 $(K', \rho') := H_2(M' || H_1(pk));$
- 4 $c' := \text{NEV-PKE-OW.Enc_internal}(pk, M', \rho');$
- 5 **if** $c' \neq c$ **then**
- 6 **return** $\perp$;
- 7 **end**
- 8 **return** $K';$

5.11 The NEV-PKE-CCA Public-Key Encryption

This section describes the Public-Key Encryption scheme NEV-PKE-CCA. It is an initialization of the scheme described in Sec 3.6. NEV-PKE-CCA consists of three algorithms: NEV-PKE-CCA.KeyGen (Algorithm 28), NEV-PKE-CCA.Enc (Algorithm 29), and NEV-PKE-CCA.Dec (Algorithm 30).

Algorithm 28: NEV-PKE-CCA.KeyGen()

Output: public key pk , secret key sk

- 1 $(pk', sk') := \text{NEV-PKE-OW.KeyGen}();$
- 2 $pk := pk';$
- 3 $sk := (sk', pk, H_1(pk));$
- 4 **return** $(pk, sk);$

Algorithm 29: NEV-PKE-CCA.Enc(pk, μ)

Input : the public key pk , the message $\mu \in \{0, 1\}^\ell$.

Output: the ciphertext c .

```

1  $M \leftarrow \{0, 1\}^{n/k}$ ;
2  $c_2 := H_3(M) \oplus \mu$ ;
3  $\rho := H_2(M || H_1(pk) || c_2)$ ;
4  $c_1 := \text{NEV-PKE-OW.Enc\_internal}(pk, M, \rho)$ ;
5  $c := (c_1, c_2)$ ;
6 return  $c$ ;
```

Algorithm 30: NEV-PKE-CCA.Dec(sk, c)

Input : the ciphertext c , the private key sk .

Output: the message μ' .

```

1  $(sk', pk, H_1(pk)) := sk$ ;
2  $(c_1, c_2) := c$ ;
3  $M' := \text{NEV-PKE-OW.Dec}(sk', c_1)$ ;
4  $\rho' := H_2(M' || H_1(pk) || c_2)$ ;
5  $c'_1 := \text{NEV-PKE-OW.Enc\_internal}(pk, M', \rho')$ ;
6 if  $c'_1 \neq c_1$  then
7   return  $\perp$  ;
8 end
9  $\mu' := c_2 \oplus H_3(M')$ ;
10 return  $\mu'$ ;
```

6 Performance and Comparison

We evaluate the performance of NEV and compare it with Kyber on the following two platforms:

x86 AMD Ryzen 9 7950X3D processor based on the Zen 4 microarchitecture. The CPU frequency was fixed at 4.20 GHz, with Core Performance Boost and Simultaneous Multithreading (SMT) disabled. All benchmarks were executed in a single-threaded configuration. The software environment consisted of Ubuntu 26.04 LTS and GCC 15.2.0. The implementations were compiled using the flags: `-O3 -march=native -fomit-frame-pointer`.

ARM HUAWEI Kirin 9000C Processor based on the AArch64 architecture. The CPU frequency was fixed at 2.487 GHz. The RAM is 8 GiB. The software environment consisted of Kylin Linux Desktop V10 SP1 and GCC 9.3.0. The implementations were compiled using the flags: `-O3 -fomit-frame-pointer -march=native`.

Section 6.1 presents the x86 results (both portable REF in plain C and AVX2 implementation) on the AMD Ryzen 9 7950X3D platform, while Section 6.2 gives the ARM results (portable AArch64 and NEON-optimized) on the HUAWEI Kirin

9000C platform. All performance figures are measured in CPU cycles and averaged over 30,000 runs per function. The Kyber baseline is profiled using the NIST Round 3 submission package¹⁰.

6.1 Performance on x86

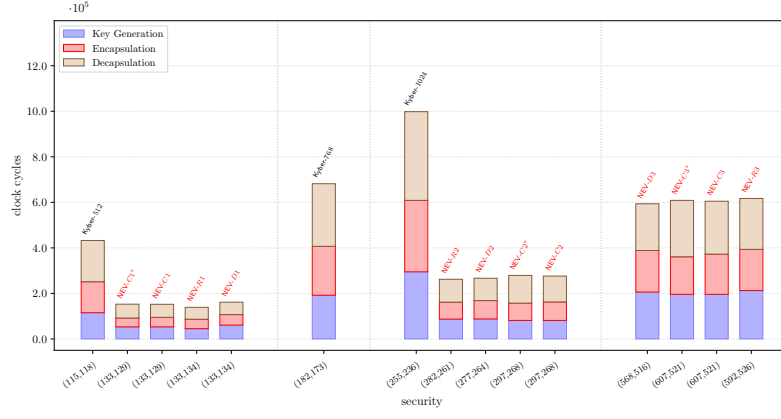
Table 4: The Performance of NEV and Kyber (CPU cycles)

Scheme	Parameter Set	REF			AVX2		
		KeyGen	Enc	Dec	KeyGen	Enc	Dec
NEV	NEV-C1*	53,068	38,828	60,868	16,651	14,005	11,672
	NEV-C2*	81,384	75,835	121,856	29,372	25,975	21,534
	NEV-C3*	196,083	164,863	247,706	84,122	64,826	45,637
	NEV-C1	53,052	41,908	57,250	16,584	17,602	16,058
	NEV-C2	81,331	81,286	113,479	29,341	31,572	28,789
	NEV-C3	196,105	176,368	232,365	84,113	76,546	60,625
	NEV-R1	45,221	41,297	52,400	19,531	17,735	11,786
	NEV-R2	87,220	74,422	100,420	32,472	27,407	18,552
	NEV-R3	212,485	181,037	223,434	87,638	80,440	49,546
	NEV-D1	60,811	46,113	54,529	25,766	23,615	17,761
	NEV-D2	87,875	80,677	98,098	39,506	35,909	24,727
	NEV-D3	205,996	182,630	205,081	101,769	93,416	56,731
Kyber	Kyber-512	115,173	136,182	181,164	20,528	21,456	21,982
	Kyber-768	191,903	214,943	275,047	33,225	33,758	35,788
	Kyber-1024	294,902	313,763	389,691	43,809	43,684	47,021

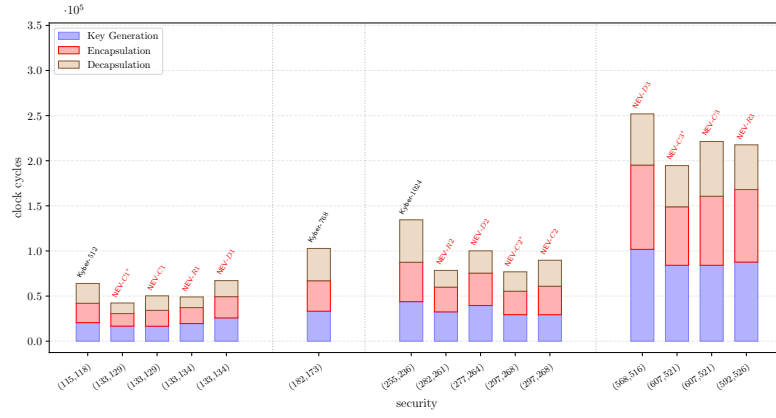
In Table 4, we compare the performance of NEV and Kyber. Under the REF implementation, all NEV parameter sets consistently outperform Kyber’s counterparts at equivalent security levels across KeyGen, Enc, and Dec. Figure 5 compares the total computational costs of NEV and Kyber (security levels from (Core-SVP, MATZOV) on the x-axis), complementing the detailed operation-wise data in Table 4. Subfigures 5a and 5b present the REF and AVX2 implementations. Under REF, all four NEV variants outperform their Kyber equivalents in total cost. Even more notably, the 512-bit NEV sets (C3*, C3, R3, D3) are still faster than Kyber-768, which only offers around 180-bit security. These results also indicate that the AVX2 implementation of NEV still has considerable room for further optimization.

Figure 6 compares the total cost (computation plus communication, with transmission priced at 2000 cycles/byte) of NEV and Kyber. Subfigures 6a and 6b break down the total cost into contributions from KeyGen, Enc, Dec, public-key trans-

¹⁰ <https://github.com/pq-crystals/kyber>



(a) REF implementations



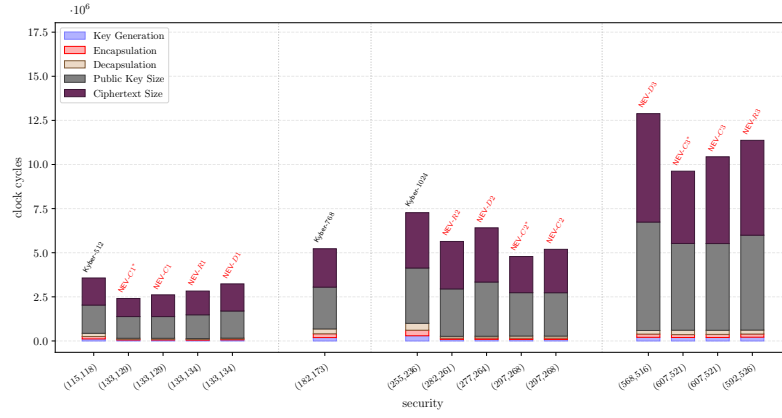
(b) AVX2 implementations

Fig. 5: Benchmarks on x86 platform

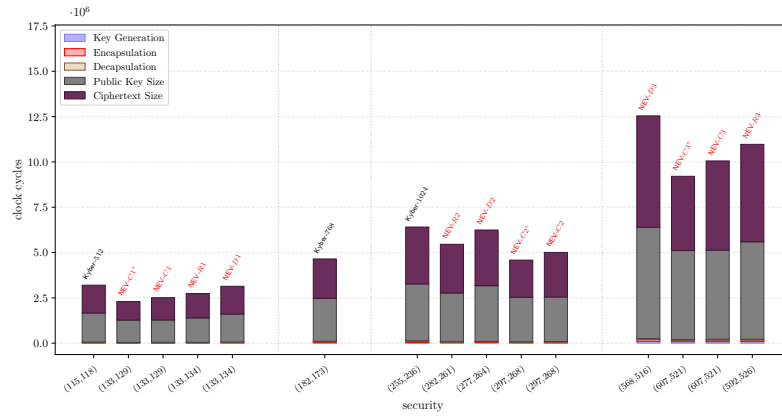
mission, and ciphertext transmission. Under this model, public-key and ciphertext transmission dominate the overall cost.

6.2 Performance on ARM

Table 5 compares NEV and Kyber on the AArch64 platform. Overall, NEV consistently requires fewer CPU cycles than Kyber while offering higher security, across both 128-bit and 256-bit security levels. Table 5 also reports the performance of NEON-optimized NEV on ARM. Compared with the portable AArch64 implementation, the NEON variants achieve substantial speedups across all operations, with the most significant reductions observed in Dec for the larger parameter sets.

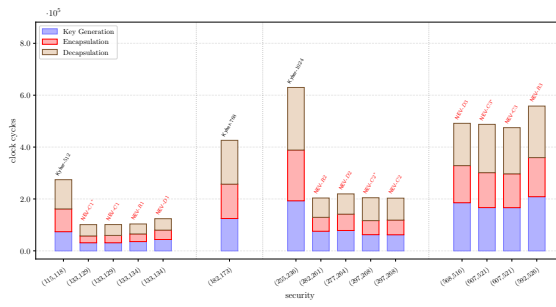


(a) REF implementations

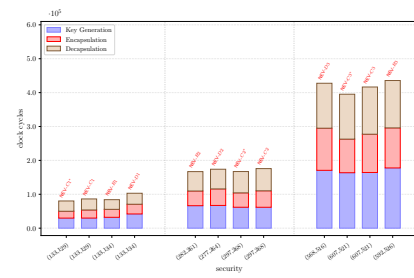


(b) AVX2 implementations

Fig. 6: Benchmarks on x86 platform with 2000 cycles/byte transmission costs



(a) AArch64 implementations

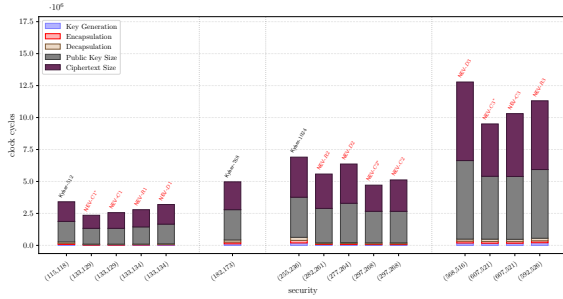


(b) NEON implementations

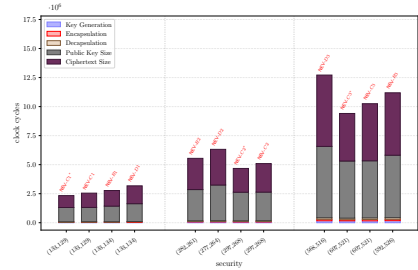
Fig. 7: Benchmarks on ARM platform

Table 5: The Performance of NEV on ARM (CPU cycles)

Scheme	Parameter Set	AArch64			NEON		
		KeyGen	Enc	Dec	KeyGen	Enc	Dec
NEV	NEV-C1*	31,168	26,239	43,692	29,731	20,214	30,308
	NEV-C2*	62,403	54,244	88,069	61,750	42,467	62,912
	NEV-C3*	166,757	134,698	186,073	163,649	99,138	132,436
	NEV-R1	35,731	29,402	38,454	32,049	23,501	28,549
	NEV-R2	75,487	53,637	74,546	66,329	43,361	57,225
	NEV-R3	208,792	150,778	198,489	177,755	118,152	139,779
	NEV-D1	43,728	36,146	44,057	41,953	28,665	32,396
	NEV-D2	78,474	62,897	78,128	67,061	48,798	58,047
	NEV-D3	185,511	143,139	162,849	170,669	124,336	132,588
	NEV-C1	31,162	28,264	41,885	29,829	23,929	32,485
	NEV-C2	61,953	56,588	84,508	61,687	48,595	65,515
	NEV-C3	166,529	130,365	177,989	164,315	112,939	139,301
Kyber	Kyber-512	73,843	87,739	112,718	—	—	—
	Kyber-768	124,635	132,371	168,949	—	—	—
	Kyber-1024	192,916	195,656	241,220	—	—	—



(a) AArch64 implementations



(b) NEON implementations

Fig. 8: Benchmarks on ARM platform with 2000 cycles/byte transmission costs

Figure 7 compares the computational performance of NEV and Kyber on ARM, covering both AArch64 and NEON implementations, with security levels (CoreSVP, MATZOV) on the x-axis. Figure 8 shows the total cost when transmission overhead (estimated at 2000 cycles/byte) is included. Notably, the NEV parameter sets consistently outperform Kyber at equivalent security strengths, while also offering smaller sizes and higher speed.

7 Provable Security

In this section we prove the security of the schemes NEV-PKE-OW, NEV-KEM-CPA, NEV-PKE-CPA, NEV-KEM-CPA, and NEV-KEM-CCA described in Sec. 3. When a security proof is achieved in the random oracle model (ROM), for completeness, we will also give the corresponding security proof in the quantum random oracle model (QROM).

7.1 Security Proof of NEV-PKE-OW

In this section we prove the OW-CPA security of NEV-PKE-OW in the standard model.

Theorem 2. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be a OW-CPA adversary against the NEV-PKE-OW scheme. Then, we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{A}) = \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2).$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 9 to prove this theorem.

Game $\mathbf{G}_0$	Game $\mathbf{G}_1$	Game $\mathbf{G}_2$	Game $\mathbf{G}_3$
1. $f' \leftarrow \chi_f, g \leftarrow \chi_g$ $f = f' + v^{-1} \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = g/f, f)$	1. $f' \leftarrow \chi_f, g \leftarrow \chi_g$ $f = vf' + 1 \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = vg/f, f)$	1. $f' \leftarrow \chi_f, g \leftarrow \chi_g, u \leftarrow R_q^*$ $f = vf' + 1 \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = u, f)$	1. $f' \leftarrow \chi_f, g \leftarrow \chi_g, u \leftarrow R_q^*$ $f = vf' + 1 \in R_q^*$ $g \in R_q^*$ $(pk, sk) = (h = u, f)$
2. $M^* \leftarrow \{0, 1\}^{n/k}$ $r_1^* \leftarrow \{0, 1\}^n, r_2^* \leftarrow \chi_r$ $m^* \leftarrow \text{Pt2noise}(M^*, \eta, r_1)$ $c^* = hr_2^* + m^*$	2. $M^* \leftarrow \{0, 1\}^{n/k}$ $r_1^* \leftarrow \{0, 1\}^n, r_2^* \leftarrow \chi_r$ $m^* \leftarrow \text{Pt2noise}(M^*, \eta, r_1)$ $c^* = hr_2^* + m^*$	2. $M^* \leftarrow \{0, 1\}^{n/k}$ $r_1^* \leftarrow \{0, 1\}^n, r_2^* \leftarrow \chi_r$ $m^* \leftarrow \text{Pt2noise}(M^*, \eta, r_1)$ $c^* = hr_2^* + m^*$	2. $m^* \leftarrow B_\eta$ $r_2^* \leftarrow \chi_r$ $M^* \leftarrow \text{Ext}(\bar{v}m^* \bmod 2)$ $c^* = hr_2^* + m^*$
3. $M' \leftarrow \mathcal{A}(pk, c^*)$	3. $M' \leftarrow \mathcal{A}(pk, c^*)$	3. $M' \leftarrow \mathcal{A}(pk, c^*)$	3. $M' \leftarrow \mathcal{A}(pk, c^*)$
4. return $[M' = M^*]$	4. return $[M' = M^*]$	4. return $[M' = M^*]$	4. return $[M' = M^*]$

Fig. 9: Games $\mathbf{G}_0$ to $\mathbf{G}_3$ in the proof of Theorem 2.

Game $\mathbf{G}_0$: This game is the OW-CPA security game of NEV-PKE-OW with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{A}) = \Pr[1 \leftarrow \mathbf{G}_0]. \quad (7)$$

Game $\mathbf{G}_1$: This game is the same as game $\mathbf{G}_0$, except that f is computed as $vf' + 1$ and h is computed as vg/f . Since $v \in R_q^*$ and the adversary $\mathcal{A}$ only gets $pk(=h)$, this is only a conceptual change, and thus

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \mathbf{G}_1]. \quad (8)$$

Game $\mathbf{G}_2$: This game is the same as game $\mathbf{G}_1$, except that the public key pk is replaced with a uniformly random value over R_q^* .

Let $\mathcal{B}_1$ be a new adversary against the $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ assumption, which is defined as follows: after getting the instance h' (g/f or a uniformly random value over R_q^*), $\mathcal{B}_1$ runs game $\mathbf{G}_1$ from the second step, replaces the public key pk with vh' , and finally outputs $\llbracket M' = M^* \rrbracket$. Obviously, $\mathcal{B}_1$ runs game $\mathbf{G}_1$ when $h' = g/f$, and runs game $\mathbf{G}_2$ when h' is a uniformly random value over R_q^* . The running time of $\mathcal{B}_1$ is about that of $\mathcal{A}$ and

$$|\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| = \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1). \quad (9)$$

Game $\mathbf{G}_3$: This game is the same as game $\mathbf{G}_2$, except that the m^* is first sampled according to B_η and then M^* is computed as $\text{Ext}(\bar{v}m^* \bmod 2)$. Here $\text{Ext} : R_2 \rightarrow \{0, 1\}^{n/k}$ is a function that returns the first n/k coefficients of its input.

According to Lemma 3, the (m^*, M^*) of game $\mathbf{G}_2$ has the same distribution as that of game $\mathbf{G}_3$, and hence we have

$$\Pr[1 \leftarrow \mathbf{G}_2] = \Pr[1 \leftarrow \mathbf{G}_3]. \quad (10)$$

Note that the element v we employ satisfies that $\bar{v}m^* \bmod 2$ is essentially a polynomial with coefficients consisting of k copies of $\text{Ext}(\bar{v}m^* \bmod 2)$. Hence we can directly derive $\bar{v}m^* \bmod 2$ after getting $\text{Ext}(\bar{v}m^* \bmod 2)$. At this point, we define a new adversary $\mathcal{B}_2$ against the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ assumption as follows: after getting the instance $(h, hr_2^* + m^*)$, $\mathcal{B}_2$ runs game $\mathbf{G}_3$ from the third step, and generates its guess for $\bar{v}m^* \bmod 2$ using the output M' from $\mathcal{A}$. Now, we have

$$\Pr[1 \leftarrow \mathbf{G}_3] = \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2). \quad (11)$$

By combining Eq. (7) to Eq. (11), we obtain

$$\text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{A}) = \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2).$$

This completes the proof of Theorem 2. $\square$

Remark 7. Note that game $\mathbf{G}_3$ requires u to be invertible; thus, the sspRLWE assumption used above has an additional condition that h is invertible. For our parameter sets, the probability that h is non-invertible for a uniformly chosen $h \leftarrow R_q$ is significant. Consequently, the condition introduced here does not make much difference to the security of the sspRLWE assumption in practice.

7.2 Security Proof of NEV-KEM-CPA

In this section, we view the hash function $H : \{0, 1\}^{n/k} \rightarrow \{0, 1\}^\kappa$ used by the KEM scheme NEV-KEM-CPA as a random oracle, and then prove the IND-CPA security of NEV-KEM-CPA in the ROM and QROM.

7.2.1 Security Proof of NEV-KEM-CPA in the ROM

Theorem 3. Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CPA adversary against NEV-KEM-CPA, making q_H classical queries to the random oracle H . Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\text{Adv}_{\text{NEV-KEM-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq q_H \cdot \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + q_H \cdot \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sSPRLWE}}(\mathcal{B}_2).$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 10 to prove this theorem. In these games, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc for notational simplicity.

Game $\mathbf{G}_0$	Game $\mathbf{G}_1$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \kappa]$	1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \kappa]$
2. $b \leftarrow \{0, 1\}$, $M^* \leftarrow \{0, 1\}^{n/k}$ $c^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K_0^* = H(M^*), K_1^* \leftarrow \{0, 1\}^\kappa$	2. $b \leftarrow \{0, 1\}$, $M^* \leftarrow \{0, 1\}^{n/k}$ $c^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K_0^* \leftarrow \{0, 1\}^\kappa, K_1^* \leftarrow \{0, 1\}^\kappa$
3. $b' \leftarrow \mathcal{A}^H(pk, c^*, K_b^*)$	3. $b' \leftarrow \mathcal{A}^H(pk, c^*, K_b^*)$
4. return $\llbracket b' = b \rrbracket$	4. return $\llbracket b' = b \rrbracket$

Fig. 10: Games $\mathbf{G}_0$ to $\mathbf{G}_1$ in the proof of Theorem 3 (where $\text{Fcs}[m, n]$ refers to the set containing all functions from $\{0, 1\}^m$ to $\{0, 1\}^n$).

Game $\mathbf{G}_0$: This game is the IND-CPA security game of NEV-KEM-CPA with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-KEM-CPA}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (12)$$

Game $\mathbf{G}_1$: This game is the same as game $\mathbf{G}_0$, except that the K_0^* is uniformly sampled over $\{0, 1\}^\kappa$.

Denote Bad_{M^*} as the event that the adversary $\mathcal{A}$ ever queries H with M^* . It is not hard to see that game $\mathbf{G}_0$ is identical to game $\mathbf{G}_1$ if Bad_{M^*} does not occur. Hence, by the fundamental difference lemma, we have

$$|\Pr[1 \leftarrow \mathbf{G}_0] - \Pr[1 \leftarrow \mathbf{G}_1]| \leq \Pr[\text{Bad}_{M^*} : \mathbf{G}_1]. \quad (13)$$

Since the bit b is independent of $\mathcal{A}$'s view in game $\mathbf{G}_1$, we have

$$\Pr[1 \leftarrow \mathbf{G}_1] = \frac{1}{2}. \quad (14)$$

Next, we define an adversary $\mathcal{B}$ against the OW-CPA security of NEV-PKE-OW as follows:

- After getting the public key pk and challenge ciphertext c^* , $\mathcal{B}$ samples $b \leftarrow \{0, 1\}$, $K_0^*, K_1^* \leftarrow \{0, 1\}^\kappa$, and runs $\mathcal{A}^H(pk, c^*, K_b^*)$. $\mathcal{B}$ simulates H on the fly and outputs $\mathcal{A}$'s i -th H queries, where $i \leftarrow \{1, \dots, q_H\}$.

The above definition implies that, if Bad_{M^*} occurs, $\mathcal{B}$ will output M^* with probability at least $1/q_H$. Thus,

$$\begin{aligned} \Pr[\text{Bad}_{M^*} : \mathbf{G}_1] &\leq q_H \cdot \text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{B}) \\ &\stackrel{(a)}{=} q_H \cdot \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + q_H \cdot \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2). \end{aligned} \quad (15)$$

Here (a) follows from Theorem 2. By combining Eq. (12) to Eq. (15), we finally obtain

$$\text{Adv}_{\text{NEV-KEM-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq q_H \cdot \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + q_H \cdot \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2).$$

This completes the proof of Theorem 3. $\square$

7.2.2 Security Proof of NEV-KEM-CPA in the QROM

Theorem 4. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CPA adversary against NEV-KEM-CPA, making q_H quantum queries to the random oracle H . Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{NEV-KEM-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2q_H \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2)}.$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 11 to prove this theorem. Similar to Fig. 10, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc in Fig. 11 for notational simplicity.

Game $\mathbf{G}_0$: This game is the IND-CPA security game of NEV-KEM-CPA with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-KEM-CPA}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (16)$$

Game $\mathbf{G}_1$: This game is the same as game $\mathbf{G}_0$, except that the K_0^* is uniformly sampled over $\{0, 1\}^\kappa$.

Let $\mathfrak{D}$ be a distribution of the tuple $(G, H, S = \{M^*\}, pk, c^*, K_b^*)$, where (pk, sk) is generated by NEV-PKE-OW.KeyGen, $b \leftarrow \{0, 1\}$, $K_0^* \leftarrow \{0, 1\}^\kappa$, $K_1^* \leftarrow \{0, 1\}^\kappa$, $M^* \leftarrow \{0, 1\}^{n/k}$, $c^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$, $H \leftarrow \text{Fcs}[n/k, \kappa]$ and $H(M^*)$ is replaced with K_0^* ; G is the same as H except that $G(M^*)$ is a fresh

Game $\mathbf{G}_0$	Game $\mathbf{G}_1$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \kappa]$	1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \kappa]$
2. $b \leftarrow \{0, 1\}, M^* \leftarrow \{0, 1\}^{n/k}$ $c^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K_0^* = H(M^*), K_1^* \leftarrow \{0, 1\}^\kappa$	2. $b \leftarrow \{0, 1\}, M^* \leftarrow \{0, 1\}^{n/k}$ $c^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K_0^* \leftarrow \{0, 1\}^\kappa, K_1 \leftarrow \{0, 1\}^\kappa$
3. $b' \leftarrow \mathcal{A}^H(pk, c^*, K_b^*)$	3. $b' \leftarrow \mathcal{A}^H(pk, c^*, K_b^*)$
4. return $\llbracket b' = b \rrbracket$	4. return $\llbracket b' = b \rrbracket$

Fig. 11: Games $\mathbf{G}_0$ to $\mathbf{G}_1$ in the proof of Theorem 4. In these two games, $\mathcal{A}$ is a quantum oracle algorithm that can make quantum queries to H .

uniformly random value over $\{0, 1\}^\kappa$. Based on the definition of $\mathfrak{D}$, by denoting $z := (pk, c^*, K_b^*)$, we have

$$\begin{aligned} \Pr[1 \leftarrow \mathbf{G}_0] &= \Pr[b = 1 : b \leftarrow \mathcal{A}^H(z), (G, H, S, z) \leftarrow \mathfrak{D}], \\ \Pr[1 \leftarrow \mathbf{G}_1] &= \Pr[b = 1 : b \leftarrow \mathcal{A}^G(z), (G, H, S, z) \leftarrow \mathfrak{D}]. \end{aligned} \quad (17)$$

By using the One-Way to Hiding theorem [6] with $\mathcal{A}$ and $\mathfrak{D}$, we get

$$\begin{aligned} &|\Pr[1 \leftarrow \mathbf{G}_0] - \Pr[1 \leftarrow \mathbf{G}_1]| \\ &\leq 2q_H \cdot \sqrt{\Pr[w \in S : w \leftarrow \mathcal{B}^G(z), (G, H, S, z) \leftarrow \mathfrak{D}]}. \end{aligned} \quad (18)$$

$\mathcal{B}^G(z)$ here samples $i \leftarrow \{1, \dots, q_H\}$, runs $\mathcal{A}^G(z)$ and measures the i -th G query of $\mathcal{A}$, and finally outputs the measurement outcome w . In game $\mathbf{G}_1$, since the bit b is independent of $\mathcal{A}$'s view, we have

$$\Pr[1 \leftarrow \mathbf{G}_1] = \frac{1}{2}. \quad (19)$$

At this point, we define an adversary $\mathcal{B}_{ow}$ against the OW-CPA security of NEV-PKE-OW as follows:

- After getting the public key pk and challenge ciphertext c^* , $\mathcal{B}$ samples $b \leftarrow \{0, 1\}$, $K_0^*, K_1^* \leftarrow \{0, 1\}^\kappa$, runs $\mathcal{B}^G(z)$ and returns its final output w . Here $z = (pk, c^*, K_b^*)$ and $\mathcal{B}_{ow}$ simulates G by using a $2q_H$ -wise independent function.

By the construction of $\mathcal{B}_{ow}$, we have

$$\begin{aligned} &\Pr[w \in S : w \leftarrow \mathcal{B}^G(z), (G, H, S, z) \leftarrow \mathfrak{D}] \\ &= \text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{B}_{ow}) \\ &\stackrel{(a)}{=} \text{Adv}_{n, q, \chi_f, \chi_g, v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n, q, \chi_r, B_\eta, \bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2). \end{aligned} \quad (20)$$

Here (a) follows from Theorem 2. By combining Eq. (16) to Eq. (20), we obtain

$$\text{Adv}_{\text{NEV-KEM-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2q_H \cdot \sqrt{\text{Adv}_{n, q, \chi_f, \chi_g, v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n, q, \chi_r, B_\eta, \bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2)}.$$

This completes the proof of Theorem 4. $\square$

7.3 Security Proof of NEV-PKE-CPA

In this section, we view the hash function $H : \{0, 1\}^{n/k} \rightarrow \{0, 1\}^\ell$ used by the PKE scheme NEV-PKE-CPA as a random oracle and then prove the IND-CPA security of NEV-PKE-CPA in the ROM and QROM.

7.3.1 Security Proof of NEV-PKE-CPA in the ROM

Theorem 5. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CPA adversary against NEV-PKE-CPA, making q_H classical queries to the random oracle H . Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{NEV-PKE-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq q_H \cdot \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + q_H \cdot \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssPRLWE}}(\mathcal{B}_2).$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 12 to prove this theorem. In these games, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc for notational simplicity.

Game $\mathbf{G}_0$	Game $\mathbf{G}_1$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \ell]$	1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \ell]$
2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^H(pk)$	2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^H(pk)$
3. $b \leftarrow \{0, 1\}$, $M^* \leftarrow \{0, 1\}^{n/k}$ $c_1^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K^* = H(M^*)$, $c_2^* = K^* \oplus M_b^*$	3. $b \leftarrow \{0, 1\}$, $M^* \leftarrow \{0, 1\}^{n/k}$ $c_1^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K^* \leftarrow \{0, 1\}^\ell$, $c_2^* = K^* \oplus M_b^*$
4. $b' \leftarrow \mathcal{A}^H(pk, c_1^*, c_2^*)$	4. $b' \leftarrow \mathcal{A}^H(pk, c_1^*, c_2^*)$
5. return $\llbracket b' = b \rrbracket$	5. return $\llbracket b' = b \rrbracket$

Fig. 12: Games $\mathbf{G}_0$ to $\mathbf{G}_1$ in the proof of Theorem 5.

Game $\mathbf{G}_0$: This game is the IND-CPA security game of NEV-PKE-CPA with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-PKE-CPA}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr [1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (21)$$

Game $\mathbf{G}_1$: This game is the same as game $\mathbf{G}_0$, except that the K^* is replaced with a uniformly sampled value over $\{0, 1\}^\ell$.

Similar with the proof of Theorem 3, game $\mathbf{G}_0$ and $\mathbf{G}_1$ are equivalent if the adversary $\mathcal{A}$ never queries H by M^* , and we can construct adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$|\Pr[1 \leftarrow \mathbf{G}_0] - \Pr[1 \leftarrow \mathbf{G}_1]| \leq q_H \cdot \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + q_H \cdot \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2). \quad (22)$$

Since the bit b is independent of $\mathcal{A}$'s view in game $\mathbf{G}_1$, we have

$$\Pr[1 \leftarrow \mathbf{G}_1] = \frac{1}{2}. \quad (23)$$

By combining Eq. (21) to Eq. (23), we obtain

$$\text{Adv}_{\text{NEV-PKE-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq q_H \cdot \text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + q_H \cdot \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2).$$

This completes the proof of Theorem 5. $\square$

7.3.2 Security Proof of NEV-PKE-CPA in the QROM

Theorem 6. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CPA adversary against NEV-PKE-CPA, making q_H quantum queries to the random oracle H . Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{NEV-PKE-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2q_H \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2)}.$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 13 to prove this theorem. Similar with Fig. 12, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc in Fig. 13 for notational simplicity.

Game $\mathbf{G}_0$	Game $\mathbf{G}_1$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \ell]$	1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H \leftarrow \text{Fcs}[n/k, \ell]$
2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^H(pk)$	2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^H(pk)$
3. $b \leftarrow \{0, 1\}$, $M^* \leftarrow \{0, 1\}^{n/k}$ $c_1^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K^* = H(M^*)$, $c_2^* = K^* \oplus M_b^*$	3. $b \leftarrow \{0, 1\}$, $M^* \leftarrow \{0, 1\}^{n/k}$ $c_1^* \leftarrow \text{NEV-PKE-OW.Enc}(pk, M^*)$ $K^* \leftarrow \{0, 1\}^\ell$, $c_2^* = K^* \oplus M_b^*$
4. $b' \leftarrow \mathcal{A}^H(pk, c_1^*, c_2^*)$	4. $b' \leftarrow \mathcal{A}^H(pk, c_1^*, c_2^*)$
5. return $\llbracket b' = b \rrbracket$	5. return $\llbracket b' = b \rrbracket$

Fig. 13: Games $\mathbf{G}_0$ to $\mathbf{G}_1$ in the proof of Theorem 6. In these two games, $\mathcal{A}$ is a quantum oracle algorithm that can make quantum queries to H .

Game $\mathbf{G}_0$: This game is the IND-CPA security game of NEV-PKE-CPA with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-PKE-CPA}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (24)$$

Game $\mathbf{G}_1$: This game is the same as game $\mathbf{G}_0$, except that the K is replaced with a uniformly sampled value over $\{0, 1\}^\ell$.

Similar with the proof of the Theorem 4, we can analyze the difference between game $\mathbf{G}_0$ and $\mathbf{G}_1$ by using the One-Way to Hiding theorem [6]. In fact, we can construct adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$|\Pr[1 \leftarrow \mathbf{G}_0] - \Pr[1 \leftarrow \mathbf{G}_1]| \leq 2q_H \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2)}. \quad (25)$$

Since the bit b is independent of $\mathcal{A}$'s view in game $\mathbf{G}_1$, we have

$$\Pr[1 \leftarrow \mathbf{G}_1] = \frac{1}{2}. \quad (26)$$

By combining Eq. (24) to Eq. (26), we obtain

$$\text{Adv}_{\text{NEV-PKE-CPA}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2q_H \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2)}.$$

This completes the proof of Theorem 6. $\square$

7.4 Security Proof of NEV-KEM-CCA

In this section, we view the hash functions $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ and $H_2 : \{0, 1\}^{2\kappa} \rightarrow \{0, 1\}^{2\kappa}$ used by the KEM scheme NEV-KEM-CCA as random oracles, and then prove the IND-CCA security of NEV-KEM-CCA in the ROM and QROM. Before giving the main theorems of this section, we first prove the following lemma related to NEV-PKE-OW, which will be used later.

Lemma 4. Denote $\epsilon = \max_{r_0 \in R} \Pr[r = r_0 : r \leftarrow \chi_r]$, then NEV-PKE-OW satisfies

$$\mathbb{E} \left[\max_{M,c} \Pr[c = \text{NEV-PKE-OW.Enc}(pk, M; \rho) : \rho \leftarrow \{0, 1\}^\kappa] \right] \leq \epsilon,$$

where the expectation is taken over $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$.

Proof. As shown in Sec. 3.2, NEV-PKE-OW is a randomized PKE scheme with randomness $\rho \leftarrow \{0, 1\}^\kappa$, which will then be used as the random seed to a secure PRG to generate the random coins to sample $r \leftarrow \chi_r$ and $m \leftarrow \text{Pt2noise}(M, \eta)$. Hence, for a fixed (pk, sk) pair, we can compute

$$\begin{aligned} & \max_{M,c} \Pr[c = \text{NEV-PKE-OW.Enc}(pk, M; \rho) : \rho \leftarrow \{0, 1\}^\kappa] \\ & \stackrel{(a)}{\leq} \max_{m,c} \Pr[c = hr + m : r \leftarrow \chi_r] \\ & \stackrel{(b)}{=} \max_{m,c} \Pr[r = h^{-1}(c - m) : r \leftarrow \chi_r] \\ & \stackrel{(c)}{\leq} \epsilon. \end{aligned}$$

Here (a) follows from the definition of NEV-PKE-OW.Enc , (b) is because $h \in R_q^*$, and (c) follows from the definition of ϵ . Based on the above equation, we have

$$\mathbb{E} \left[\max_{M,c} \Pr [c = \text{NEV-PKE-OW.Enc}(pk, M; \rho) : \rho \leftarrow \{0,1\}^\kappa] \right] \leq \epsilon$$

where the expectation is taken over $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$. This completes the proof of Lemma 4. $\square$

Remark 8. Denote $\gamma := -\log_2 \epsilon$, then the above lemma actually indicates that NEV-PKE-OW is γ -spread, a particular property used by numerous related works.

7.4.1 Security Proof of NEV-KEM-CCA in the ROM

Theorem 7. Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CCA adversary against NEV-KEM-CCA , making q_{H_1} (resp. q_{H_2}) classical queries to the random oracle H_1 (resp. H_2), and making q_D classical queries to the decapsulation oracle. Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\begin{aligned} \text{Adv}_{\text{NEV-KEM-CCA}}^{\text{IND-CCA}}(\mathcal{A}) \leq & (q_D + 2q_{H_2}) \cdot \left(\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2) \right) \\ & + q_D \cdot \epsilon + q_{H_2} \cdot \delta. \end{aligned}$$

Here δ is the decryption failure rate of NEV-PKE-OW , $\epsilon = \max_{r_0 \in R} \Pr[r = r_0 : r \leftarrow \chi_r]$, and the running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A.1. $\square$

7.4.2 Security Proof of NEV-KEM-CCA in the QROM

Theorem 8. Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CCA adversary against NEV-KEM-CCA , making q_{H_1} (resp. q_{H_2}) quantum queries to the random oracle H_1 (resp. H_2), and making q_D classical queries to the decapsulation oracle. Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\begin{aligned} \text{Adv}_{\text{NEV-KEM-CCA}}^{\text{IND-CCA}}(\mathcal{A}) \leq & (24q_{H_2} + 8q_D) \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2)} \\ & + (32q_{H_2} + 16) \cdot \sqrt{\delta} + 128q_{H_2} \cdot \delta + 6q_D \cdot \sqrt{\epsilon}. \end{aligned}$$

Here δ is the decryption failure rate of NEV-PKE-OW , $\epsilon = \max_{r_0 \in R} \Pr[r = r_0 : r \leftarrow \chi_r]$, and the running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A.2. $\square$

7.5 Security Proof of NEV-PKE-CCA

In this section, we view the hash functions $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$, $H_2 : \{0, 1\}^{n/k+\kappa+\ell} \rightarrow \{0, 1\}^\kappa$, and $H_3 : \{0, 1\}^{n/k} \rightarrow \{0, 1\}^\ell$ used by the NEV-PKE-CCA scheme as random oracles, and then prove the IND-CCA security of NEV-PKE-CCA in the ROM and QROM.

7.5.1 Security Proof of NEV-PKE-CCA in the ROM

Theorem 9. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CCA adversary against NEV-PKE-CCA, making q_{H_1} , q_{H_2} , and q_{H_3} classical queries to the random oracle H_1 , H_2 , and H_3 , respectively, and making q_D classical queries to the decryption oracle. Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{NEV-PKE-CCA}}^{\text{IND-CCA}}(\mathcal{A}) \leq & (q_D + q_{H_2} + q_{H_3}) \cdot \left(\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2) \right) \\ & + q_D \cdot \epsilon + q_{H_2} \cdot \delta. \end{aligned}$$

Here δ is the decryption failure rate of NEV-PKE-OW, $\epsilon = \max_{r_0 \in R} \Pr[r = r_0 : r \leftarrow \chi_r]$, and the running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A.3. $\square$

7.5.2 Security Proof of NEV-PKE-CCA in the QROM

Theorem 10. *Let $n, q, k, \eta \in \mathbb{Z}$ and $v = 1 - x^{n/k}$, $\bar{v} = (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q$ be defined as above. Let χ_f, χ_g, χ_r be three probability distributions over R_q . Let $\mathcal{A}$ be an IND-CCA adversary against NEV-PKE-CCA, making q_{H_1} , q_{H_2} , and q_{H_3} quantum queries to the random oracle H_1 , H_2 , and H_3 , respectively, and making q_D classical queries to the decryption oracle. Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{NEV-PKE-CCA}}^{\text{IND-CCA}}(\mathcal{A}) \leq & (24q_{H_2} + 8q_D) \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2)} \\ & + (32q_{H_2} + 16) \cdot \sqrt{\delta} + 128q_{H_2} \cdot \delta + 6q_D \cdot \sqrt{\epsilon} \\ & + \frac{3}{2} \sqrt{\frac{q_{H_3} + 1}{2^{n/k}}} + \frac{8(q_{H_2} + 1)}{\sqrt{2^{n/k+\ell}}}. \end{aligned}$$

Here δ is the decryption failure rate of NEV-PKE-OW, $\epsilon = \max_{r_0 \in R} \Pr[r = r_0 : r \leftarrow \chi_r]$, and the running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A.4. $\square$

8 Known Attacks

As shown in Sec. 7, all schemes within the NEV family are provably secure under the hardness of the NTRU and sspRLWE problems, in both the classical and quantum random oracle models. Leveraging the (tight) reduction from RLWE to sspRLWE presented in Sec. 2.4, together with the fact that NTRU can be viewed as a special case of RLWE, we first examine the most effective known attacks against RLWE problems, and then discuss how these attacks can be adapted to NTRU and sspRLWE. Since the state-of-the-art attacks on RLWE do not exploit the lattice structure, we begin by considering the best known attacks against asymmetric LWE.

8.1 Attacks against LWE

Solvers for LWE mainly include primal attacks, dual attacks (against the underlying lattice problems) and direct solving algorithms such as BKW and Arora-Ge [4]. BKW and Arora-Ge attacks need sub-exponentially (or even exponentially) many samples, and thus they are not relevant to the public-key cryptography scenario where only a restricted amount of samples are available. Therefore, for analyzing and evaluating practical lattice-based crypto-systems, we typically consider primal attacks and dual attacks.

Since any bounded centrally symmetric distribution can be regarded as subgaussian for a certain parameter, for simplicity and without loss of generality, we consider the case that secret vector and error vector in $\text{ALWE}_{n,q,m,\alpha_1,\alpha_2}$ are sampled from subgaussians with parameters α_1 and α_2 respectively. Formally, the problem is to recover $\mathbf{s}$ from samples

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m,$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. In the following, we first consider the primal/dual attacks adapted to the asymmetric setting, namely, asymmetric LWE (ALWE) in [68], and then consider a variant of the MATZOV attack in [17] also adapted to the asymmetric setting.

8.1.1 Primal Attack. The basic intuition behind primal attack is to convert ALWE problem to the corresponding bounded distance decoding (BDD) problem or shortest vector problem by embedding. First, define a m -dimensional lattice

$$\Lambda = \{\mathbf{y} \in \mathbb{Z}^m \mid \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\}.$$

For $m > n$ the $m \times n$ matrix $\mathbf{A}$ has full rank with high probability. Assume that the first n rows of $\mathbf{A}$ are independent (otherwise swap the rows to make it), and denote by $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ the submatrix by keeping the first n rows of $\mathbf{A}$, i.e., $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \end{pmatrix}$. Denote $\mathbf{A}_1$'s inverse by $\mathbf{A}_1^{-1} \in \mathbb{Z}_q^{n \times n}$, and let $\mathbf{A}' = \begin{pmatrix} \mathbf{I}_n \\ \mathbf{A}_2 \mathbf{A}_1^{-1} \end{pmatrix}$. Then, we have

$$\begin{aligned} \Lambda &= \{\mathbf{y} \mid \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \\ &= \{\mathbf{y} \mid \mathbf{y} = \mathbf{A}'\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \subset \mathbb{Z}^m, \end{aligned}$$

and the columns of the following matrix

$$\mathbf{B} = \begin{pmatrix} \mathbf{I}_n & 0 \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q \mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitutes a basis of Λ . It is easy to see $\det(\Lambda) = q^{m-n}$. We have by $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$ that the distance between vector $\mathbf{b}$ and lattice Λ is $\|\mathbf{e}\|$. If we could find the closest lattice point $\mathbf{u} \in \Lambda$ to $\mathbf{b}$, then we have $\mathbf{e} = \mathbf{b} - \mathbf{u}$ with high probability. In other words, the problem can be reduced to bounded distance decoding (BDD) problem on Λ , and then one applies the nearest planes algorithm [47] to solve. Furthermore, using Kannan's embedding [46,2] one can reduce BDD to unique shortest vector problem (uSVP). In detail, consider lattice Λ' spanned by the column vectors of the following matrix:

$$\mathbf{B}' = \begin{pmatrix} \mathbf{I}_n & 0 & \mathbf{b} \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q \mathbf{I}_{m-n} & t \\ 0 & 0 & t \end{pmatrix} \in \mathbb{Z}^{(m+1) \times (m+1)},$$

where $t \in \mathbb{Z}$ is an adjustable parameter. Theoretically, the algorithm works best when $t = \|\mathbf{e}\|$. However, empirical experiments suggest that best performance is achieved when $t = 1$, which is the typical value we choose. In this case, we have that $\mathbf{v} = (\mathbf{e}^T, 1)^T \in \mathbb{Z}^{m+1}$ is the unique shortest vector in Λ' with high probability. To summarize, we work on uSVP problem in Λ' to find out the error vector $\mathbf{e} \in \mathbb{Z}^m$, which is in turn used to solve the system of linear congruences to recover $\mathbf{s} \in \mathbb{Z}_q^n$, i.e., $\mathbf{A}\mathbf{s} = \mathbf{b} - \mathbf{e}$. Besides, since every element in vector $\mathbf{e}$ is sampled from subgaussian χ_{α_2} , we have $\|\mathbf{v}\| \approx \|\mathbf{e}\| \approx \alpha_2 \sqrt{m}$, namely, $\mathbf{v}$ is of small norm.

One can also make use of the fact that $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ has small norm in practical lattice-based cryptosystems by considering a variant of the LWE embedding [7,5]. Formally, let $c = \alpha_2/\alpha_1$ and consider lattice

$$\Lambda = \left\{ \mathbf{v} = \begin{pmatrix} c\mathbf{x} \\ \mathbf{y} \\ \alpha_2 z \end{pmatrix} \in \mathbb{R}^{n+m+1} \left| (\mathbf{A}\|\mathbf{I}_m\| - \mathbf{b})\mathbf{u} = 0 \bmod q, \mathbf{u} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ z \end{pmatrix} \in \mathbb{Z}^{n+m+1} \right. \right\},$$

Then the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_n & 0 & 0 \\ -\mathbf{A} & q\mathbf{I}_m & \mathbf{b} \\ 0 & 0 & \alpha_2 \end{pmatrix} \in \mathbb{R}^{(n+m+1) \times (n+m+1)},$$

constitutes a basis of Λ . We observe that Λ is of dimension $m+n+1$, $\det(\Lambda) = \alpha_2 c^n q^m$, and that $\mathbf{v} = (c\mathbf{s}^T, \mathbf{e}^T, \alpha_2)^T \in \mathbb{R}^{m+n+1}$ is a short vector in Λ (and $\|\mathbf{v}\| \approx \alpha_2 \sqrt{n+m+1}$). Therefore, we recover $\mathbf{s}$ by solving uSVP on lattice Λ .

The Cost of Primal Attack. As shown above, the computational costs of primal attack is mainly dominated by the cost of solving uSVP problem. The best known SVP solver is the BKZ- b lattice basis reduction and its variants. Given a d -dimensional basis as input, the algorithm translates the lattice basis reduction problem to the SVP problem on b -dimensional sublattice ($b < d$). BKZ- b repeats

the SVP solving procedure $O(d)$ times (the actual number may depend on the specific algorithm used) on different b -dimensional sublattices to reduce the norm of the basis (of the original d -dimensional basis). Therefore, the cost of BKZ- b basis reduction is mainly decided by SVP solving algorithm for b -dimensional lattice.

Assume that the reduced basis obtained from running BKZ- b on d -dimensional lattice basis $\mathbf{B} \in \mathbb{Z}^{d \times d}$ satisfies the Geometry Series Assumption (GSA), which is the best scenario for adversary. Let $\hat{\mathbf{B}} = (\hat{\mathbf{b}}_1, \dots, \hat{\mathbf{b}}_d)$ be a reduced basis, then we have

$$\|\hat{\mathbf{b}}_1\| = \delta^d \det(\Lambda)^{1/d}, \quad \|\hat{\mathbf{b}}_i^*\| = \delta^{-2d(i-1)/(d-1)} \|\hat{\mathbf{b}}_1\|,$$

where $\delta = ((\pi b)^{1/b} \cdot b / 2\pi e)^{\frac{1}{2(b-1)}} [20]$, $\hat{\mathbf{B}}^* = (\hat{\mathbf{b}}_1^*, \dots, \hat{\mathbf{b}}_d^*)$ is the orthogonalized matrix of $\hat{\mathbf{B}}$ (i.e., $\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_1^*$). Specifically, the literatures [5,2] show that when the norm of the projection of the unique shortest vector $\mathbf{v}$ into the space spanned by the last b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ is less than $\|\hat{\mathbf{b}}_{d-b+1}^*\|$, then the BKZ- b lattice basis reduction recovers $\mathbf{v}$.

The norm of $\mathbf{v}$'s projection into the space spanned by b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ approximately equals $\ell \sqrt{b/d}$, where $\ell = \|\mathbf{v}\|$. That is, we assume every component of vector $\mathbf{v}$'s projection is of roughly the same size. In this case, the computational cost of primal attack is mainly decided by the minimal value of b that satisfies the constraint

$$\ell \sqrt{b/d} \leq \delta^{(-d^2+2db-d)/(d-1)} \det(\Lambda)^{1/d}. \quad (27)$$

Following [5], our estimation of the complexity of solving uSVP on d -dimensional lattice is based on that of solving SVP on b -dimensional lattice, where b satisfy inequality (27) (namely, the Core-SVP model). Furthermore, we use $\text{cost}_b = 2^{0.292b}$ [5,13] and $\text{cost}_b = 2^{0.257b}$ [18] to estimate the complexities of classical and quantum algorithms (for solving SVP on b -dimensional lattice) respectively.

8.1.2 Dual Attack. Dual attack translates the LWE problem to a ℓ_2 -norm SIS problem, whose solution is used to solve the decisional LWE by distinguishing between subgaussians (for certain parameters) and uniform distribution over $\mathbb{Z}_q^n$. The first dual attack was given in [52], which only cares about the error distribution of the LWE-like problem, and thus does not distinguish between LWE and ALWE. In other words, the original dual attack solves ALWE the same way as it does to LWE, and it appears more efficient when $\alpha_1 \gg \alpha_2$. Formally, let

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\},$$

be a lattice of dimension $d = m \gg n$. Note that $\mathbf{A}$ has full rank with high probability and we assume the submatrix $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ made up by the first n columns of $\mathbf{A}^T$ has rank n and $\mathbf{A}^T = (\mathbf{A}_1 \parallel \mathbf{A}_2) \in \mathbb{Z}_q^{n \times m}$. Let $\mathbf{A}' = (\mathbf{I}_n \parallel \mathbf{A}_1^{-1} \mathbf{A}_2)$. Then, we have that

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\} = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}' \mathbf{x} = \mathbf{0} \bmod q\},$$

and the column vectors of

$$\mathbf{B} = \begin{pmatrix} q\mathbf{I}_n & -\mathbf{A}_1^{-1}\mathbf{A}_2 \\ 0 & \mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitute a basis of lattice Λ and $\det(\Lambda) = q^n$. The dual attack first solves the SIS problem by finding out a short vector $\mathbf{v} \in \mathbb{Z}^m$ with $\mathbf{A}^T \mathbf{v} = \mathbf{0} \bmod q$ in lattice Λ with basis $\mathbf{B}$. It then computes $u = \langle \mathbf{v}, \mathbf{b} \rangle = \langle \mathbf{v}, \mathbf{e} \rangle \bmod q$ in order to distinguish between subgaussian and uniform distributions. Specifically, if the norm $\ell = \|\mathbf{v}\|$ is relatively small, then the value of $\langle \mathbf{v}, \mathbf{e} \rangle$ is relatively small too and u can be seen as random variable drawn from subgaussian distribution with standard variance $\ell\alpha_2$, which can be distinguished from uniform distribution on $\mathbb{Z}_q$ with probability $4 \exp(-2\pi^2\tau^2)$, where $\tau = \ell\alpha_2/q$.

Similar as for primal attacks, we can also make use of the fact that $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ has small norm [5] to optimize the dual attack. Formally, define lattice

$$\Lambda = \{(\mathbf{c}\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n} \mid \mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q, \mathbf{x} \in \mathbb{Z}^m, \mathbf{y} \in \mathbb{Z}^n\},$$

where Λ has dimension $d = m + n$ and $c = \frac{\alpha_2}{\alpha_1}$. Then, the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_m & 0 \\ \mathbf{A}^T & q\mathbf{I}_n \end{pmatrix} \in \mathbb{R}^{(m+n) \times (m+n)}$$

constitutes a basis of lattice Λ with $\det(\Lambda) = c^m q^n$. Likewise, we find out a short vector $\mathbf{v} = (\mathbf{c}\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n}$ with $\mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q$ from $(m+n)$ -dimensional lattice Λ with basis $\mathbf{B}$, and then compute

$$u = \langle \mathbf{x}, \mathbf{b} \rangle = \langle \mathbf{y}, \mathbf{s} \rangle + \langle \mathbf{c}\mathbf{x}, \mathbf{e}/c \rangle \bmod q,$$

and therefore distinguish the above u from uniform. Specifically, the values of $\langle \mathbf{y}, \mathbf{s} \rangle$ and $\langle \mathbf{x}, \mathbf{e} \rangle$ are relatively small for small $\ell = \|\mathbf{v}\|$. Thus, $u = \langle \mathbf{y}, \mathbf{s} \rangle + \langle \mathbf{c}\mathbf{x}, \mathbf{e}/c \rangle$ can be almost regarded as following subgaussian distribution with parameter $\ell\alpha_1$. This enable efficient algorithms that distinguish the subgaussian-distributed u from uniform with probability $4 \exp(-2\pi^2\tau^2)$, where $\tau = \ell\alpha_1/q$.

The Cost of Dual Attack. As shown above, the computational costs of dual attack is mainly dominated by the cost of solving SIS problem and transforming distinguishing attacks into secret recovery attacks (with success probability more than half) for the ALWE problem. We use the model of primal attack to estimate the cost of solving SIS problem. That is, applying BKZ- b to reduce the basis of d -dimensional lattice yields a short vector $\mathbf{v}$ of norm $\ell = \|\mathbf{v}\| = \delta^d \det(\Lambda)^{1/d}$. Substituting this into the previous analysis, we distinguish ALWE problem with advantage $\epsilon = 4 \exp(-2\pi^2\tau^2)$, where τ is fully decided by ℓ and the actual dual attack algorithm. To obtain the final secret recovery attack with probability greater than $1/2$, we need $1/\epsilon^2$ short vectors. Taking into account that each invocation of the sieving yields $2^{0.2075b}$ short vectors, we need to repeat it $R = 1/\max(1, 1/(2^{0.2075b}\epsilon^2))$ times.

Following the cost estimation framework for primal attacks, we adopt the Core-SVP model, which uses the complexity of solving SVP on a b -dimensional lattice as a (conservative) estimate for solving SVP on a d -dimensional lattice. Specifically, we set $\text{cost}_b = 2^{0.292b}$ for classical algorithms and $\text{cost}_b = 2^{0.257b}$ for quantum

algorithms when solving SVP on b -dimensional lattices. Under this model, the complexity of dual attack is estimated as

$$1/\max(1, 1/(2^{0.2075b} \cdot 16 \exp(-4\pi^2\tau^2))) \cdot \text{cost}_b, \quad (28)$$

where τ is decided by b and other parameters of the actual dual attack.

8.1.3 Refined MATZOV Attack (MATZOV[†]). The MATZOV attack given in [50] is basically a variant of the hybrid-dual attack which splits the attack into three parts: a BKZ lattice reduction stage, a guessing stage and a FFT-based distinguishing stage. Formally, given a LWE problem

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m,$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. Let $n_{\text{lat}}, n_{\text{enu}}, n_{\text{fft}}$ be integers satisfying $n = n_{\text{lat}} + n_{\text{enu}} + n_{\text{fft}}$. The matrix $\mathbf{A}$ and secret $\mathbf{s}$ can be divided accordingly:

$$\mathbf{A} = [\mathbf{A}_{\text{enu}}, \mathbf{A}_{\text{fft}}, \mathbf{A}_{\text{lat}}] \in \mathbb{Z}_q^{m \times n_{\text{enu}}} \times \mathbb{Z}_q^{m \times n_{\text{fft}}} \times \mathbb{Z}_q^{m \times n_{\text{lat}}},$$

$$\mathbf{s} = \begin{pmatrix} \mathbf{s}_{\text{enu}} \\ \mathbf{s}_{\text{fft}} \\ \mathbf{s}_{\text{lat}} \end{pmatrix} \in \mathbb{Z}_q^{n_{\text{enu}}} \times \mathbb{Z}_q^{n_{\text{fft}}} \times \mathbb{Z}_q^{n_{\text{lat}}}.$$

The attack works by first performing BKZ lattice reduction on the lattice generated by

$$\mathbf{B}_{\text{lat}} = \begin{pmatrix} \mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{Z}_q^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}$$

to obtain a set $\mathcal{S}$ of N vectors $(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}_q^{m+n_{\text{lat}}}$ with small Euclidean norm such that $\mathbf{y} = \mathbf{A}_{\text{lat}}^T \mathbf{x}$. Then, for each vector $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$, by the equation $\mathbf{b} = \mathbf{A}_{\text{enu}}\mathbf{s}_{\text{enu}} + \mathbf{A}_{\text{fft}}\mathbf{s}_{\text{fft}} + \mathbf{A}_{\text{lat}}\mathbf{s}_{\text{lat}} + \mathbf{e}$ we have that

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}}\mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{x}, \mathbf{A}_{\text{fft}}\mathbf{s}_{\text{fft}} \rangle}_{= \langle \hat{\mathbf{a}}, \mathbf{s}_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle}_{= \hat{e}}, \quad (29)$$

where $\hat{\mathbf{a}} = \mathbf{A}_{\text{fft}}^T \mathbf{x}$, $\hat{b} = \mathbf{x}^T (\mathbf{b} - \mathbf{A}_{\text{enu}}\mathbf{s}_{\text{enu}})$ and $\hat{e} = \mathbf{x}^T \mathbf{e} + \mathbf{y}^T \mathbf{s}_{\text{lat}}$. Clearly, if $(\mathbf{e}, \mathbf{s}_{\text{lat}})$ has small Euclidean norm, then $\hat{e}$ is small. Thus, by correctly guessing $\mathbf{s}_{\text{enu}}$, we can obtain a new LWE sample $\hat{b} = \hat{\mathbf{a}}^T \mathbf{s}_{\text{fft}} + \hat{e}$ in much smaller dimension $n_{\text{fft}} \ll n$ for each $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$. Finally, the attack ends by running a FFT-based distinguishing algorithm over $\mathbb{Z}_q^{n_{\text{fft}}}$ which can determine whether a guessed value for $\mathbf{s}_{\text{enu}}$ is correct or not. The core improvement of the MATZOV attack over previous hybrid-dual attacks such as [39] is the introduction of modulus-switching to speedup the FFT-based distinguishing stage, which greatly reduces the FFT cost from previous $O(n_{\text{fft}}q^{n_{\text{fft}}})$ to $O(n_{\text{fft}}p^{n_{\text{fft}}})$ for some much smaller $p \ll q$ by switching the obtained LWE samples in Equation (29) from $\mathbb{Z}_q^{n_{\text{fft}}}$ to $\mathbb{Z}_p^{n_{\text{fft}}}$. The main problem of the MAZOV attack is the requirement of some controversial assumptions in the complexity analysis [26,60].

Recently, Carrier et al. [17] replaced the modulus switching part of the MATZOV attack with a lossy source/quantizing approach, and obtained a variant that is supported by experimentally validated assumptions. Specifically, given integers $k_{\text{fft}} \ll n_{\text{fft}}$ and a linear code generated by some $\mathbf{G} \in \mathbb{Z}_q^{n_{\text{fft}} \times k_{\text{fft}}}$ such that for each $\mathbf{A}_{\text{fft}}^T \mathbf{x}$ there exists a vector $\mathbf{u} \in \mathbb{Z}_q^{k_{\text{fft}}}$ and a small vector $\mathbf{e}_{\mathbf{x}} \in \mathbb{Z}_q^{n_{\text{fft}}}$ satisfying $\mathbf{A}_{\text{fft}}^T \mathbf{x} = \mathbf{G}\mathbf{u} + \mathbf{e}_{\mathbf{x}}$, we can rewrite Equation (29) as follows:

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{u}, \mathbf{G}^T \mathbf{s}_{\text{fft}} \rangle}_{= \langle \mathbf{u}, \mathbf{s}'_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle + \langle \mathbf{e}_{\mathbf{x}}, \mathbf{s}_{\text{fft}} \rangle}_{= \hat{e}'}. \quad (30)$$

Now, the attack can be finished by running the FFT-based distinguishing algorithm over $\mathbb{Z}_q^{k_{\text{fft}}}$, and the FFT cost is reduced from previous $O(n_{\text{fft}} q^{n_{\text{fft}}})$ to $O(k_{\text{fft}} q^{k_{\text{fft}}})$. In [17], Carrier et al. demonstrated that their variant of the MATZOV attack is not only supported by experimental evidence but also outperforms the original MATZOV attack [50]. Thus, for more conservative concrete security estimates, one should consider the MATZOV attack in [17], as also remarked in [37].

Since Carrier et al.'s attack [17] only considers symmetric distributions (namely, $\alpha_1 = \alpha_2$), we slightly adapt their attack to handle asymmetric distributions. Formally, let $c = \alpha_2/\alpha_1$, we scale the lattice used in the BKZ lattice reduction stage as follows:

$$\mathbf{B}'_{\text{lat}} = \begin{pmatrix} c\mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{R}^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}.$$

By doing this, we can model the distribution of $\hat{e}'$ in Equation (30) as a Gaussian distribution with parameter $\ell\alpha_1$, where $\ell = \|(c\mathbf{x}, \mathbf{y}, \mathbf{e}_{\mathbf{x}})\|_2$ for each $\mathbf{y} = \mathbf{A}_{n_{\text{lat}}}^T \mathbf{x}$. For ease of presentation, we refer to our above refined MATZOV attack as MATZOV[†].

The Cost of MATZOV[†] Attack. As shown in [17], the total cost of the MATZOV[†] attack depends on the cost T_{sample} for obtaining the set $\mathcal{S}$ of N short vectors, R for the expected number of times to guess $\mathbf{s}_{\text{enu}}$ correctly, T_{dec} for decoding $\mathbf{e}_{\mathbf{x}}$ from $\mathbf{A}_{\text{fft}}^T \mathbf{x}$, and T_{fft} for running the FFT-based distinguishing algorithm, namely,

$$T_{\text{cost}} = T_{\text{sample}} + R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}}),$$

where $T_{\text{sample}} = 2^{0.292b}$ and $N = 2^{0.2075b}$ for performing SVP sieving algorithm in b -dimensional sublattice. For quantum security estimates, in addition to considering quantum SVP sieving cost $T'_{\text{sample}} = 2^{0.257b}$, we additionally assume that the guessing-then-FFT part can achieve a quantum quadratic speedup [3], namely,

$$T'_{\text{cost}} = T'_{\text{sample}} + \sqrt{R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}})}.$$

We note that this will only make our concrete estimates more conservative. Because T_{dec} and T_{fft} depend strongly on the modulus size, and R on the sparsity of the secret vector, the above analysis immediately imply that the MATZOV[†] attack may be particularly advantageous for sparse secrets and small modulus.

8.2 Solving the $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ problem

As discussed in Sec. 2.4, for the setting that $v = 1 - x^{n/k} \in R_q$ is invertible, the NTRU problem $\text{DNTRU}_{n,q,\chi_f,\chi_g,v}$ is essentially equivalent to the standard NTRU problem (with $v = 3$) up to the choices of the secret key distribution. Moreover, the ℓ_2 -norm of $vf' + 1$ is only roughly about $\sqrt{2}$ times larger than that of f' , which is small as long as f' is chosen from a small distribution. Thus, one can either solve the NTRU problem by taking $f = vf' + 1$ as whole just as in the standard lattice attacks against the NTRU problem with secret distributions $(\chi'_f = v\chi_f, \chi_g)$ in lattice $\mathcal{L}_0^\perp(h)$ for $h = g/f$, or solve the BDD problem on the shifted lattice $\mathcal{L}_h^\perp(-vh)$ by treating it as an RLWE instance $(vh, h = -vhf' + g)$ with secret distribution χ_f and noise distribution χ_g , where

$$\mathcal{L}_c^\perp(a) = \{(u, w) \in R_q = \mathbb{Z}[x]/(x^n + 1) : au + w = c \in R_q\}.$$

We adopt the latter approach for concrete estimates of our NTRU problem, since the norm of the short vector $(g, f', 1)$ in the latter case—which is independent of v —is smaller than that of $(f = vf' + 1, -g)$ in the former case. Finally, dual-like attacks—including $\text{MATZOV}^\dagger$ —are not directly applicable to NTRU problems [1].

8.3 Solving the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem

In Sec. 2.4, we have shown that the sspRLWE problem is polynomially equivalent to the standard DRLWE problem under a reduction that is significantly tighter than reductions exist from RLWE to a shortest vector problem. Moreover, as discussed in [27] regarding the concrete hardness of the RLWE2 problem (which can be seen as a special case of the sspRLWE problem), we believe that, given the current state of the art in solving DRLWE problems, the best known attack against the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem must also solve the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem. First, if the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem is hard, then a given DRLWE sample $(a, b = ar + e)$ is indistinguishable from uniform. This implies that no PPT algorithm can extract useful information about e from such a sample. Second, according to the study in [23, 51, 16], if there exists an algorithm that solves the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem with lower complexity than directly solving the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem, then this would immediately yield a new, more efficient algorithm for the $\text{DRLWE}_{n,q,\chi_r,B_\eta}$ problem—which is not known to exist. Therefore, we directly estimate the concrete hardness of the $\text{sspRLWE}_{n,q,\chi_r,B_\eta,\bar{v}}$ problem by treating it as a standard DRLWE problem $\text{DRLWE}_{n,q,\chi_r,B_\eta}$.

8.4 Concrete Security Estimates Against Lattice Attacks

The complexity of the attacks depends on the attack type, the number of samples m , and the block size $b \in \mathbb{Z}$ used in the $\text{BKZ-}b$ algorithm. Therefore, to obtain a conservative overall security estimation for **NEV** under the nine concrete parameter sets, we enumerate all possible values of m and b . In detail, Table 6 estimates the complexities of the underlying NTRU and RLWE problems against the aforementioned attacks, where the concrete security estimates against $\text{MATZOV}^\dagger$

are derived from a code that modifies the publicly available implementation by Carrier et al. [17], which can be found at <https://github.com/kevin-carrier/CodedDualAttack>. For comparison, we also provide the figures against the original MATZOV attack [50] given by the LWE estimator [4]. As shown in Table 6, the MATZOV[†] attack appears especially efficient for sparse keys and small moduli—particularly in the high-dimension setting—and it nearly dominates the concrete estimates for all parameters.

Table 6: The Concrete Security Estimates of NEV (where C and Q denotes classical and quantum security, respectively)

Parameter Set	RLWE Hardness (m, b, sec)				NTRU Hardness (m, b, sec)
	Primal	Dual	MATZOV	MATZOV [†]	Primal
NEV-C1	C: (461,463,135) Q: (461,463,118)	C: (476,457,133) Q: (476,457,117)	C: (512,442,129) Q: (512,445,115)	C: (448,432,129) Q: (426,399,103)	C: (446,453,132) Q: (446,453,116)
NEV-R1	C: (486,464,135) Q: (486,464,119)	C: (491,458,133) Q: (491,458,117)	C: (512,450,131) Q: (512,453,116)	C: (504,455,134) Q: (467,414,107)	C: (486,464,135) Q: (486,464,119)
NEV-D1	C: (511,463,135) Q: (511,463,118)	C: (511,457,133) Q: (511,457,117)	C: (512,478,140) Q: (512,481,123)	C: (509,453,134) Q: (467,430,111)	C: (511,463,135) Q: (511,463,118)
NEV-C2	C: (806,1030,300) Q: (806,1030,264)	C: (841,1019,297) Q: (841,1019,261)	C: (1024,953,279) Q: (1024,966,248)	C: (729,914,268) Q: (688,841,217)	C: (751,912,266) Q: (751,912,234)
NEV-R2	C: (826,975,284) Q: (826,975,250)	C: (831,966,282) Q: (831,966,248)	C: (1024,916,268) Q: (1024,934,240)	C: (860,892,261) Q: (740,817,211)	C: (826,975,284) Q: (826,975,250)
NEV-D2	C: (921,959,280) Q: (921,959,246)	C: (951,950,277) Q: (951,950,244)	C: (1024,911,266) Q: (1024,921,237)	C: (923,900,264) Q: (901,845,218)	C: (921,959,280) Q: (921,959,246)
NEV-C3	C: (1242,2090,610) Q: (1242,2090,537)	C: (1282,2080,607) Q: (1282,2080,534)	C: (2048,1936,565) Q: (2048,1966,505)	C: (1366,1782,521) Q: (1162,1590,410)	C: (1296,1871,546) Q: (1296,1871,480)
NEV-R3	C: (1421,2043,596) Q: (1421,2043,525)	C: (1491,2029,592) Q: (1491,2029,521)	C: (2048,1897,554) Q: (2048,1925,494)	C: (1386,1797,526) Q: (1262,1638,422)	C: (1431,1998,583) Q: (1431,1998,513)
NEV-D3	C: (1545,1958,572) Q: (1545,1958,503)	C: (1556,1945,568) Q: (1556,1945,499)	C: (2048,1832,535) Q: (2048,1857,477)	C: (1466,1765,516) Q: (1383,1630,420)	C: (1545,1958,572) Q: (1545,1958,503)

8.5 Attacks Exploiting Decryption Failures

In line with standard practice for practical lattice-based schemes such as Kyber [13], we select concrete parameters that achieve a negligible decryption failure probability to ensure compactness. Although our estimation model is quite conservative as discussed in Sec. 3.2, it only considers the scenario where all random coins are honestly sampled from the intended distributions. In practice, however, an adversary may depart from this setting and search across many different values of M (and μ for NEV-PKE-CCA) until finding one for which the induced random coins ρ result in a decryption failure, thus leaking information about the secret key. More alarmingly, after triggering the first failure, the adversary can exploit the directional failure boosting attack [24] to obtain additional decryption failures at a significantly reduced cost. Consequently, to remain on the safe side, one must ensure that even a single failure is difficult to provoke.

We follow the approach of [13] to demonstrate that, for our concrete parameters in Table 2, the corresponding probability of decryption failures is sufficiently low to resist attacks that exploit such failures. Our analysis crucially relies on the simple decryption formula of **NEV**. Formally, from inequalities (4) and (6) in Sec. 3.2, the decryption failure of **NEV** is determined by the term

$$\begin{aligned} p_j &= \sum_y \Pr \left[\langle \hat{e}_j'', y \rangle \geq \frac{k \cdot (q-1)}{4} \right] \\ &= \sum_y \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i) + \hat{e}_j', y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right], \end{aligned}$$

where the probability is taken over the random choice of $\hat{g}_i \leftarrow \chi_g, \hat{f}_i' \leftarrow \chi_f, \hat{r}_i \leftarrow \chi_r, \hat{e}_i \leftarrow B_\eta$. For a fixed y , let

$$p_{j,y} = \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i) + \hat{e}_j', y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right],$$

and $t_y = \left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i), y \right\rangle$. Clearly, the probability of $p_{j,y}$ is primarily determined by the value of t_y . Moreover, let $\hat{\mathbf{R}}_i, \hat{\mathbf{E}}_i$ be the coefficient matrices of $\hat{r}_i, \hat{e}_i$ defined via multiplication over the subring $S = \mathbb{Z}[Y]/(Y^k + 1)$, and let $\hat{\mathbf{g}}_i, \hat{\mathbf{f}}_i', \mathbf{y}$ be the coefficient vectors of $\hat{g}_i, \hat{f}_i', y$ in column representation. Then,

$$\begin{aligned} t_y &= \left\langle \sum_{i \in [n/k]} (\hat{g}_i \hat{r}_i + \hat{f}_i' \hat{e}_i), y \right\rangle = \left\langle \sum_{i \in [n/k]} (\hat{\mathbf{R}}_i \hat{\mathbf{g}}_i + \hat{\mathbf{E}}_i \hat{\mathbf{f}}_i'), \mathbf{y} \right\rangle \\ &= \mathbf{y}^T \begin{pmatrix} \hat{\mathbf{R}}_1 \dots \hat{\mathbf{R}}_{n/k} & \hat{\mathbf{E}}_1 \dots \hat{\mathbf{E}}_{n/k} \end{pmatrix} \begin{pmatrix} \hat{\mathbf{g}}_1 \\ \vdots \\ \mathbf{g}_{n/k} \\ \hat{\mathbf{f}}_1' \\ \vdots \\ \hat{\mathbf{f}}_{n/k}' \end{pmatrix}. \end{aligned} \quad (31)$$

For clarity, we first consider the simplified case where $\chi_g = \chi_f$ is a subgaussian distribution with parameter σ —this exactly matches our parameter sets **NEV-C3**, **NEV-R-1/2/3** and **NEV-D-1/2**. We then explain how to generalize the analysis to the broader setting. Define

$$\mathbf{v}_y = \mathbf{y}^T (\hat{\mathbf{R}}_1 \dots \hat{\mathbf{R}}_{n/k} \hat{\mathbf{E}}_1 \dots \hat{\mathbf{E}}_{n/k}) \in \mathbb{Z}^{2n}.$$

By concentration inequalities for subgaussian variables [65, 5], we have

$$\Pr[t_y > \kappa \sigma \|\mathbf{v}_y\|] \leq e^{-\kappa^2/2}. \quad (32)$$

For each $y \in \{0, 1\}^k$, we compute $p_{j,y}$ numerically and denote the largest such value by $p_{j,y}^* = \max_y p_{j,y}$. Setting the right-hand side of inequality (32) equal to $p_{j,y}^*$ gives a specific value $\kappa = \kappa^*$ (reported in Table 7).

Note that the adversary can make at most a bounded number N of decryption queries—for example, $N = 2^{80}$ as specified in the NGCC Call for Proposals [44]. To obtain a decryption failure, the adversary must increase $\|\mathbf{v}_y\|$ by a factor c (e.g., by appropriately choosing (r, e) and M). This factor c reduces κ^* by the same factor in inequality (32), so that the resulting failure probability becomes at least $1/N$. Let c^* denote the threshold value satisfying

$$e^{-(\kappa^*/c^*)^2/2} = 2^{-80-\log(n/k)-k}, \quad (33)$$

where the terms $\log(n/k) + k$ account for the union bound over $y \in \{0, 1\}^k$ in equation (4) and over $j \in [n/k]$ in equation (6). This union bound is introduced artificially for the analysis of decryption failures and only relaxes the conditions under which the adversary can trigger a decryption failure within $N = 2^{80}$ queries. The resulting values of c^* for our concrete parameters are listed in Table 7.

Now it suffices to estimate the probability of obtaining a vector $\mathbf{v}_y$ whose norm is c times larger than the expected value. Let

$$\mathbf{v}_{i,y} = \mathbf{y}^T (\hat{\mathbf{R}}_i \hat{\mathbf{E}}_i) \in \mathbb{Z}^{2k}.$$

All $\mathbf{v}_{i,y}$ are identically distributed, and

$$\|\mathbf{v}_y\|^2 = \sum_{i \in \{1, \dots, n/k\}} \|\mathbf{v}_{i,y}\|^2.$$

Let D_y be the distribution of $\|\mathbf{v}_{i,y}\|^2$ with mean μ . Let $M_{D_y}(\theta) = \mathbb{E}_{X \sim D_y} e^{\theta X}$ the moment-generating function of D_y , assumed finite for some $\theta > 0$. Then, by the Chernoff-Cramer inequality [14], we have that

$$\Pr[\|\mathbf{v}_y\|^2 \geq c^2 \cdot n\mu/k] \leq e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k} \quad (34)$$

We perform a numerical computation of $\|\mathbf{v}_{i,y}\|^2$ to determine the distribution (D_y, μ) . Since an adversary may maliciously choose $M \in \{0, 1\}^{n/k}$ to influence the distribution of $\hat{\mathbf{E}}_i$ and thereby increase decryption failures, our numerical computation accounts for the choice of M as well. Specifically, the i -th bit M_i of M affects only the single matrix $\hat{\mathbf{E}}_i$. Hence, we can enumerate all possibilities $M_i \in \{0, 1\}$ and $y \in \{0, 1\}^k$ in the numerical computation. For the subsequent analysis, we fix the pair (M_i, y) that maximizes the probability in equation (34) — this choice represents the most favorable conditions for the adversary’s attack. Interestingly, because **NEV** always XORs M_i with random bits, our numerical results show that when $\chi_e = B_\eta$ with $\eta \geq 2$, the choice of M_i has no effect on the probability in equation (34). Otherwise, $M_i = 0$ is optimal for the adversary, which increases the probability by approximately 2^{20} for **NEV-C3**. This phenomenon coincides with our analysis of decryption failures in Sec. 3.2, where the two distributions in Eq. (5) differ in variance only for $\eta = 1$, and further confirms that **NEV** is robust against malicious message attacks.

Finally, the total cost for the adversary to trigger a single decryption failure is given by

$$\min_{c \geq c^*} \frac{1}{e^{-(\kappa^*/c)^2/2} \cdot e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k}}$$

in the classical setting, and

$$\min_{c \geq c^*} \frac{1}{e^{-(\kappa^*/c)^2/2} \cdot \sqrt{e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k}}}$$

in the quantum setting. Here, we directly assume that Grover’s algorithm yields a square-root speed-up for quantum search, which only makes our analysis more conservative.

Now, we show how to extend the above analysis for the case $\chi_g \neq \chi_f$. Formally, let χ_g and χ_f be subgaussian distributions with parameter σ_g and σ_f , respectively. By the properties of subgaussian distributions, we have $\frac{\sigma_f}{\sigma_g}\chi_g$ is subgaussian with parameter σ_f . Using the identity

$$\begin{aligned} t_y &= \left\langle \sum_{i \in [n/k]} (\hat{\mathbf{R}}_i \hat{\mathbf{g}}_i + \hat{\mathbf{E}}_i \hat{\mathbf{f}}'_i), \mathbf{y} \right\rangle \\ &= \mathbf{y}^T \left(\frac{\sigma_g}{\sigma_f} \hat{\mathbf{R}}_1, \dots, \frac{\sigma_g}{\sigma_f} \hat{\mathbf{R}}_{n/k}, \hat{\mathbf{E}}_1, \dots, \hat{\mathbf{E}}_{n/k} \right) \begin{pmatrix} \frac{\sigma_f}{\sigma_g} \hat{\mathbf{g}}_1 \\ \vdots \\ \frac{\sigma_f}{\sigma_g} \hat{\mathbf{g}}_{n/k} \\ \hat{\mathbf{f}}'_1 \\ \vdots \\ \hat{\mathbf{f}}'_{n/k} \end{pmatrix}, \end{aligned}$$

we can handle this general case naturally by instead performing a numerical computation of $\|\mathbf{v}'_{i,y}\|^2$, where

$$\mathbf{v}'_{i,y} = \mathbf{y}^t \left(\frac{\sigma_g}{\sigma_f} \hat{\mathbf{R}}_i \hat{\mathbf{E}}_i \right) \in \mathbb{Z}^{2k}.$$

We develop a Python script to compute the adversary’s optimal cost—attained at $c = c^*$ for all parameter sets—and report the resulting values in Table 7. In this table, θ is the constant appearing in the Chernoff–Cramér inequality, μ^* denotes the optimal μ for the adversary’s choice of $M_i \in \{0, 1\}$, and C and Q represent the minimum classical and quantum security bits, respectively. These results confirm that our concrete parameters maintain the claimed security levels against attacks that exploit decryption failures.

8.6 Attacks against Symmetric Primitives

The overall security of NEV relies not only on the hardness of the underlying mathematical problems, but also closely on the security of several symmetric primitives it invokes, such as collision-resistant hash functions and pseudorandom number generators. In Sec. 5.3, we instantiate these primitives with SHA3. This instantiation rests on a clear security assumption: SHA3 provides the required collision resistance, preimage resistance, and pseudorandomness at the level matching our concrete security parameter in both the classical and quantum setting.

Table 7: Cost to trigger a single decryption failure within 2^{80} decryption queries

Parameters	Dec. Failure	$p_{f,y}^*$	κ^*	c^*	μ^*	θ	Costs (C,Q)
NEV-C1	2^{-161}	2^{-170}	15.35	1.37	37.47	0.061	(288, 184)
NEV-R1	2^{-134}	2^{-145}	14.18	1.26	48	0.024	(151, 115)
NEV-D1	2^{-136}	2^{-147}	14.28	1.27	112	0.009	(149, 114)
NEV-C2	2^{-162}	2^{-173}	15.49	1.37	21.37	0.059	(324, 202)
NEV-R2	2^{-158}	2^{-170}	15.35	1.36	32	0.05	(369, 224)
NEV-D2	2^{-217}	2^{-229}	17.82	1.58	64	0.028	(707, 393)
NEV-C3	2^{-163}	2^{-175}	15.58	1.37	18.74	0.152	(1098, 589)
NEV-R3	2^{-147}	2^{-160}	14.89	1.31	26.67	0.056	(542, 311)
NEV-D3	2^{-301}	2^{-313}	20.83	1.84	48	0.055	(2994, 1537)

Crucially, **NEV** is designed and proven secure by treating these symmetric primitives as idealised black boxes throughout the security reduction. This means that the proof does not depend on the internal details or specific algebraic structure of SHA3, but only on its external input-output behaviour and the expected security metrics. Such a highly modular abstraction offers strong cryptographic agility: in the future, should any major cryptanalytic breakthrough against SHA3 emerge—for instance, collision or preimage attacks significantly more efficient than anticipated—that would undermine the prescribed security strength in either the classical or quantum setting, we would not need to redesign the core public-key framework of **NEV** nor invalidate its existing security reduction. In practice, one would simply replace SHA3 with another well-analysed, standardised algorithm that provides an equivalent interface, and the overall scheme architecture and core logic would remain unchanged, thereby swiftly restoring and maintaining the actual security margin of the scheme.

9 Side-Channel Vulnerabilities and Countermeasures

To the best of our knowledge, existing design and implementation of **NEV-KEM-CCA** and **NEV-PKE-CCA** follow a constant-time design paradigm, where the execution time is independent of secret-dependent values. Consequently, timing-based side-channel leakage is not expected to be exploitable in practice.

Regarding physical side channels such as power consumption and electromagnetic leakage, although there exist constructions that aim to achieve leakage resilience, they often incur prohibitive overhead and are therefore impractical for real-world deployments. Instead, we argue that having a clear and systematic understanding of the inherent side-channel vulnerabilities of the algorithm, together with targeted and efficient countermeasures, is of primary importance for achieving practical security.

NEV-KEM-CCA and **NEV-PKE-CCA** share similar structures in their KeyGen and Decap (resp., Dec) procedures. As a result, they exhibit essentially the same

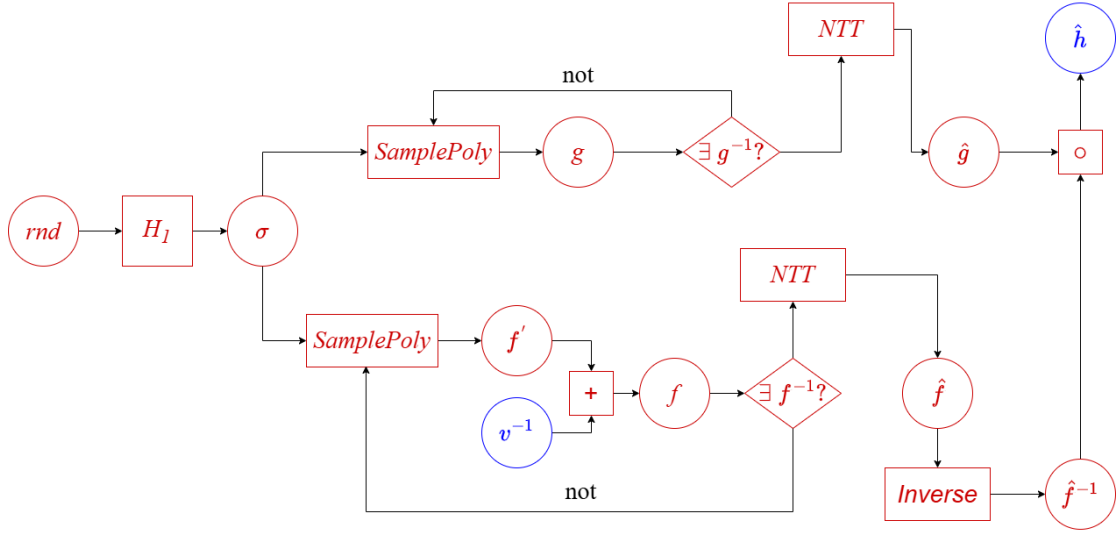


Fig. 14: NEV-KEM-CCA key-generation procedure and side-channel vulnerabilities

side-channel leakage behaviors, and admit equivalent countermeasure designs. To avoid redundant exposition, we present our analysis only for NEV-KEM-CCA, and the same reasoning applies to NEV-PKE-CCA in a straightforward manner.

9.1 Security Analysis

9.1.1 Key Generation Phase. In NEV-KEM-CCA, sensitive information mainly includes the secret key sk and its derived intermediate variables during computation. Among them, the secret key vector sk is the most critical confidential data, whose security directly determines the overall security of the scheme. Figure 14 illustrate NEV-KEM-CCA key generation flow, where red denotes sensitive information and operations, and blue denotes non-sensitive information.

During key generation, two polynomials f' and g are first randomly sampled and checked for invertibility. Since both f' and g are sensitive, the sampling procedure and invertibility testing must be protected. Subsequently, these polynomials are transformed into the NTT domain, where polynomial inversion is performed. Therefore, both the NTT transformation and inversion operations in the NTT domain require protection.

9.1.2 Decapsulation Phase. The decapsulation procedure in NEV-KEM-CCA consists of decryption, re-encryption, and ciphertext comparison. The secret key sk and its derived variables remain sensitive throughout the computation. Figure 15 illustrate NEV-KEM-CCA decapsulation flow, where red denotes sensitive information and operations, and blue denotes non-sensitive information.

In the decryption stage, the secret key sk is first multiplied with the ciphertext c in the NTT domain. The result is then transformed back to the arithmetic domain, and the plaintext is recovered via the Noise2Pt algorithm.

In the re-encryption stage, the decrypted plaintext is hashed together with public information $H_1(pk)$ using H_2 to derive a shared key K' and a sampling

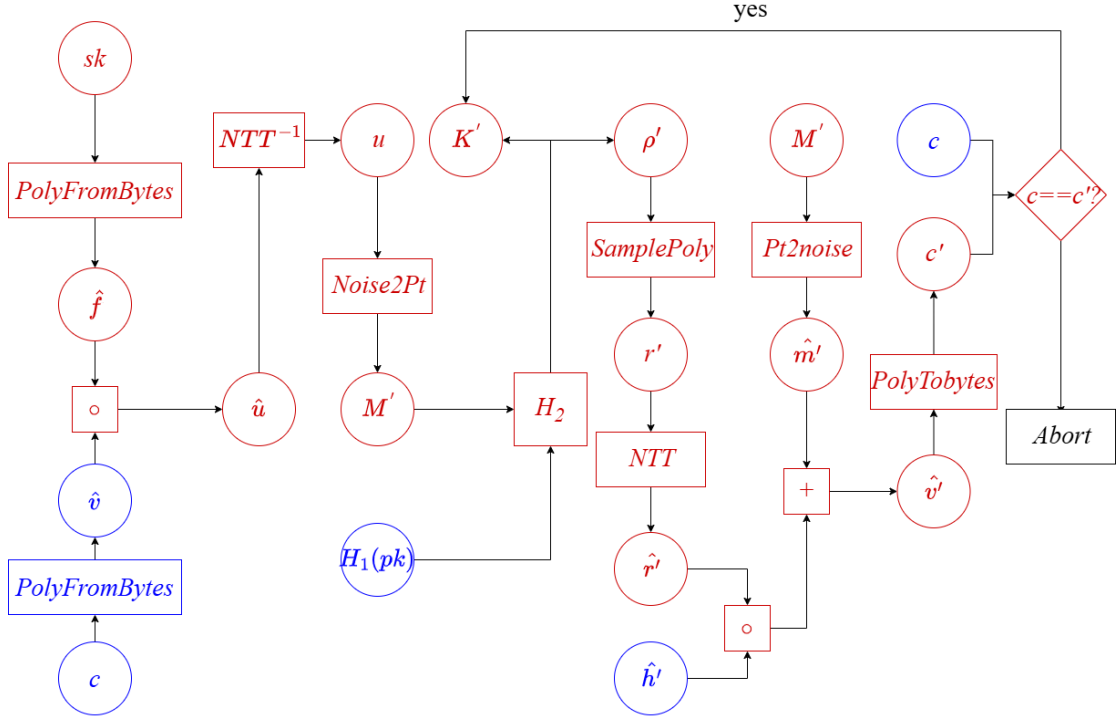


Fig. 15: NEV-KEM-CCA decapsulation procedure and side-channel vulnerabilities.

seed ρ' . Then, a random polynomial r' is generated from ρ' and multiplied with the public key in the NTT domain. The plaintext is mapped to a polynomial via Pt2Noise, and added to obtain the re-encrypted ciphertext c' .

Finally, c' is compared with the input ciphertext c . If they match, the shared key K' is returned; otherwise, the algorithm aborts.

Throughout the decapsulation process, the following components require protection:

- NTT operations,
- Noise2Pt,
- SamplePoly,
- Pt2Noise,
- Ciphertext comparison.

9.2 A Discussion on Masking Countermeasures

In the propagation paths of sensitive information in NEV, linear operations such as polynomial addition and multiplication in the NTT domain can be computed in a share-wise manner.

In contrast, nonlinear operations, including polynomial sampling, polynomial inversion, invertibility testing, Noise2Pt, Pt2Noise, and ciphertext comparison, constitute the main challenges for masking and require dedicated secure designs. In the remainder of this subsection, we denote by d the masking order.

9.2.1 Polynomial Sampling. The polynomial sampling procedure in NEV first generates random bytes using a pseudorandom generator (PRG), then produces polynomials following a binomial distribution via the CBD algorithm, and finally generates polynomials with specific distributions using the Ternary algorithm.

Masked PRG. The PRG is implemented using the Keccak algorithm. Masking for Keccak has been extensively studied in the literature. Existing masking schemes for Keccak are well developed, and their main computational overhead is typically dominated by the nonlinear layer, resulting in a cost of $O(d^2)$ for a masking order of d .

Masked CBD Function. The CBD_η function maps random bits to polynomial coefficients following a discrete Gaussian-like distribution. The standard method computes $f_i = a - b$, where a and b are Hamming weights of bit partitions (say, x and y). The main challenge of Masked CBD Function is the conversion from Boolean-masked inputs to arithmetic outputs.

A straightforward approach is proposed in [57]. It first applies multiple Boolean-to-Arithmetic (B2A) conversions to map the input into the arithmetic domain, where the Hamming weight is subsequently computed. More precisely, for a k -bit Boolean-masked input with two shares (x_1, x_2) , the bits are processed individually. For each bit position j , a one-bit B2A conversion is performed to transform the j -th bit from Boolean masking to arithmetic masking, yielding arithmetic shares. The masked Hamming weight is then obtained by summing over all k bit positions. This approach requires $2k$ B2A calls and incurs a complexity of $O(kd^2)$. A more efficient method is introduced in [62]. Leveraging a bit-slicing technique, it computes the Hamming weight directly in the Boolean domain, and performs only a single B2A conversion at the end, with input bit-width $\lambda = \log_2(k + 1) + 1$, to convert the result into the arithmetic domain. This approach achieves a reduced complexity of $O(d^2 \log_2 k)$.

Masked Ternary Function. The Ternary function first invokes the PRG and CBD functions. Since the masking strategies for these components have already been given, we only need to replace the AND operation with masked multiplication. For masked multiplication [45] proposed an NI-secure masked multiplication scheme with complexity $O(d^2)$, requiring $\frac{d(d-1)}{2}$ random values and no additional storage. The Ternary function invokes AND a total of $\frac{N}{8} \cdot \frac{N}{8}$ times, and thus the total complexity of masked multiplication is $O(N^2 d^2)$. The CBD module inside the Ternary function has been discussed in previous parts. In general, the cost overhead is approximately $O(d^2)$.

9.2.2 Masked Pt2Noise Module. The Pt2Noise module converts a plaintext bit string M of length $\ell = n/k$ to be encoded into a polynomial m over the ring $\mathbb{Z}_q[x]/(x^n + 1)$ whose coefficients follow a centered binomial distribution. More specifically, the algorithm consists of the following three stages:

1. Use XOR random masking to mix the plaintext. This stage consists entirely of linear operations.
2. Randomize and encode the plaintext bit matrix. This part partitions and permutes the bit stream block by block and is also linear.
3. Generate the plaintext polynomial m following a centered binomial distribution. This part corresponds to the CBD module. This function inside the **Ternary** function has been discussed in previous parts. In general, the cost overhead is approximately $O(d^2)$.

In general, the cost overhead is approximately $O(d^2 \log_2 k)$.

9.2.3 Masked Noise2Pt Module. The Noise2Pt module converts a polynomial $w = w_0 + w_1x + \dots + w_{n-1}x^{n-1}$ whose coefficients follow a centered binomial distribution into a plaintext bit string M of length $\ell = n/k$.

For each coefficient, the procedure consists of three stages:

1. **Centering:** $\tilde{w}_i = w_i - \frac{q+1}{2} \bmod \pm q$,
2. **Aggregation:** $t_j = \sum_{i \equiv j \bmod (n/k)} |\tilde{w}_i|$;
3. **Decision:** $M_j = \begin{cases} 1, & t_j < \frac{k(q-1)}{4}, \\ 0, & \text{otherwise.} \end{cases}$

First of all, we consider the masked centering. The centering operation is essentially a conditional reduction: $\tilde{w}_i = \begin{cases} w_i, & w_i < \frac{q}{2}, \\ w_i - q, & \text{otherwise,} \end{cases}$ which is nonlinear

and cannot be implemented directly in the arithmetic masked domain. A practical approach is to convert this decision into the Boolean domain using the most significant bit (MSB): $\text{MSB}(w_i) = \begin{cases} 1, & w_i < \frac{q}{2}, \\ 0, & \text{otherwise} \end{cases}$. For w_i in arithmetic masking

form $w_i = w_i^{(1)} + \dots + w_i^{(d)} \bmod q$, the procedure is as follows:

1. Compute masked shares of $w_i - \frac{q+1}{2}$.
2. Convert to Boolean masking using an Arithmetic-to-Boolean (A2B) conversion (e.g., [62]), with complexity $O(d^2 \log k)$, where k is the bit-length.
3. Extract the MSB in the Boolean domain share-wise.
4. Compute the centered value using masked multiplication: $\tilde{w}_i = w_i \cdot \text{MSB}(w_i) + (w_i - q) \cdot (1 - \text{MSB}(w_i))$.

Therefore, the cost of centering a single coefficient is $O(d^2 \log p)$.

Then, we consider the masked absolute value and aggregation. The computation of $t_j = \sum |\tilde{w}_i|$ consists of following two steps.

- Step 1: Absolute value $|\tilde{w}_i| = \begin{cases} \tilde{w}_i, & \tilde{w}_i < \frac{q}{2}, \\ q - \tilde{w}_i, & \text{otherwise} \end{cases}$ is nonlinear and implemented in the Boolean domain using MSB: $|\tilde{w}_i| = \tilde{w}_i \cdot (1 - \text{MSB}(\tilde{w}_i))$. Note that the A2B conversion computed during centering can be reused here. A masked refreshing step is applied to obtain fresh Boolean shares with cost $O(d^2)$, followed by one masked multiplication with cost $O(d^2)$.

- Step 2: Summation. Since summation is not modulo q , it must be performed in a larger domain $\mathbb{Z}_r$ such that $r > k^2$.
 - First, convert $|\tilde{w}_i|$ from Boolean to arithmetic masking in $\mathbb{Z}_r$ using B2A with overhead complexity of $O(d^2 \log(\log r))$.
 - Then directly perform masked summation $t_j = \sum |\tilde{w}_i|$.
- Thus, the total complexity of aggregation is $O(d^2 \log(\log r))$.

Finally, we consider the masked decision. It is also nonlinear and must be implemented in the Boolean domain. This requires an A2B conversion with complexity $O(d^2 \log(\log r))$ and a Boolean comparison. Therefore, the total complexity of the decision step is dominated by A2B conversion $O(d^2 \log(\log r))$.

9.2.4 Masked Polynomial Inversion Module. Polynomial inversion is performed through a sequence of polynomial multiplications. Each polynomial multiplication can be protected using a standard masked multiplication scheme such as ISW [45]. Concretely, the multiplication of two shared values is carried out securely in the masked domain, with a cost of $O(d^2)$ per multiplication.

In addition, polynomial inversion requires checking whether the coefficients in the NTT domain are zero, in order to determine invertibility. Since these coefficients are secret-shared values over the prime field, this step reduces to a masked zero-testing problem.

9.2.5 Masked Zero-Testing. Let $x \in \mathbb{F}_p$ be a secret value represented in arithmetic masking with d shares:

$$x = x_1 + x_2 + \cdots + x_d \bmod p.$$

The goal is to determine whether $x = 0$, i.e., to compute a Boolean output

$$b = \begin{cases} 1, & x = 0, \\ 0, & x \neq 0, \end{cases}$$

where the output b is not required to be masked.

Coron et al. [21] gives an approach for this problem. The approach iteratively multiplies by random invertible elements u_i and performs linear mask refreshing, thereby transforming the shares into a form satisfying

$$u_1 \cdot \dots \cdot u_d \cdot x = B_1 + \cdots + B_d \bmod q.$$

Afterwards, all shares are added together to reconstruct b . If the original secret satisfies $x = 0$, then $B = 0$; otherwise, B is a uniformly random nonzero value in $\mathbb{Z}_q^*$. Then, one can directly check if B is zero or not, without leaking the value of x . This approach incurs an overhead of complexity $O(d^2)$.

9.2.6 Masked Ciphertext Comparison and Masked Zero-Testing for Multiple Variables. The masked ciphertext comparison can be achieved by applying masked zero-testing for multiple variables. For zero-testing k masked variables, a straightforward approach is to perform masked zero-testing independently k times, which leads to a high computational cost. Instead, we can adopt the method of [12]. The main idea is to construct κ independent random linear combinations of the k variables (denoted as $a_1, \dots, a_k$), and perform zero-testing on these combinations. If all κ linear combinations evaluate to zero, then the probability that all k masked variables are zero increases exponentially with κ (equivalently, the error probability decreases exponentially). Consequently, instead of performing k independent masked zero-tests, it suffices to perform only κ masked zero-tests on the random linear combinations, significantly reducing the computational cost. Compared to ML-KEM, NEV-KEM-CCA does not require protecting the result of the ciphertext comparison $c = c'$. This simplifies the masking design and reduces the overall implementation overhead.

References

1. Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the LWE, NTRU schemes! Cryptology ePrint Archive, Paper 2018/331, 2018.
2. Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
3. Martin R. Albrecht and Yixin Shen. Quantum augmented dual attack. Cryptology ePrint Archive, Paper 2022/656, 2022.
4. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. In *Journal of Mathematical Cryptology*, 9:169–203, oct 2015.
5. Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange—a new hope. In *USENIX Security Symposium 2016*, 2016.
6. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In *Advances in Cryptology – CRYPTO 2019*, pages 269–295. Springer, 2019.
7. Shi Bai and Steven D. Galbraith. Lattice decoding attacks on binary lwe. In Willy Susilo and Yi Mu, editors, *Information Security and Privacy*, pages 322–337, Cham, 2014. Springer International Publishing.
8. Shi Bai, Adeline Langlois, Tancrede Lepoint, Damien Stehlé, and Ron Steinfeld. Improved security proofs in lattice-based cryptography: Using the rényi divergence rather than the statistical distance. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015*, pages 3–24. Springer, 2015.
9. Daniel V. Bailey, Daniel Coffin, Adam Elbirt, Joseph H. Silverman, and Adam D. Woodbury. NTRU in constrained devices. In Çetin K. Koç, David Naccache, and Christof Paar, editors, *CHES 2001*, pages 262–272. Springer, 2001.
10. W. Banaszczyk. New bounds in some transference theorems in the geometry of numbers. *Mathematische Annalen*, 296:625–635, 1993.
11. Mihir Bellare and Phillip Rogaway. Random oracles are practical: a paradigm for designing efficient protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security, CCS '93*, page 62–73, New York, NY, USA, 1993. Association for Computing Machinery.
12. Shivam Bhasin, Jan-Pieter D’Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel Van Beirendonck. Attacking and defending masked polynomial comparison for lattice-based cryptography. *IACR Cryptol. ePrint Arch.*, 2021:104, 2021.
13. J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle. Crystals - Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS P)*, pages 353–367, 2018.
14. Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration inequalities: A nonasymptotic theory of independence*. Oxford University Press, 2013.

15. Zvika Brakerski and Nico Döttling. Lossiness and Entropic Hardness for Ring-LWE. In *Theory of Cryptography Conference*, pages 1–27. Springer, 2020.
16. Jinzheng Cao, Haodong Jiang, and Qingfeng Cheng. Refined attack on lwe with hints: Constructing lattice via gaussian elimination. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 385–416, Cham, 2025. Springer Nature Switzerland.
17. Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In *Advances in Cryptology – CRYPTO 2025*, pages 444–476. Springer, 2025.
18. André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 63–91, Cham, 2021. Springer International Publishing.
19. Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M. Schanck, Tsunekazu Saito, Peter Schwabe, William Whyte, Keita Xagawa, Takashi Yamakawa, and Zhenfei Zhang. NTRU – submission to the NIST post-quantum project, 2019.
20. Yuanmi Chen. Lattice reduction and concrete security of fully homomorphic encryption. *Dept. Informatique, ENS, Paris, France, PhD thesis*, 2013.
21. Jean-Sébastien Coron, François Gérard, Simon Montoya, and Rina Zeitoun. High-order polynomial comparison and masking lattice-based encryption. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(1):153–192, 2023.
22. Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *CRYPTO ’98*, volume 1462 of *LNCS*, pages 13–25. Springer, 1998.
23. Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. LWE with side information: Attacks and concrete security estimation. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020*, pages 329–358. Springer, 2020.
24. Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (one) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020*, pages 3–33, Cham, 2020. Springer.
25. Léo Ducas, Vadim Lyubashevsky, and Thomas Prest. Efficient identity-based encryption over NTRU lattices. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014*, pages 22–41. Springer, 2014.
26. Léo Ducas and Ludo N. Pulles. Does the dual-sieve attack on learning with errors even work? In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 37–69, Cham, 2023. Springer Nature Switzerland.
27. Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, Gregor Seiler, and Dominique Unruh. A thorough treatment of highly-efficient ntru instantiations. In *IACR International Conference on Public-Key Cryptography*, pages 65–94. Springer, 2023.
28. Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, Gregor Seiler, and Dominique Unruh. A thorough treatment of highly-efficient NTRU instantiations. *Cryptology ePrint Archive*, Paper 2021/1352, 2021.
29. Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. The impact of error dependencies on ring/mod-lwe/lwr based schemes. In *International Conference on Post-Quantum Cryptography*, pages 103–115. Springer, 2019.
30. Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. The impact of error dependencies on ring/mod-lwe/lwr based schemes. In *International Conference on Post-Quantum Cryptography*, pages 103–115. Springer, 2019.
31. Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-fourier lattice-based compact signatures over NTRU, 2016.
32. Pierre-Alain Fouque, Paul Kirchner, Thomas Pornin, and Yang Yu. BAT: Small and fast kem over NTRU lattices. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(2):240–265, Feb. 2022.
33. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael Wiener, editor, *CRYPTO’ 99*, pages 537–554. Springer, 1999.
34. Nicolas Gama and Phong Q. Nguyen. New chosen-ciphertext attacks on NTRU. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007*, pages 89–106. Springer, 2007.
35. Jiangxia Ge, Tianshu Shan, and Rui Xue. Tighter qcca-secure key encapsulation mechanism with explicit rejection in the quantum random oracle model. In *Advances in Cryptology – CRYPTO 2023*, pages 292–324. Springer, 2023.
36. Jiangxia Ge, Kang Yang, Yang Yu, and Yu Yu. Fujisaki-okamoto transformation under average-case decryption error: Tighter and more general proofs with applications to pqc. *Cryptology ePrint Archive, Paper 2025/1641*, 2025.

37. Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. FrodoKEM: A CCA-secure learning with errors key encapsulation mechanism. *IACR Communications in Cryptology*, 2(3), 2025.
38. Alex B Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the qrom. In *Advances in Cryptology – ASIACRYPT 2021*, pages 637–667. Springer, 2021.
39. Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving lwe with applications to crystals. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 33–62, Cham, 2021. Springer International Publishing.
40. Jens Hermans, Frederik Vercauteren, and Bart Preneel. Speed records for NTRU. In Josef Pieprzyk, editor, *Topics in Cryptology – CT-RSA 2010*, pages 73–88. Springer, 2010.
41. Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In Joe P. Buhler, editor, *Algorithmic Number Theory*, pages 267–288, Berlin, Heidelberg, 1998. Springer.
42. Jeffrey Hoffstein and Joseph H. Silverman. Optimizations for NTRU. In Joe P. Buhler, editor, *Proc. the Conf. on Public Key Cryptography and Computational Number Theory*, page 77–88. Springer, 2000.
43. Nick Howgrave-Graham, Phong Q. Nguyen, David Pointcheval, John Proos, Joseph H. Silverman, Ari Singer, and William Whyte. The impact of decryption failures on the security of NTRU encryption. In Dan Boneh, editor, *CRYPTO 2003*, pages 226–246. Springer, 2003.
44. China Institute of Commercial Cryptography Standards. Call for proposals for the next-generation public-key cryptographic algorithms, 2025.
45. Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, Lecture Notes in Computer Science, pages 463–481. Springer, 2003.
46. Ravi Kannan. Minkowski’s convex body theorem and integer programming. *Mathematics of Operations Research*, 12(3):415–440, 1987.
47. Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In Aggelos Kiayias, editor, *Topics in Cryptology – CT-RSA 2011*, volume 6558 of *LNCS*, pages 319–339. Springer, 2011.
48. Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *STOC ’12*, pages 1219–1234, 2012.
49. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, 2010.
50. IDF MATZOV. Report on the security of lwe: improved dual lattice attack, 2022. *URL: <https://zenodo.org/record/6412487>*, 2022.
51. Alexander May and Julian Nowakowski. Too many hints—when lll breaks lwe. In *Advances in Cryptology – ASIACRYPT 2023*, pages 106–137. Springer, 2023.
52. Daniele Micciancio and Oded Regev. Lattice-based cryptography. In Daniel J. Bernstein, Johannes Buchmann, and Erik Dahmen, editors, *Post-Quantum Cryptography*, pages 147–191. Springer Berlin Heidelberg, 2009.
53. Phong Q. Nguyen and David Pointcheval. Analysis and improvements of NTRU encryption paddings. In Moti Yung, editor, *CRYPTO 2002*, pages 210–225. Springer, 2002.
54. NIST. FIPS PUB 202: SHA-3 Standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication 202, U.S. Department of Commerce, 2015.
55. NIST. Status report on the second round of the nist post-quantum cryptography standardization process, 2020.
56. NIST. Status report on the third round of the nist post-quantum cryptography standardization process, 2022.
57. Tobias Oder, Tobias Schneider, Thomas Pöppelmann, and Tim Güneysu. Practical cca2-secure and masked ring-lwe implementation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):142–174, 2018.
58. Chris Peikert. An efficient and parallel gaussian sampler for lattices. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 80–97. Springer, 2010.
59. Alice Pellet-Mary and Damien Stehlé. On the hardness of the NTRU problem. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021*, pages 3–35. Springer, 2021.
60. Amaury Pouly and Yixin Shen. Provable dual attacks on learning with errors. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 256–285, Cham, 2024. Springer Nature Switzerland.

61. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC '05*, pages 84–93. ACM, 2005.
62. Tobias Schneider, Clara Paglialonga, Tobias Oder, and Tim Güneysu. Efficiently masking binomial sampling at arbitrary orders for lattice-based crypto. In Dongdai Lin and Kazue Sako, editors, *Public-Key Cryptography - PKC 2019 - 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14-17, 2019, Proceedings, Part II*, Lecture Notes in Computer Science, pages 534–564. Springer, 2019.
63. Damien Stehlé and Ron Steinfeld. Making NTRU as secure as worst-case problems over ideal lattices. In Kenneth Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 27–47. Springer, 2011.
64. Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. Efficient public key encryption based on ideal lattices. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 617–635. Springer, 2009.
65. Roman Vershynin. Introduction to the non-asymptotic analysis of random matrices. In Yonina C. Eldar and Gitta Kutyniok, editors, *Compressed Sensing: Theory and Applications*, pages 210–268. Cambridge University Press, 2012.
66. Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In *Annual International Cryptology Conference*, pages 239–268. Springer, 2019.
67. Jiang Zhang, Dengguo Feng, and Di Yan. Nev: Faster and smaller ntru encryption using vector decoding. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 157–189, Singapore, 2023. Springer Nature Singapore.
68. Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020*, pages 37–65. Springer, 2020.
69. Yiming Zhu, Zhen Liu, and Yanbin Pan. When ntt meets karatsuba: Preprocess-then-ntt technique revisited. In Debin Gao, Qi Li, Xiaohong Guan, and Xiaofeng Liao, editors, *Information and Communications Security*, pages 249–264. Springer, 2021.

Appendix A

A.1 Proof of Theorem 7

Proof. To prove this theorem, we introduce a sequence of games as shown in Fig. 16. In these games, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc for notational simplicity.

GAMES $\mathbf{G_0-G_4}$	$F(x_1, x_2)$ // $\mathbf{G_1-G_4}$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H_1 \leftarrow \text{Fcs}[\kappa]$ $H_2 \leftarrow \text{Fcs}[2\kappa, 2\kappa]$ $G \leftarrow \text{Fcs}[2\kappa, \kappa]$	// $\mathbf{G_0-G_4}$ 1. if $\exists y$ s.t. $((x_1, x_2), y) \in L_F$, return y // $\mathbf{G_0-G_4}$ 2. else $y \leftarrow \{0, 1\}^\kappa$, $L_F := L_F \cup ((x_1, x_2), y)$ // $\mathbf{G_0-G_4}$ return y // $\mathbf{G_0-G_4}$
2. $M^* \leftarrow \{0, 1\}^\kappa$	// $\mathbf{G_0-G_4}$ $\text{NEV-KEM-CCA.Decap}(sk, c)$ // $\mathbf{G_0-G_1}$
3. $(K_0^*, \rho^*) := H_2(M^*, H_1(pk))$ $K_0^* := G(M^*, H_1(pk))$ $\rho^* := F(M^*, H_1(pk))$ $K_0^* \leftarrow \{0, 1\}^\kappa$ $\rho^* \leftarrow \{0, 1\}^\kappa$	// $\mathbf{G_0}$ 1. if $c = c^*$, return $\perp$ // $\mathbf{G_1-G_3}$ else $M := \text{NEV-PKE-OW.Dec}(sk, c)$ // $\mathbf{G_1-G_3}$ 2. if $M = \perp$, return $\perp$ // $\mathbf{G_4}$ else $(K, \rho) := H_2(M, H_1(pk))$ // $\mathbf{G_0}$ // $\mathbf{G_4}$ else $K := G(M, H_1(pk))$ // $\mathbf{G_1}$ // $\mathbf{G_4}$ $\rho := F(M, H_1(pk))$ // $\mathbf{G_1}$
4. $c^* := \text{NEV-PKE-OW.Enc}(pk, M^*; \rho^*)$ $b \leftarrow \{0, 1\}$ $K_1^* \leftarrow \{0, 1\}^\kappa$	// $\mathbf{G_0-G_4}$ 3. if $c = \text{NEV-PKE-OW.Enc}(pk, M; \rho)$ // $\mathbf{G_0-G_4}$ return K // $\mathbf{G_0-G_4}$ else return $\perp$
5. $b' \leftarrow \mathcal{A}^{H_1, H_2, \text{NEV-KEM-CCA.Decap}}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_1^{F, G, H_2, \text{NEV-KEM-CCA.Decap}}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_1^{F, G, H_2, \text{Decap}^*}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_2^{F, G, H_1}(pk, c^*, K_b^*)$	// $\mathbf{G_0}$ Decap $^*(pk, c)$ // $\mathbf{G_2}$ // $\mathbf{G_1}$ // $\mathbf{G_2}$ // $\mathbf{G_3-G_4}$ 1. if $c = c^*$, return $\perp$ // $\mathbf{G_0-G_4}$ else $x := \text{Search}(pk, c, L_F)$ // $\mathbf{G_0-G_4}$ 2. if $x \neq \perp$, return $G(x, H_1(pk))$ // $\mathbf{G_0-G_4}$ else return $\perp$
6. return $[b' = b]$	

Fig. 16: Games $\mathbf{G_0}$ to $\mathbf{G_4}$ in the proof of Theorem 7. The record lists L_F used here is an empty set at the beginning of the game.

Game $\mathbf{G_0}$: This game is the IND-CCA security game of NEV-KEM-CCA with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-KEM-CCA}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G_0}] - \frac{1}{2} \right|. \quad (35)$$

Game $\mathbf{G_1}$: Compared with game $\mathbf{G_0}$, this game has the following three differences:

1. Two new random oracles F and G are introduced, and F is simulated on-the-fly by using the record list L_F .
2. When computing (c^*, K_b^*) and $\text{NEV-KEM-CCA.Decap}(sk, c)$, the $H_2(x_1, x_2)$ is replaced with $(G(x_1, x_2), F(x_1, x_2))$, where $G(x_1, x_2)$ is the (potential) session key and $F(x_1, x_2)$ is the randomness used by NEV-PKE-OW.Enc .

3. The adversary $\mathcal{A}$ is replaced with $\mathcal{A}_1$, which runs $\mathcal{A}$ and outputs its result. In particular, for $\mathcal{A}$'s H_2 query (x_1, x_2) , $\mathcal{A}_1$ simulates it by querying F and G and returns $(G(x_1, x_2), F(x_1, x_2))$ to $\mathcal{A}$.

One can note that $\mathcal{A}_1$ makes q_{H_1} queries to H_1 , q_{H_2} queries to both F and G , and q_D queries to NEV-KEM-CCA.Decap .

According to the basic rules of random oracle, game hop $\mathbf{G}_0$ to $\mathbf{G}_1$ is only a conceptual change, thus we have

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \mathbf{G}_1]. \quad (36)$$

Game $\mathbf{G}_2$: This game is the same as $\mathbf{G}_1$, except that the NEV-KEM-CCA.Decap is replaced by Decap^* , a new decapsulation oracle does not use the secret key sk . For a decapsulation query c , Decap^* only uses the public key pk to compute $x := \text{Search}(pk, c, L_F)$, and then returns $G(x, H_1(pk))$ if $x \neq \perp$. The function Search used here is defined as

$$\text{Search}(pk, c, L_F) := \begin{cases} M & M \text{ is the minimum value s.t. } L_F(M, H_1(pk)) \neq \perp \\ & \wedge c = \text{NEV-PKE-OW.Enc}(pk, M; L_F(M, H_1(pk))), \\ \perp & \text{if such } M \text{ does not exist.} \end{cases} \quad (37)$$

By the definitions of the oracles NEV-KEM-CCA.Decap and Decap^* , their outputs can only differ in the following three cases:

1. The decapsulation query c and the record list L_F just before such query satisfy $c \neq c^* \wedge \text{Search}(pk, c, L_F) \neq \perp$, but $\text{NEV-PKE-OW.Dec}(sk, c)$ equals $\perp$ or $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$ and

$$c \neq \text{NEV-PKE-OW.Enc}(pk, M; F(M, H_1(pk))).$$

In this case, $\text{NEV-KEM-CCA.Decap}(sk, c) = \perp$ but $\text{Decap}^*(pk, c) \neq \perp$.

2. The decapsulation query c and the record list L_F just before such query satisfy $c \neq c^* \wedge \text{Search}(pk, c, L_F) = \perp$, but $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$ and

$$c = \text{NEV-PKE-OW.Enc}(pk, M; F(M, H_1(pk))).$$

In this case, $\text{NEV-KEM-CCA.Decap}(sk, c) \neq \perp$ but $\text{Decap}^*(pk, c) = \perp$.

3. The decapsulation query c and the record list L_F just before such query satisfy $c \neq c^* \wedge \text{Search}(pk, c, L_F) \neq \perp$, $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$ and

$$c = \text{NEV-PKE-OW.Enc}(pk, M; F(M, H_1(pk))),$$

but $\text{Search}(pk, c, L_F) \neq M$. In this case, neither $\text{NEV-KEM-CCA.Decap}(sk, c)$ nor $\text{Decap}^*(pk, c)$ equals $\perp$ but these two values are different.

Based on the above three cases, we define the following two classical events **Error** and **Guess** that may occur in games $\mathbf{G}_1$ and $\mathbf{G}_2$.

- **Error:** $\mathcal{A}_1$ ever makes a decapsulation query that triggers the above case 1 or 3.

- **Guess:** $\mathcal{A}_1$ ever makes a decapsulation query that triggers the above case 2.

At this point, it is obvious that games $\mathbf{G}_1$ and $\mathbf{G}_2$ proceed identically if the event $\text{Error} \vee \text{Guess}$ does not occur. Hence, by the fundamental lemma, we have

$$|\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| \leq \Pr[\text{Error} : \mathbf{G}_2] + \Pr[\text{Guess} : \mathbf{G}_2]. \quad (38)$$

We first consider the event **Guess**. In fact, if a decapsulation query c makes **Guess** occurs, we have $\text{Search}(pk, c, L_F) = \perp$, $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$, and $c = \text{NEV-PKE-OW.Enc}(pk, M; F(M, H_1(pk)))$, where L_F is the record list just before the decapsulation query c . Now we can conclude that $L_F(M, H_1(pk)) = \perp$, otherwise by the basic rules of the on-the-fly simulation we have $F(M, H_1(pk)) = L_F(M, H_1(pk))$ and thus $c = \text{NEV-PKE-OW.Enc}(pk, M; L_F(M, H_1(pk)))$, which contradicts with $\text{Search}(pk, c, L_F) = \perp$ according to the definition of **Search** given in Eq. (37). Since $L_F(M, H_1(pk)) = \perp$ means $F(M, H_1(pk))$ is a uniformly random value over $\{0, 1\}^\kappa$, the probability of $c = \text{NEV-PKE-OW.Enc}(pk, M; F(M, H_1(pk)))$ does not exceed

$$\max_{M, c} \Pr[c = \text{NEV-PKE-OW.Enc}(pk, M; \rho) : \rho \leftarrow \{0, 1\}^\kappa].$$

Recall that $\mathcal{A}_1$ makes q_D decapsulation queries, by the union bound and Lemma 4, we have

$$\Pr[\text{Guess} : \mathbf{G}_2] \leq q_D \cdot \epsilon, \quad (39)$$

where $\epsilon = \max_{r_0 \in R} \Pr[r = r_0 : r \leftarrow \chi_r]$.

As for the event **Error**, we first consider a fixed (pk, sk) pair. It can be observed that **Error** must imply $\mathcal{A}_1$ has queried F with $(M, H_1(pk))$ such that $c = \text{NEV-PKE-OW.Enc}(pk, M; L_F(M, H_1(pk)))$, but $\text{NEV-PKE-OW.Dec}(sk, c) \neq M$. Since F is simulated on-the-fly and is queried at most q_{H_2} times by $\mathcal{A}_1$, by the union bound and denoting

$$\delta_{pk, sk} = \max_M \Pr_{\rho \leftarrow \{0, 1\}^\kappa} [\text{NEV-PKE-OW.Dec}(sk, \text{NEV-PKE-OW.Enc}(pk, M; \rho)) \neq M],$$

we can conclude that

$$\Pr[\text{Error} : \mathbf{G}_2, (pk, sk)] \leq q_{H_2} \cdot \delta_{pk, sk}.$$

Here $\Pr[\text{Error} : \mathbf{G}_2, (pk, sk)]$ denotes the probability of **Error** in game $\mathbf{G}_2$ under a fixed (pk, sk) pair. Then by averaging over $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ and using the fact that **NEV-PKE-OW** has decryption failure rate δ , we have

$$\Pr[\text{Error} : \mathbf{G}_2] \leq q_{H_2} \cdot \delta. \quad (40)$$

Game $\mathbf{G}_3$: This game is the same as $\mathbf{G}_2$, except that $\mathcal{A}_1$ is replaced with a new adversary $\mathcal{A}_2$ that does not query the decapsulation oracle. Here $\mathcal{A}_2$ runs $\mathcal{A}_1$, outputs its final result, and simulates F , G , H_1 , Decap^* for $\mathcal{A}_1$ as follows:

1. For the F queries, $\mathcal{A}_2$ replies by querying its own oracle F . Meanwhile, $\mathcal{A}_2$ sets up a record list $L_{\mathcal{A}, F}$ to record these queries and the corresponding responses.

2. For the G (resp. H_1) queries, $\mathcal{A}_2$ replies by querying G (resp. H_1).
3. For the decapsulation query c , $\mathcal{A}_2$ returns $\perp$ if $c = c^*$. Otherwise, $\mathcal{A}_2$ computes $x := \text{Search}(pk, c, L_{\mathcal{A},F})$. If $x \neq \perp$, $\mathcal{A}_2$ returns $G(x, H_1(pk))$; if $x = \perp$, $\mathcal{A}_2$ returns $\perp$.

By the definition of Search given in Eq. (37), $\mathcal{A}_2$ queries F , G and H_1 at most q_{H_2} , $q_D + q_{H_2}$ and q_{H_1} times, respectively.

In game $\mathbf{G}_2$, the function Search used by Decap^* will never return M^* since c^* is forbidden from being used to query Decap^* . That is to say, the function Search used by Decap^* will only search the list that records the F queries made by $\mathcal{A}_1$, i.e., the list $L_{\mathcal{A},F}$ introduced above. This means that game $\mathbf{G}_3$ is actually an equivalent rewrite of game $\mathbf{G}_2$, and thus we have

$$\Pr[1 \leftarrow \mathbf{G}_2] = \Pr[1 \leftarrow \mathbf{G}_3]. \quad (41)$$

Game $\mathbf{G}_4$: This game is the same as $\mathbf{G}_3$, except that the ρ^* and K^* used to generate c^* are both replaced with fresh random values.

Denote Bad_{M^*} as the event that the adversary $\mathcal{A}_2$ ever queries F or G with $(M^*, *)$. It is not hard to see that game $\mathbf{G}_3$ is identical to game $\mathbf{G}_4$ if Bad_{M^*} does not occur. Hence, by the fundamental difference lemma, we have

$$|\Pr[1 \leftarrow \mathbf{G}_3] - \Pr[1 \leftarrow \mathbf{G}_4]| \leq \Pr[\text{Bad}_{M^*} : \mathbf{G}_4]. \quad (42)$$

Since the bit b is independent of $\mathcal{A}_2$'s view in game $\mathbf{G}_4$, we have

$$\Pr[1 \leftarrow \mathbf{G}_4] = \frac{1}{2}. \quad (43)$$

In the following, we define a new adversary $\mathcal{B}$ against the OW-CPA security of NEV-PKE-OW as follows:

- After getting the public key pk and challenge ciphertext c^* , $\mathcal{B}$ samples $b \leftarrow \{0, 1\}$, $K_0^*, K_1^* \leftarrow \{0, 1\}^\kappa$ and runs $\mathcal{A}_2^{F,G,H_1}(pk, c^*, K_b^*)$ in game $\mathbf{G}_4$. $\mathcal{B}$ random chooses i from $\{1, \dots, q_D + 2q_{H_2}\}$ and then outputs the first part of $\mathcal{A}_2$'s i -th F/G queries.

The above definition implies that, if Bad_{M^*} occurs, $\mathcal{B}$ will output M^* with probability at least $1/(q_D + 2q_{H_2})$. Thus,

$$\begin{aligned} \Pr[\text{Bad}_{M^*} : \mathbf{G}_4] &\leq (q_D + 2q_{H_2}) \cdot \text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{B}) \\ &\stackrel{(a)}{=} (q_D + 2q_{H_2}) \cdot \left(\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2) \right). \end{aligned} \quad (44)$$

Here (a) follows from Theorem 2. By combining Eq. (35), Eq. (36), and Eq. (38) to Eq. (44), we finally obtain

$$\begin{aligned} \text{Adv}_{\text{NEV-KEM-CCA}}^{\text{IND-CCA}}(\mathcal{A}) &\leq (q_D + 2q_{H_2}) \cdot \left(\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2) \right) \\ &\quad + q_D \cdot \epsilon + q_{H_2} \cdot \delta. \end{aligned}$$

This completes the proof of Theorem 7. $\square$

A.2 Proof of Theorem 8

Before proving this theorem, we first introduce the compressed standard oracle proposed by Zhandry [66], and then introduce some lemmas based on it. These lemmas will be used in later proofs.

A.2.1 Compressed Standard Oracle

Definition 8 (Database). Define $\mathcal{X} := \{0, 1\}^m$ and $\mathcal{Y} := \{0, 1\}^n$. Introduce a symbol $\perp$ that $\perp \notin \mathcal{X} \cup \mathcal{Y}$. Then a database D is an ordered list of q -pairs as:

$$((x_1, y_1), (x_2, y_2), \dots, (x_l, y_l), (\perp, 0^n), \dots, (\perp, 0^n)),$$

where $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ for $i \in [l]$, $x_1 < \dots < x_l$, and all $(\perp, 0^n)$ pairs are at the end of the list. The database set $\mathbf{D}_q$ contains all the above databases.

For an $x \in \mathcal{X}$, we write $D(x) = y$ if there exists a $y \in \mathcal{Y}$ s.t. $(x, y) \in D$, otherwise we write $D(x) = \perp$. Let $n(D)$ denote the number of (x, y) pairs in the database D such that $D(x) \neq \perp$. For any pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ and any database D which satisfies $D(x) = \perp$ and $n(D) < q$, let $D \cup (x, y)$ be the updated database which first deletes a pair $(\perp, 0^n)$ and then inserts (x, y) appropriately into D to maintain its order. Conversely, for any pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ and any database D which satisfies $D(x) = y$, let $D \setminus (x, y)$ denote the new database which first removes the pair (x, y) from D and then adds a pair $(\perp, 0^n)$ at the end.

Denote $\mathbf{D}_q$ as the quantum register defined over the database set $\mathbf{D}_q$, that is, $\mathbf{D}_q$ is the Hilbert space with orthonormal basis $\{|D\rangle\}_{D \in \mathbf{D}_q}$, where each basis state is labeled by an element in $\mathbf{D}_q$. In fact, this basis is also called the computational basis by the basic quantum computation. In the following, we refer to $\mathbf{D}_q$ as the database register. Define a superposition of the database state as

$$|D \cup (x, \beta_r)\rangle := \frac{1}{\sqrt{2^n}} \sum_{y \in \{0, 1\}^n} (-1)^{y \cdot r} |D \cup (x, y)\rangle, \quad (r \in \{0, 1\}^n).$$

Then for any $x \in \mathcal{X}$, we define the local decompression procedure StdDecomp_x to act on $\mathbf{D}_q$ as follows:

- For $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) < q$, $\text{StdDecomp}_x |D\rangle = |D \cup (x, \beta_{0^n})\rangle$.
- For $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) < q$, $\text{StdDecomp}_x |D \cup (x, \beta_{0^n})\rangle = |D\rangle$ and $\text{StdDecomp}_x |D \cup (x, \beta_r)\rangle = |D \cup (x, \beta_r)\rangle$ for $r \neq 0^n$.
- For $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) = q$, $\text{StdDecomp}_x |D\rangle = |D\rangle$.

It is obvious that StdDecomp_x is an involution, since applying it twice results in the identity operation. Meanwhile, it is easy to check that

$$\text{StdDecomp}_x |D \cup (x, y)\rangle = |D \cup (x, y)\rangle + \frac{1}{\sqrt{2^n}} |D\rangle - \frac{1}{\sqrt{2^n}} |D \cup (x, \beta_{0^n})\rangle \quad (45)$$

for any $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) < q$, which is a very useful property.

Definition 9 (Compressed Standard Oracle). Let X (resp. Y) denote the input (resp. output) register defined over $\mathcal{X}$ (resp. $\mathcal{Y}$). Let $|D^\perp\rangle$ be the initial state of the database register D_q , where $D^\perp$ is the database containing q $(\perp, 0^n)$ pairs. A quantum query to the compressed standard oracle is simulated by performing the following unitary operation on the registers XYD_q .

$$\text{CStO} := \sum_{x \in \mathcal{X}} |x\rangle\langle x|_X \otimes \text{StdDecomp}_x \circ \text{CNOT}^x \circ \text{StdDecomp}_x.$$

Here $\text{CNOT}^x|y, D\rangle_{YD_q} = |y \oplus D(x), D\rangle_{YD_q}$ if $D(x) \neq \perp$ and $|y, D\rangle_{YD_q}$ otherwise.

Lemma 5 ([66, Lemma 4]). For any adversary $\mathcal{A}$ making at most q oracle queries, the compressed standard oracle as defined in Definition 9 and the quantum random oracle $H : \mathcal{X} \rightarrow \mathcal{Y}$ are perfectly indistinguishable.

Let f be an arbitrary function with domain $\mathcal{X} \times D_q$ and codomain $\mathcal{Y}$, where $\mathcal{X}$ and $\mathcal{Y}$ are finite sets and D_q is the database set defined in Definition 8. Then we call the following unitary operation Read_f a database read.

$$\text{Read}_f : |x, D, y\rangle_{XD_qY} \mapsto |x, D, y + f(x, D)\rangle_{XD_qY}. \quad (46)$$

Here X/Y is defined over $\mathcal{X}/\mathcal{Y}$, D_q is the database register, and $+: \mathcal{Y} \times \mathcal{Y} \rightarrow \mathcal{Y}$ is a group operation over $\mathcal{Y}$. In fact, a key property of the database read operation Read_f is that it does not change the database state in the computational basis.

Definition 10 (Compressed Semi-Classical Oracle [35]). Let $S \subseteq D_q$. Define function f_S such that $f_S(D) = 1$ if $D \in S$ and $f_S(D) = 0$ otherwise. The compressed semi-classical oracle $\mathcal{O}_S^{\text{CSC}}$ performs the following operation on the input state $\sum_{z \in \{0,1\}^*, D \in D_q} \alpha_{z,D} |z, D\rangle$:

1. Initialize a single qubit B with $|0\rangle_B$, transform $\sum_{z \in \{0,1\}^*, D \in D_q} \alpha_{z,D} |z, D\rangle |0\rangle_B$ into $\sum_{z \in \{0,1\}^*, D \in D_q} \alpha_{z,D} |z, D\rangle |f_S(D)\rangle_B$.
2. Measure B and output the measurement outcome.

During the execution of a quantum oracle algorithm $\mathcal{A}^{\mathcal{O}_S^{\text{CSC}}}$, we denote **Find** as the classical event that the measurement of $\mathcal{O}_S^{\text{CSC}}$ (on B) ever has outcome 1.

Lemma 6 (Compressed Semi-Classical O2H [35, Theorem 1]). Let D_q be the database register defined over D_q and $H : \{0,1\}^m \rightarrow \{0,1\}^n$ be the compressed standard oracle implemented on D_q . Let S be a subset of D_q such that $D^\perp \notin S$ and z be a random string, where $D^\perp$ is the database only contains q pairs $(\perp, 0^n)$. S and z may have arbitrary joint distribution $\mathfrak{D}$. Let $H \setminus S$ be an oracle that first queries H and then queries $\mathcal{O}_S^{\text{CSC}}$ on the database register D_q .

Let $\mathcal{A}$ be a quantum oracle algorithm (not necessarily unitary) that has oracle access to H and Read_f , and $\mathcal{A}$ queries H (resp. Read_f) at most $q_H \leq q^{11}$ (resp. q_R)

¹¹ This limitation on q_H is because the database register D_q can only be used to perfectly simulate at most q quantum random oracle queries.

times. Here Read_f (f may relate to z) is simulated by the database read operation Read_f defined in Eq. (46). Define

$$\begin{aligned} P_{\text{left}} &:= \Pr [1 \leftarrow \mathcal{A}^{\text{H}, \text{Read}_f}(z) : (S, z) \leftarrow \mathfrak{D}], \\ P_{\text{right}} &:= \Pr [1 \leftarrow \mathcal{A}^{\text{H} \setminus S, \text{Read}_f}(z) : (S, z) \leftarrow \mathfrak{D}], \\ P_{\text{find}} &:= \Pr [\text{Find} : \mathcal{A}^{\text{H} \setminus S, \text{Read}_f}(z), (S, z) \leftarrow \mathfrak{D}]. \end{aligned}$$

Then we have

$$|P_{\text{left}} - P_{\text{right}}| \leq \sqrt{(q_{\text{H}} + 1) \cdot P_{\text{find}}}, \quad \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq \sqrt{(q_{\text{H}} + 1) \cdot P_{\text{find}}}.$$

Define a projector on the database register D_q as $\text{J}_S := \sum_{D \in S} |D\rangle\langle D|$, then the P_{find} defined above and the CStO defined in Definition 9 satisfy

$$P_{\text{find}} \leq q_{\text{H}} \cdot \mathbb{E}_{(S, z) \leftarrow \mathfrak{D}} \|\text{J}_S, \text{CStO}\|^2.$$

Lemma 7 (Check Query Replacement [36, Theorem 1]). Let $\text{H} : \{0, 1\}^m \rightarrow \{0, 1\}^n$ be a quantum random oracle. Define quantum oracle $\text{O}_0 : |w, x, b\rangle \mapsto |w, x, b \oplus \llbracket \text{H}(x) \in S_{w,x} \rrbracket\rangle$, which has input/output register WX/B , and W , X , B is defined over the set W , $\{0, 1\}^m$, $\{0, 1\}$ respectively. The set $S_{w,x} \subseteq \{0, 1\}^n$ used by O_0 is determined by $w \in W$ and $x \in \{0, 1\}^m$. Let $\text{H}_c : \{0, 1\}^m \rightarrow \{0, 1\}^n$ be a compressed standard oracle with the database register $\text{D}_{q_{\text{H}}}$. Define quantum oracle $\text{O}_1 : |w, x, \text{D}_{\text{H}}, b\rangle \mapsto |w, x, \text{D}_{\text{H}}, b \oplus \llbracket \text{D}_{\text{H}}(x) \in S_{w,x} \rrbracket\rangle$, which has the same input/output register as O_0 and additionally uses $\text{D}_{q_{\text{H}}}$ as the work register.

Let $\mathcal{A}$ be a quantum oracle algorithm that makes at most q_{H} queries to H/H_c and at most q_0 queries to O_0/O_1 . Define

$$P_{\text{left}} := \Pr [b = 1 : b \leftarrow \mathcal{A}^{\text{H}, \text{O}_0}] \text{ and } P_{\text{right}} := \Pr [b = 1 : b \leftarrow \mathcal{A}^{\text{H}_c, \text{O}_1}].$$

Let $\Gamma_S := \max_{w \in W, x \in \{0, 1\}^m} |S_{w,x}|$, then we have

$$|P_{\text{left}} - P_{\text{right}}| \leq 6q_0 \cdot \sqrt{\Gamma_S/2^n} \text{ and } \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq 6q_0 \cdot \sqrt{\Gamma_S/2^n}.$$

A.2.2 Detailed Proof

Proof (of Theorem 8). We introduce a sequence of games as shown in Fig. 17 to prove this theorem. Similar to Fig. 16, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc in Fig. 17 for notational simplicity.

Game G_0 : This game is the IND-CCA security game of NEV-KEM-CCA with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-KEM-CCA}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr [1 \leftarrow \text{G}_0] - \frac{1}{2} \right|. \quad (47)$$

Game G_1 : Compared with game G_0 , this game has the following three differences:

1. Two new random oracles F and G are introduced.

2. When computing (c^*, K_b^*) and $\text{NEV-KEM-CCA.Decap}(sk, c)$, the $H_2(x_1, x_2)$ is replaced with $(G(x_1, x_2), F(x_1, x_2))$, where $G(x_1, x_2)$ is the (potential) session key and $F(x_1, x_2)$ is the randomness used by NEV-PKE-OW.Enc .
3. The adversary $\mathcal{A}$ is replaced with $\mathcal{A}_1$, which runs $\mathcal{A}$ and outputs its result. In particular, for $\mathcal{A}$'s H_2 query with the query state $\sum_{x_1, x_2} \alpha_{x_1, x_2} |x_1, x_2, y\rangle$, by (quantumly) querying F and G once each, $\mathcal{A}_1$ simulates it by transforming this state into $\sum_{x_1, x_2} \alpha_{x_1, x_2} |x_1, x_2, y \oplus (G(x_1, x_2), F(x_1, x_2))\rangle$.

One can note that $\mathcal{A}_1$ makes q_{H_1} queries to H_1 , q_{H_2} queries to both F and G , and q_D queries to NEV-KEM-CCA.Decap . According to the basic rules of random oracle, game hop $\mathbf{G}_0$ to $\mathbf{G}_1$ is only a conceptual change and we have

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \mathbf{G}_1]. \quad (48)$$

Game $\mathbf{G}_2$: This game is the same as $\mathbf{G}_1$, except that the ρ^* is replaced with a uniformly random value over $\{0, 1\}^\kappa$, and the F queried by NEV-KEM-CCA.Decap and $\mathcal{A}_1$ is replaced with $F[M^* \rightarrow \rho^*]$, a quantum oracle defined as follows:

- For the query state $|x, y\rangle$, a control operation is performed, that is, query F if $x \neq (M^*, H_1(pk))$, and act $U_{\rho^*} : |y\rangle \mapsto |y \oplus \rho^*\rangle$ on the output register otherwise.

The implementation of $F[M^* \rightarrow \rho^*]$ in game $\mathbf{G}_2$ is given in Appendix A.2.3, which requires two queries to F . Therefore, F is queried $2q_{H_2}$ times in game $\mathbf{G}_2$.

In fact, the value of $F(M^*, H_1(pk))$ in game $\mathbf{G}_1$ is reprogrammed into ρ^* in game $\mathbf{G}_2$. Since this reprogramming can be regarded as occurring prior to the adversary's query to F and ρ^* is a uniformly random value over $\{0, 1\}^\kappa$, games $\mathbf{G}_1$ and $\mathbf{G}_2$ are equivalent. Hence, we have

$$\Pr[1 \leftarrow \mathbf{G}_1] = \Pr[1 \leftarrow \mathbf{G}_2]. \quad (49)$$

Game $\mathbf{G}_3$: In this game, the decapsulation oracle NEV-KEM-CCA.Decap additionally checks whether $M = M^*$ and directly queries F to compute ρ . If $M = M^*$, NEV-KEM-CCA.Decap outputs $\perp$.

In game $\mathbf{G}_2$, during the computation of $\text{NEV-KEM-CCA.Decap}(sk, c \neq c^*)$, if $M^* = \text{NEV-PKE-OW.Dec}(sk, c)$, then the next re-encryption check cannot succeed since $c \neq c^*$ and $\text{NEV-PKE-OW.Enc}(pk, M^*; F(M^*, H_1(pk))) = c^*$, and this makes the oracle outputs $\perp$. Hence, the additional check added in game $\mathbf{G}_3$ has long been implicitly present in game $\mathbf{G}_2$. As for the $F[M^* \rightarrow \rho^*]$ queried by NEV-KEM-CCA.Decap , after we explicitly write out that additional check, $F[M^* \rightarrow \rho^*]$ will never be queried by $(M^*, H_1(pk))$, and hence is equivalent to F . Thus,

$$\Pr[1 \leftarrow \mathbf{G}_2] = \Pr[1 \leftarrow \mathbf{G}_3]. \quad (50)$$

Game $\mathbf{G}_4$: In this game, a new quantum oracle Test_F is introduced, which maps $|c, M, M, H_1(pk), b\rangle$ into $|c, M, M, H_1(pk), b \oplus \llbracket F(M, H_1(pk)) \in S_{c,M} \rrbracket\rangle$, where

$$S_{c,M} := \{r \in \{0, 1\}^\kappa : \text{NEV-PKE-OW.Enc}(pk, M; r) = c\}. \quad (51)$$

GAMES $\mathbf{G_0-G_9}$	$F(x, y\rangle)$ // $\mathbf{G_1-G_9}$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H_1 \leftarrow \text{Fcs}[\kappa]$ $H_2 \leftarrow \text{Fcs}[2\kappa, 2\kappa]$ $F \leftarrow \text{Fcs}[2\kappa]$ $G \leftarrow \text{Fcs}[2\kappa, \kappa]$ 2. $M^* \leftarrow \{0, 1\}^\kappa$ $(K_0^*, \rho^*) := H_2(M^*, H_1(pk))$ $K_0^* := G(M^*, H_1(pk))$ $K_0^* \leftarrow \{0, 1\}^\kappa$ $\rho^* := F(M^*, H_1(pk))$ $\rho^* \leftarrow \{0, 1\}^\kappa$ $c^* := \text{NEV-PKE-OW.Enc}(pk, M^*; \rho^*)$ $b \leftarrow \{0, 1\}, K_1^* \leftarrow \mathcal{K}$ 3. $b' \leftarrow \mathcal{A}^{H_1, H_2, \text{NEV-KEM-CCA.Decap}}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_1^{H_1, F, G, \text{NEV-KEM-CCA.Decap}}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_1^{H_1, F[M^* \rightarrow \rho^*], G, \text{NEV-KEM-CCA.Decap}}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_1^{H_1, F[M^* \rightarrow \rho^*], G, \text{Decap}}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_1^{H_1, F[M^* \rightarrow \rho^*], G, \text{Decap}^*}(pk, c^*, K_b^*)$ $b' \leftarrow \mathcal{A}_1^{H_1, F, G, \text{Decap}^*}(pk, c^*, K_b^*)$ 4. return $[b' = b]$	1. return $ x, y \oplus F(x)\rangle$ // $\mathbf{G_1-G_4}$ 2. perform CStO // $\mathbf{G_5-G_9}$ $F[M^* \rightarrow \rho^*](x, y\rangle)$ // $\mathbf{G_2-G_4}$ 1. if $x \neq (M^*, H_1(pk))$, query F with $ x, y\rangle$ else perform U_{ρ^*} to return $ x, y \oplus \rho^*\rangle$ $\text{NEV-KEM-CCA.Decap}(sk, c)$ // $\mathbf{G_0-G_3}$ 1. if $c = c^*$, return $\perp$ else $M := \text{NEV-PKE-OW.Dec}(sk, c)$ 2. if $M = \perp$, return $\perp$ if $M = \perp \vee M = M^*$, return $\perp$ // $\mathbf{G_3}$ else $(K, \rho) := H_2(M, H_1(pk))$ // $\mathbf{G_0}$ else $K := G(M, H_1(pk))$ // $\mathbf{G_1}$ $\rho := F(M, H_1(pk))$ // $\mathbf{G_1, G_3}$ $\rho := F[M^* \rightarrow \rho^*](M, H_1(pk))$ // $\mathbf{G_2}$ 3. if $c = \text{NEV-PKE-OW.Enc}(pk, M; \rho)$ return K else return $\perp$ $\text{Decap}(sk, c)$ // $\mathbf{G_4-G_6}$ 1. if $c = c^*$, return $\perp$ else $M := \text{NEV-PKE-OW.Dec}(sk, c)$ 2. if $M = \perp \vee M = M^*$, return $\perp$ if $M = \perp$, return $\perp$ // $\mathbf{G_6}$ else query Test_F with $ c, M, M, H_1(pk), 0\rangle$ // $\mathbf{G_4}$ query Test_D with $ c, M, M, H_1(pk), 0\rangle$ // $\mathbf{G_5-G_6}$ measure the final register to get b 3. if $b = 1$, return $G(M, H_1(pk))$ else return $\perp$ $\text{Decap}^*(sk, c)$ // $\mathbf{G_7-G_9}$ 1. if $c = c^*$, return $\perp$ else query Ext with $ c, 0^*\rangle$ // $\mathbf{G_7}$ else query Sea with $ c, 0^*\rangle$ // $\mathbf{G_8, G_9}$ measure the final register to get M 2. if $M \neq \perp$, return $G(M, H_1(pk))$ else return $\perp$
$\text{Test}_F(c, M, M, H_1(pk), b\rangle)$ // $\mathbf{G_4}$ 1. return $ c, M, M, H_1(pk), b \oplus [F(M, H_1(pk)) \in S_{c, M}]\rangle$	
$\text{Test}_D(c, M, M, H_1(pk), b\rangle)$ // $\mathbf{G_5}$ 1. Use $\text{D}_{2q_{H_2}}$ as the work register 2. Transform $ c, M, M, H_1(pk), D, b\rangle$ into $ c, M, M, H_1(pk), D, b \oplus [D(M, H_1(pk)) \in S_{c, M}]\rangle$	
$\text{Ext}(c, y\rangle)$ // $\mathbf{G_7}$ 1. Use $\text{D}_{2q_{H_2}}$ as the work register 2. Transform $ c, D, y\rangle$ into $ c, D, y \oplus \text{Extract}(c, D)\rangle$	
$\text{Sea}(c, y\rangle)$ // $\mathbf{G_8-G_9}$ 1. Use $\text{D}_{2q_{H_2}}$ as the work register 2. Transform $ c, D, y\rangle$ into $ c, D, y \oplus \text{Search}(c, D)\rangle$	

Fig. 17: Games $\mathbf{G_0}$ to $\mathbf{G_9}$ in the proof of Theorem 8.

Then, the NEV-KEM-CCA.Decap is replaced with Decap , a decapsulation oracle identical to NEV-KEM-CCA.Decap except that the computation of $G(M, H_1(pk))$ is moved to the end and the re-encryption check is replaced with $b = 1$, where b is generated as follows:

- After getting $M(\notin \{\perp, M^*\})$, query Test_F with the state $|c, M, H_1(pk), 0\rangle$, then measure the last qubit and denote the measurement outcome as b .

Since the Decap can only be classically queried in game $\mathbf{G}_4$, the Test_F query performed by Decap in game $\mathbf{G}_4$ is essentially a classical query to a quantum oracle. At this point, based on the definition of Test_F , it is not hard to verify that NEV-KEM-CCA.Decap is equivalent to Decap , and thus

$$\Pr[1 \leftarrow \mathbf{G}_3] = \Pr[1 \leftarrow \mathbf{G}_4]. \quad (52)$$

Game $\mathbf{G}_5$: Compared with game $\mathbf{G}_4$, this game has the following two changes:

1. The random oracle F queried by $F[M^* \rightarrow \rho^*]$ is implemented as a compressed standard oracle based on the database register $D_{2q_{H_2}}$.
2. The quantum oracle Test_F queried by Decap is replaced with Test_D , a new oracle uses $D_{2q_{H_2}}$ as the work register and transforms $|c, M, M, H_1(pk), D, b\rangle$ into $|c, M, M, H_1(pk), D, b \oplus \llbracket D(M, H_1(pk)) \in S_{c,M} \rrbracket\rangle$.

In fact, Test_D here performs a purified projective measurement related to c and M on the database register $D_{2q_{H_2}}$.

To analyze the game hop $\mathbf{G}_4$ to $\mathbf{G}_5$, we first consider a fixed (pk, sk) pair and define a quantum oracle algorithm $\mathcal{B}$ as follows:

- $\mathcal{B}(pk, sk)$ has quantum oracle access to F and Test_F , where F is a random oracle. Then, $\mathcal{B}$ runs game $\mathbf{G}_4$ and outputs its final result. In particular, $\mathcal{B}$ replies to the F (resp. Test_F) queries performed in game $\mathbf{G}_4$ by querying its own oracle F (resp. Test_F).

In fact, the F queries performed by $\mathcal{B}$ is used to implement the oracle $F[M^* \rightarrow \rho^*]$ defined in game $\mathbf{G}_2$. As shown in Appendix A.2.3, such implementation is valid whether F is a random oracle or a compressed standard oracle, and it requires two queries to F in either case. In addition, notice that simulating one query to Decap requires one query to Test_F in game $\mathbf{G}_4$. Hence, $\mathcal{B}$ queries F (resp. Test_F) at most $2q_{H_2}$ (resp. q_D) times, and we have

$$\Pr[1 \leftarrow \mathbf{G}_4 : (pk, sk)] = \Pr[1 \leftarrow \mathcal{B}^{F, \text{Test}_F}(pk, sk)].$$

Here $\Pr[1 \leftarrow \mathbf{G} : (pk, sk)]$ denotes the probability of game $\mathbf{G}$ outputs 1 under a fixed (pk, sk) pair.

Next, according to the definition of game $\mathbf{G}_5$, if F is implemented as a compressed standard oracle based on the database register $D_{2q_{H_2}}$ and Test_F is replaced with Test_D , $\mathcal{B}$ will run game $\mathbf{G}_5$. That is,

$$\Pr[1 \leftarrow \mathbf{G}_5 : (pk, sk)] = \Pr[1 \leftarrow \mathcal{B}^{F, \text{Test}_D}(pk, sk)].$$

In the above equation, we write “ F_c ” to stress that it is a compressed standard oracle. Now by using Lemma 7 with $\mathcal{B}$, we can compute

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_4 : (pk, sk)] - \Pr[1 \leftarrow \mathbf{G}_5 : (pk, sk)]| \\ & \leq 6q_D \cdot \sqrt{\max_{c,M} |S_{c,M}| / 2^\kappa} \\ & \stackrel{(a)}{=} 6q_D \cdot \sqrt{\max_{c,M} \Pr[c = \text{NEV-PKE-OW.Enc}(pk, M; r) : r \leftarrow \{0, 1\}^\kappa]}. \end{aligned}$$

Here (a) follows from the definition of $S_{c,M}$ as given in Eq. (51). Then, by averaging over $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ and using Lemma 4, we get

$$|\Pr[1 \leftarrow \mathbf{G}_4] - \Pr[1 \leftarrow \mathbf{G}_5]| \leq 6q_D \cdot \sqrt{\epsilon}. \quad (53)$$

Game $\mathbf{G}_6$: This game is the same as $\mathbf{G}_5$, except that the decapsulation oracle Decap no longer checks whether $M = M^*$.

In games $\mathbf{G}_5$ and $\mathbf{G}_6$, although the oracle Test_D involves the database register, it does not change the database state (in the computational basis) and only extract certain information from it. Thus, only the implementation of $F[M^* \rightarrow \rho^*]$ can change the database state. For the query state $|x, y\rangle$, $F[M^* \rightarrow \rho^*]$ queries the compressed standard oracle F if $x \neq (M^*, H_1(pk))$, while directly returning $|x, y \oplus \rho^*\rangle$ otherwise. This implies that during the execution of games $\mathbf{G}_5$ and $\mathbf{G}_6$, the database D superposed before and after every implementation of $F[M^* \rightarrow \rho^*]$ must satisfy $D(M^*, H_1(pk)) = \perp$, and thus $D(M^*, H_1(pk)) \notin S_{c,M}$ for any c and M . Therefore, in Decap , if $M = M^*$, the next single bit b obtained by first querying Test_D and then performing a measurement will always be 0, which further makes Decap output $\perp$. That is, game $\mathbf{G}_5$ is equivalent to game $\mathbf{G}_6$ and

$$\Pr[1 \leftarrow \mathbf{G}_5] = \Pr[1 \leftarrow \mathbf{G}_6]. \quad (54)$$

Game $\mathbf{G}_7$: In this game, the Decap is replaced with Decap^* , a new decapsulation oracle does not compute M and extracts it instead from the database register $D_{2q_{H_2}}$ by querying the quantum oracle Ext . Here Ext uses $D_{2q_{H_2}}$ as the work register and transforms $|c, D, y\rangle$ into $|c, D, y \oplus \text{Extract}(c, D)\rangle$. The function Extract used by Ext is defined as

$$\text{Extract}(c, D) := \begin{cases} M & \text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp \\ & \wedge D(M, H_1(pk)) \in S_{c,M}, \\ \perp & \text{otherwise.} \end{cases} \quad (55)$$

Since the computation of Extract requires the secret key sk , the decapsulation oracle Decap^* in game $\mathbf{G}_7$ still uses sk .

Compared with the Decap in game $\mathbf{G}_6$, the Decap^* in game $\mathbf{G}_7$ merely encapsulates the computation of M in Ext via the function Extract , while Extract also performs the checks of whether M equals $\perp$ and whether $D(M, H_1(pk))$ belongs to $S_{c,M}$. Hence, Decap^* is essentially a rewrite of Decap , and thus

$$\Pr[1 \leftarrow \mathbf{G}_6] = \Pr[1 \leftarrow \mathbf{G}_7]. \quad (56)$$

Game $\mathbf{G_8}$: In this game, the quantum oracle Ext queried by Decap^* is replaced by Sea , which is identical to Ext except that the internally computed function Extract is replaced by

$$\text{Search}(c, D) = \begin{cases} M & M \text{ is the minimum value s.t. } D(M, H_1(pk)) \neq \perp \\ & \wedge c = \text{NEV-PKE-OW.Enc}(pk, M; D(M, H_1(pk))), \\ \perp & \text{if such } M \text{ does not exist.} \end{cases} \quad (57)$$

Since the computation of Search does not require the secret key sk , the decapsulation oracle Decap^* in game $\mathbf{G_8}$ no longer uses sk and only uses the public key pk .

As for the difference between the success probabilities of games $\mathbf{G_7}$ and $\mathbf{G_8}$, we can prove the following lemma, which demonstrates that the decryption failure rate δ of the NEV-PKE-OW can be used to bound this difference. The detailed proof is given in Appendix A.2.4.

Lemma 8. $|\Pr[1 \leftarrow \mathbf{G_7}] - \Pr[1 \leftarrow \mathbf{G_8}]| \leq (32q_{H_2} + 16) \cdot \sqrt{\delta} + 128q_{H_2} \cdot \delta.$

Game $\mathbf{G_9}$: This game is the same as $\mathbf{G_8}$, except that K_0^* is replaced by a uniformly random value and the $F[M^* \rightarrow \rho^*]$ queried by $\mathcal{A}_1$ is changed back to F . Since the bit b is independent of $\mathcal{A}_1$'s view in game $\mathbf{G_9}$, we have

$$\Pr[1 \leftarrow \mathbf{G_9}] = \frac{1}{2}. \quad (58)$$

In fact, games $\mathbf{G_8}$ and $\mathbf{G_9}$ are equivalent if $\mathcal{A}_1$ has never queried G nor $F/F[M^* \rightarrow \rho^*]$ by $(M^*, H_1(pk))$, but what makes this somewhat complex is that $\mathcal{A}_1$ can perform quantum queries to these oracles. Based on this, we use the Semi-Classical One-Way to Hiding theorem [6] to prove the following lemma, and the detailed proof is given in Appendix A.2.5.

Lemma 9. *There exists a DNTRU adversary $\mathcal{B}_1$ and a sspRLWE adversary $\mathcal{B}_2$ whose running times are roughly the same as those of $\mathcal{A}$ and $|\Pr[1 \leftarrow \mathbf{G_8}] - \Pr[1 \leftarrow \mathbf{G_9}]| \leq (24q_{H_2} + 8q_D) \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2)}.$*

Finally, by combining Eq. (47) to Eq. (50), Eq. (52) to Eq. (54), Eq. (56), Eq. (58), Lemma 8, and Lemma 9, we obtain

$$\begin{aligned} \text{Adv}_{\text{NEV-KEM-CCA}}^{\text{IND-CCA}}(\mathcal{A}) &\leq (24q_{H_2} + 8q_D) \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2)} \\ &\quad + (32q_{H_2} + 16) \cdot \sqrt{\delta} + 128q_{H_2} \cdot \delta + 6q_D \cdot \sqrt{\epsilon}. \end{aligned}$$

This completes the proof of Theorem 8. $\square$

A.2.3 Implementation of $F[M^* \rightarrow \rho^*]$ In this subsection, we give an implementation of the oracle $F[M^* \rightarrow \rho^*]$ in game $\mathbf{G_2}$.

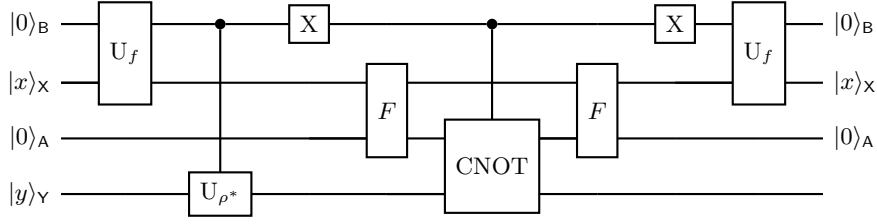


Fig. 18: The quantum circuit used to implement $F[M^* \rightarrow \rho^*]$.

Let f be a function that $f(x) = 1$ if $x = (M^*, H_1(pk))$, and $f(x) = 0$ otherwise. Obviously, f can be efficiently computed after getting the $(M^*, H_1(pk))$, and thus the unitary operation $U_f : |x, b\rangle \rightarrow |x, b \oplus f(x)\rangle$ can also be efficiently implemented by the basic rules of quantum computation. Based on U_f , the $F[M^* \rightarrow \rho^*]$ can be implemented by the following procedure, and a corresponding quantum circuit is also given in Fig. 18.

1. Denote X/Y as the input/output register of $F[M^* \rightarrow \rho^*]$. Initialize two new registers A and B to $|0\rangle$, where B is a single qubit register.
2. Apply U_f to XB .
3. Apply a conditional operation on BY : the controlling register is B , and apply $U_{\rho^*} : |y\rangle \mapsto |y \oplus \rho^*\rangle$ to Y if the state on B is $|1\rangle$.
4. Query F with the input/output register X/A .
5. Flip the state on B by applying an X gate.
6. Apply a conditional operation on BAY : the controlling register is B , and applying a CNOT gate on AY if the state on B is $|1\rangle$. Here CNOT maps $|y_1, y_2\rangle_{AY}$ into $|y_1, y_2 \oplus y_1\rangle_{AY}$.
7. Flip the state on B by applying an X gate.
8. Query F again with the input/output register X/A to uncompute the state on A , and apply U_f again to XB to uncompute the state on B . Now the states on A and B are both $|0\rangle$.

In the implementation above, F is treated as a random oracle and is queried two times. In fact, even if we introduce the database register $D_{2q_{H_2}}$ and regard F as a compressed standard oracle based on $D_{2q_{H_2}}$, it is not hard to check that the above implementation of $F[M^* \rightarrow \rho^*]$ is still valid, and the effect achieved by $F[M^* \rightarrow \rho^*]$ in this case is as follows:

- For the query state $|x, y\rangle$, a control operation is performed, that is, query F (a compressed standard oracle) if $x \neq (M^*, H_1(pk))$, and act $U_{\rho^*} : |y\rangle \mapsto |y \oplus \rho^*\rangle$ on the output register otherwise.

This is actually the $F[M^* \rightarrow \rho^*]$ of game $\mathbf{G}_2$ in the proof of Theorem 8.

A.2.4 Proof of Lemma 8

Proof. Based on the definition of **Extract** given in Eq. (55) and the definition of **Search** given in Eq. (57), we can conclude that $\text{Extract}(c, D) \neq \text{Search}(c, D)$ must

imply that the database D satisfies $D \in R_{pk,sk}$, where

$$R_{pk,sk} := \left\{ D \in \mathbf{D}_{2q_{H_2}} \left| \begin{array}{l} \exists M \text{ s.t. } D(M, H_1(pk)) \neq \perp, \\ c' := \text{NEV-PKE-OW.Enc}(pk, M; D(M, H_1(pk))) \\ \text{but NEV-PKE-OW.Dec}(sk, c') \neq M \end{array} \right. \right\}. \quad (59)$$

In the following, based on the above set $R_{pk,sk}$ and the games $\mathbf{G}_7$ and $\mathbf{G}_8$ defined in Fig. 17, we introduce two new games as follows:

Game $\mathbf{G}_7^a$: This game is the same as $\mathbf{G}_7$, except that the compressed semi-classical oracle $\mathcal{O}_{R_{pk,sk}}^{CSC}$ is queried (on the database register $\mathbf{D}_{2q_{H_2}}$) just after each query to the compressed standard oracle F .

Game $\mathbf{G}_8^a$: This game is the same as $\mathbf{G}_8$, except that the compressed semi-classical oracle $\mathcal{O}_{R_{pk,sk}}^{CSC}$ is queried (on the database register $\mathbf{D}_{2q_{H_2}}$) just after each query to the compressed standard oracle F .

We will use Lemma 6 to complete the proof. We first design a new quantum oracle algorithm $\mathcal{C}$ as follows to run game $\mathbf{G}_7$ under a fixed (pk, sk) pair.

- $\mathcal{C}(pk, sk)$ has oracle access to F and Ext , where F is a compressed standard oracle. Then, $\mathcal{C}$ runs game $\mathbf{G}_7$ and outputs its final result. In particular, $\mathcal{C}$ replies to the F (resp. Ext) queries performed in game $\mathbf{G}_7$ by querying its own oracle F (resp. Ext).

Similar to the quantum oracle algorithm $\mathcal{B}$ designed in the analysis of game hop $\mathbf{G}_4$ to $\mathbf{G}_5$, $\mathcal{C}$ queries F (resp. Ext) at most $2q_{H_2}$ (resp. q_D) times. Let $\mathfrak{D}$ be a distribution of the tuple $(R_{pk,sk}, pk, sk)$, where $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ and the set $R_{pk,sk}$ (Eq. (59)) is completely determined by (pk, sk) . Now, we have

$$\begin{aligned} \Pr[1 \leftarrow \mathbf{G}_7] &= \Pr[1 \leftarrow \mathcal{C}^{F, \text{Ext}}(pk, sk) : (R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}], \\ \Pr[1 \leftarrow \mathbf{G}_7^a] &= \Pr[1 \leftarrow \mathcal{C}^{F \setminus R_{pk,sk}, \text{Ext}}(pk, sk) : (R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}]. \end{aligned}$$

It is not hard to find that, if we replace Ext with Sea , $\mathcal{C}$ will run game $\mathbf{G}_8$ and

$$\begin{aligned} \Pr[1 \leftarrow \mathbf{G}_8] &= \Pr[1 \leftarrow \mathcal{C}^{F, \text{Sea}}(pk, sk) : (R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}], \\ \Pr[1 \leftarrow \mathbf{G}_8^a] &= \Pr[1 \leftarrow \mathcal{C}^{F \setminus R_{pk,sk}, \text{Sea}}(pk, sk) : (R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}]. \end{aligned}$$

According to the definitions of Ext and Sea given in Fig. 17, these two oracles both are simulated by a database read operation with respect to a particular function related to the tuple (pk, sk) . Hence, by using Lemma 6 with $\mathcal{C}$, F , Ext and $\mathfrak{D}$, we get

$$|\Pr[1 \leftarrow \mathbf{G}_7] - \Pr[1 \leftarrow \mathbf{G}_7^a]| \leq \sqrt{2q_{H_2}(2q_{H_2} + 1) \cdot \mathbb{E} \left\| [\mathbf{J}_{R_{pk,sk}}, \text{CStO}] \right\|^2}. \quad (60)$$

Similarly, by using Lemma 6 with $\mathcal{C}$, F , Sea and $\mathfrak{D}$, we get

$$|\Pr[1 \leftarrow \mathbf{G}_8] - \Pr[1 \leftarrow \mathbf{G}_8^a]| \leq \sqrt{2q_{H_2}(2q_{H_2} + 1) \cdot \mathbb{E} \left\| [\mathbf{J}_{R_{pk,sk}}, \text{CStO}] \right\|^2}. \quad (61)$$

The expectations in the above two equations are taken over $(R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}$. In addition, by the analysis given in the beginning, we know that $\text{Extract}(c, D) = \text{Search}(c, D)$ if $D \notin R_{pk,sk}$. Hence, $\mathcal{C}^{F \setminus R_{pk,sk}, \text{Ext}}(pk, sk)$ and $\mathcal{C}^{F \setminus R_{pk,sk}, \text{Sea}}(pk, sk)$ have identical execution process if the compressed semi-classical oracle $\mathcal{O}_{R_{pk,sk}}^{CSC}$ never has a measurement outcome of 1. This means that

$$\begin{aligned} & \Pr [\text{Find} : \mathcal{C}^{F \setminus R_{pk,sk}, \text{Ext}}(pk, sk), (R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}] \\ &= \Pr [\text{Find} : \mathcal{C}^{F \setminus R_{pk,sk}, \text{Sea}}(pk, sk), (R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}] \end{aligned}$$

and

$$\begin{aligned} & |\Pr [1 \leftarrow \mathbf{G}_7^a] - \Pr [1 \leftarrow \mathbf{G}_8^a]| \\ & \leq \Pr [\text{Find} : \mathcal{C}^{F \setminus R_{pk,sk}, \text{Sea}}(pk, sk), (R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}] \\ & \stackrel{(a)}{\leq} 2q_{H_2} \cdot \mathbb{E} \left\| [\mathbf{J}_{R_{pk,sk}}, \text{CStO}] \right\|^2. \end{aligned} \quad (62)$$

Here (a) uses Lemma 6 again. By combining Eq. (60) to Eq. (62), we get

$$\begin{aligned} |\Pr [1 \leftarrow \mathbf{G}_7] - \Pr [1 \leftarrow \mathbf{G}_8]| & \leq 2 \cdot \sqrt{2q_{H_2}(2q_{H_2} + 1) \cdot \mathbb{E} \left\| [\mathbf{J}_{R_{pk,sk}}, \text{CStO}] \right\|^2} \\ & \quad + 2q_{H_2} \cdot \mathbb{E} \left\| [\mathbf{J}_{R_{pk,sk}}, \text{CStO}] \right\|^2, \end{aligned} \quad (63)$$

where the expectation is taken over $(R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}$.

Next, we compute the upper bound of $\|[\mathbf{J}_{R_{pk,sk}}, \text{CStO}]\|$. For a fixed (pk, sk) pair, we define function $g : \{0, 1\}^{n/k} \times \{0, 1\}^\kappa \rightarrow \{0, 1\}$ as

$$g(x, y) = \begin{cases} 1 & \text{if NEV-PKE-OW.Dec}(sk, \text{NEV-PKE-OW.Enc}(pk, x; y)) \neq x \\ 0 & \text{otherwise.} \end{cases} \quad (64)$$

Then, we define a relation R_1^g and a corresponding parameter $\Gamma_{R_1^g}$ as follows:

$$\begin{aligned} R_1^g &:= \{(x, y) \in \{0, 1\}^{n/k} \times \{0, 1\}^\kappa \mid g(x, y) = 1\}, \\ \Gamma_{R_1^g} &:= \max_{x \in \{0, 1\}^{n/k}} |\{y \in \{0, 1\}^\kappa \mid g(x, y) = 1\}|. \end{aligned}$$

Furthermore, by denoting $\mathbf{I}$ as the identity operator, we define several projectors related to R_1^g on the database register $\mathbf{D}_{2q_{H_2}}$ as

$$\Sigma^x := \sum_{\substack{D \text{ s.t. } (x, D(x)) \in R_1^g \\ x' < x, (x', D(x')) \notin R_1^g}} |D\rangle\langle D| \quad (x \in \{0, 1\}^{n/k}), \quad \Sigma^\perp := \mathbf{I} - \sum_{x \in \{0, 1\}^{n/k}} \Sigma^x.$$

By the definition of $R_{pk,sk}$ given in Eq. (59) and the definition of g given in Eq. (64), it is not hard to verify that $\mathbf{J}_{R_{pk,sk}} = \sum_{x \in \{0, 1\}^{n/k}} \Sigma^x$, and thus $\Sigma^\perp = \mathbf{I} - \mathbf{J}_{R_{pk,sk}}$. Hence, we can compute

$$\|[\mathbf{J}_{R_{pk,sk}}, \text{CStO}]\| \stackrel{(a)}{=} \|\mathbf{I} - \mathbf{J}_{R_{pk,sk}}, \text{CStO}\| = \|\Sigma^\perp, \text{CStO}\| \stackrel{(b)}{\leq} 8 \cdot \sqrt{\Gamma_{R_1^g}/2^\kappa}.$$

Here (a) follows from the basic property of commutator, and (b) follows from [35, Lemma 4]. Then, we have

$$\begin{aligned} \mathbb{E}_{(R_{pk,sk}, pk, sk) \leftarrow \mathfrak{D}} \left\| [J_{R_{pk,sk}}, \text{CStO}] \right\|^2 &\leq 64 \cdot \mathbb{E}_{(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}} \Gamma_{R_1^g} / 2^\kappa \\ &\stackrel{(a)}{\leq} 64 \cdot \delta \end{aligned} \quad (65)$$

Here (a) follows from the definitions of $\Gamma_{R_1^g}$ and δ (Definition 1). Finally, by combining Eq. (63) and Eq. (65), we get

$$|\Pr[1 \leftarrow \mathbf{G}_7] - \Pr[1 \leftarrow \mathbf{G}_8]| \leq (32q_{H_2} + 16) \cdot \sqrt{\delta} + 128q_{H_2} \cdot \delta.$$

This completes the proof of Lemma 8. $\square$

A.2.5 Proof of Lemma 9 Before giving the proof, we first introduce the semi-classical oracle and some related lemmas.

Definition 11 (Semi-Classical Oracle [6]). For a subset $S \subseteq \mathcal{X}$, the indicator function f_S from $\mathcal{X}$ to $\{0, 1\}$ equals 1 iff its input $x \in S$. The semi-classical oracle, $\mathcal{O}_S^{\text{SC}}$, is a quantum accessible oracle that performs the following two operations on its query state $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle$:

1. Initialize a single qubit register $\mathbf{L}$ with the state $|0\rangle_{\mathbf{L}}$.
2. Transform $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |0\rangle_{\mathbf{L}}$ into $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |f_S(x)\rangle_{\mathbf{L}}$ and then measure $\mathbf{L}$.

During the execution of a quantum oracle algorithm $\mathcal{A}^{\mathcal{O}_S^{\text{SC}}}$, let **Find** be the classical event that the measurement of $\mathcal{O}_S^{\text{SC}}$ (on $\mathbf{L}$) ever has outcome 1.

Lemma 10. Let $S \subseteq \mathcal{X}$ be random. Let z be a random bit string. (S, z may have arbitrary joint distribution $\mathfrak{D}$.) Let $\mathcal{A}$ be a quantum oracle algorithm with query times q (not necessarily unitary). Let

$$\begin{aligned} P_{\text{left}} &:= \Pr[b = 1 : b \leftarrow \mathcal{A}^{\emptyset}(z), (S, z) \leftarrow \mathfrak{D}], \\ P_{\text{right}} &:= \Pr[b = 1 : b \leftarrow \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}]. \end{aligned}$$

Here $\emptyset$ denotes an oracle that does nothing on the query state. Let $P_{\text{find}} := \Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}]$. Then

$$|P_{\text{left}} - P_{\text{right}}| \leq \sqrt{(q+1) \cdot P_{\text{find}}} \text{ and } \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq \sqrt{(q+1) \cdot P_{\text{find}}}.$$

Proof. This is a direct corollary of [6, Theorem 1] by setting the function H it used as a constant zero function.

Lemma 11 (Search in Semi-Classical Oracle [6, Theorem 2]). Let $S \subseteq \mathcal{X}$ be random and z be a random bitstring. (S, z) may have an arbitrary joint distribution $\mathfrak{D}$. Let $\mathcal{A}$ be any quantum oracle algorithm making at most q queries to a semi-classical oracle with domain $\mathcal{X}$.

Let $\mathcal{B}$ be an algorithm that on input z , chooses $i \leftarrow_{\$} [q]$; runs $\mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z)$ until (just before) the i -th query, measures the input register and outputs the measurement outcome x . Then

$$\Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}] \leq 4q \cdot \Pr[x \in S : x \leftarrow \mathcal{B}(z), (S, z) \leftarrow \mathfrak{D}].$$

Now we prove Lemma 9. Denote $S := \{(M^*, H_1(pk))\}$, and let $\mathcal{O}_S^{SC}$ be a semi-classical oracle [6] with respect to the set S . Then, we introduce two new games:

Game $\mathbf{G}_8^a$: This game is the same as $\mathbf{G}_8$, except that just before each query to the $F[M^* \rightarrow \rho^*]$ and G , the $\mathcal{O}_S^{SC}$ is queried on their input registers.

Game $\mathbf{G}_9^a$: This game is the same as $\mathbf{G}_9$, except that just before each query to the F and G , the $\mathcal{O}_S^{SC}$ is queried on their input registers.

Next, we design a new quantum oracle algorithm $\mathcal{C}$ with the input (pk, c^*, M^*) and oracle access to $O \in \{\emptyset, \mathcal{O}_S^{SC}\}$, where $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$, $M^* \leftarrow \{0, 1\}^{n/k}$, $\rho^* \leftarrow \{0, 1\}^\kappa$ and $c^* = \text{NEV-PKE-OW.Enc}(pk, M^*; \rho^*)$. The running steps of $\mathcal{C}$ are given below.

- $\mathcal{C}(pk, c^*, M^*)$ runs $\mathcal{A}_1^{F[M^* \rightarrow \rho^*], G, H_1, \text{Decap}^*}(pk, c^*, K_b^*)$ in game $\mathbf{G}_8$ and finally outputs $\llbracket b = b' \rrbracket$, where $b \leftarrow \{0, 1\}$, $K_0^* = G(M^*, H_1(pk))$, $K_1^* \leftarrow \{0, 1\}^\kappa$, and b' is the output of $\mathcal{A}_1$. During the running of $\mathcal{A}_1$, $\mathcal{C}$ will query its oracle O on the input registers of $F[M^* \rightarrow \rho^*]$ and G just before each query to them.

Recall that the decapsulation oracle Decap^* also has potential classical queries to G . Hence, according to the definition of $\mathcal{C}$, before performing these queries the oracle O should be (classically) queried first. Thus, $\mathcal{C}$ queries O at most $3q_{H_2} + q_D$ times. Now, for games $\mathbf{G}_8$ and $\mathbf{G}_8^a$, we have

$$\Pr[1 \leftarrow \mathbf{G}_8] = \Pr[1 \leftarrow \mathcal{C}^\emptyset(pk, c^*, M^*)], \Pr[1 \leftarrow \mathbf{G}_8^a] = \Pr[1 \leftarrow \mathcal{C}^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)].$$

Here and in the following, we omit the sampling of (pk, c^*, M^*) for simplicity. Let $\mathcal{C}_1$ be a quantum oracle algorithm the same as $\mathcal{C}$, except that it runs the $\mathcal{A}_1^{F, G, H_1, \text{Decap}^*}(pk, c^*, K_b^*)$ in game $\mathbf{G}_9$, where $b \leftarrow \{0, 1\}$ and $K_0^*, K_1^* \leftarrow \{0, 1\}^\kappa$. Similarly, we have

$$\Pr[1 \leftarrow \mathbf{G}_9] = \Pr[1 \leftarrow \mathcal{C}_1^\emptyset(pk, c^*, M^*)], \Pr[1 \leftarrow \mathbf{G}_9^a] = \Pr[1 \leftarrow \mathcal{C}_1^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)].$$

By applying Lemma 10, we can obtain

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_8] - \Pr[1 \leftarrow \mathbf{G}_8^a]| &\leq \sqrt{(3q_{H_2} + q_D) \cdot \Pr[\text{Find} : \mathcal{C}^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)]}, \\ |\Pr[1 \leftarrow \mathbf{G}_9] - \Pr[1 \leftarrow \mathbf{G}_9^a]| &\leq \sqrt{(3q_{H_2} + q_D) \cdot \Pr[\text{Find} : \mathcal{C}_1^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)]}. \end{aligned}$$

On the condition that Find does not occur, i.e., the projective measurement performed by $\mathcal{O}_S^{SC}$ always yields the result 0, $F[M^* \rightarrow \rho^*]$ and F are equivalent and $G(M^*, H_1(pk))$ is uniformly random in the adversary's view. This means that

$$\Pr[\text{Find} : \mathcal{C}^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)] = \Pr[\text{Find} : \mathcal{C}_1^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)]$$

and

$$\begin{aligned} &|\Pr[1 \leftarrow \mathbf{G}_8^a] - \Pr[1 \leftarrow \mathbf{G}_9^a]| \\ &= \left| \Pr[1 \leftarrow \mathcal{C}^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)] - \Pr[1 \leftarrow \mathcal{C}_1^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)] \right| \\ &\leq \Pr[\text{Find} : \mathcal{C}_1^{\mathcal{O}_S^{SC}}(pk, c^*, M^*)] \stackrel{(a)}{=} \Pr[\text{Find} : \mathcal{C}_1^{\mathcal{O}_S^{SC}}(pk, c^*)]. \end{aligned}$$

The equation (a) holds because the input M^* to $\mathcal{C}_1$ is redundant and can thus be directly omitted. In fact, in accordance with the definition of $\mathcal{C}_1$, it no longer needs to use M^* to generate K_0^* and simulate $F[M^* \rightarrow \rho^*]$ when running $\mathcal{A}_1$.

At this point, by using Lemma 11, we can upper bound the probability of Find during the running of $\mathcal{C}_1^{\mathcal{O}_{SC}}(pk, c^*)$ using the success probability of the following quantum algorithm $\mathcal{D}(pk, c^*)$.

- After getting the public key pk and challenge ciphertext c^* , $\mathcal{D}$ samples $i \leftarrow \{1, \dots, 3q_{H_2} + q_D\}$ and $K \leftarrow \{0, 1\}^\kappa$. Then, $\mathcal{D}$ runs $\mathcal{A}_1^{F, G, H_1, \text{Decap}^*}(pk, c^*, K)$ and simulates the oracles for $\mathcal{A}_1$ in the following manner:
 - By preparing the database register $D_{2q_{H_2}}$, $\mathcal{D}$ implements F as a compressed standard oracle.
 - $\mathcal{D}$ uses a $2q$ -wise independent function to simulate G , and uses a $2q$ -wise independent function to simulate H_1 .
 - $\mathcal{D}$ simulates Decap^* in accordance with the game $\mathbf{G}_9$.
- $\mathcal{D}$ measures the i -th query to F/G . Denote the measurement outcome as (M, x) , then $\mathcal{D}$ outputs M .

That is to say, we have

$$\begin{aligned} \Pr \left[\text{Find} : \mathcal{C}_1^{\mathcal{O}_{SC}}(pk, c^*) \right] &\leq 4(3q_{H_2} + q_D) \cdot \Pr [M = M^* : M \leftarrow \mathcal{D}(pk, c^*)] \\ &= 4(3q_{H_2} + q_D) \cdot \text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{D}) \\ &\stackrel{(a)}{=} 4(3q_{H_2} + q_D) \cdot \left(\text{Adv}_{n, q, \chi_f, \chi_g, v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n, q, \chi_r, B_\eta, \bar{v}}^{\text{ssPRLWE}}(\mathcal{B}_2) \right). \end{aligned}$$

Here (a) uses Theorem 2. Finally, by combining the above equations, we obtain

$$|\Pr[1 \leftarrow \mathbf{G}_8] - \Pr[1 \leftarrow \mathbf{G}_9]| \leq (24q_{H_2} + 8q_D) \cdot \sqrt{\text{Adv}_{n, q, \chi_f, \chi_g, v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n, q, \chi_r, B_\eta, \bar{v}}^{\text{ssPRLWE}}(\mathcal{B}_2)}.$$

This completes the proof of Lemma 9. $\square$

A.3 Proof of Theorem 9

Proof. To prove this theorem, we introduce a sequence of games as shown in Fig. 19. In these games, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc for notational simplicity.

Game $\mathbf{G}_0$: This game is the IND-CCA security game of NEV-PKE-CCA with the adversary $\mathcal{A}$. The random oracle H_2 is simulated on-the-fly by using the record list L_H . Obviously, we have

$$\text{Adv}_{\text{NEV-PKE-CCA}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (66)$$

Game $\mathbf{G}_1$: This game is the same as $\mathbf{G}_0$, except that the NEV-PKE-CCA.Dec is replaced with Dec^* , which is a new decryption oracle does not use the secret key sk . For a decryption query (c, d) , Dec^* only uses the public key pk to compute

GAMES $\mathbf{G_0-G_3}$	$H_2(x)$ // $\mathbf{G_0-G_3}$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW, KeyGen}$ $H_1 \leftarrow \text{Fcs}[* , \kappa]$ $H_3 \leftarrow \text{Fcs}[n/k, \ell]$	// $\mathbf{G_0-G_3}$ 1. if $\exists y$ s.t. $(x, y) \in L_H$, return y // $\mathbf{G_0-G_3}$ 2. else $y \leftarrow \{0, 1\}^\kappa$, $L_H := L_H \cup (x, y)$ // $\mathbf{G_0-G_3}$ return y
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{NEV-PKE-CCA.Dec}}(pk)$ $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{Dec}^*}(pk)$ $(m_0^*, m_1^*) \leftarrow \mathcal{A}_1^{H_1, H_2, H_3}(pk)$	// $\mathbf{G_0}$ $\text{NEV-PKE-CCA.Dec}(sk, c, d)$ // $\mathbf{G_0}$ // $\mathbf{G_1}$ // $\mathbf{G_2-G_3}$ 1. if (c^*, d^*) is generated and $(c, d) = (c^*, d^*)$ // $\mathbf{G_0-G_3}$ return $\perp$ // $\mathbf{G_0-G_3}$ else $M := \text{NEV-PKE-OW.Dec}(sk, c)$ // $\mathbf{G_0-G_2}$ 2. if $M = \perp$, return $\perp, \rho := H_2(M, H_1(pk), d)$ // $\mathbf{G_3}$ 3. if $c = \text{NEV-PKE-OW.Enc}(pk, M; \rho)$ // $\mathbf{G_0-G_2}$ return $H_3(M) \oplus d$ // $\mathbf{G_3}$ else return $\perp$
3. $b \leftarrow \{0, 1\}$ $M^* \leftarrow \{0, 1\}^{n/k}$ $d^* := H_3(M^*) \oplus m_b^*$ $d^* \leftarrow \{0, 1\}^\ell$ $\rho^* := H_2(M^*, H_1(pk), d^*)$ $\rho^* \leftarrow \{0, 1\}^\kappa$ $c^* := \text{NEV-PKE-OW.Enc}(pk, M^*; \rho^*)$	// $\mathbf{G_0-G_3}$ $\text{Dec}^*(pk, c, d)$ // $\mathbf{G_1}$ // $\mathbf{G_0}$ 1. if (c^*, d^*) is generated and $(c, d) = (c^*, d^*)$ // $\mathbf{G_1}$ return $\perp$ // $\mathbf{G_2-G_3}$ else $x := \text{Search}(pk, c, d, L_H)$ // $\mathbf{G_0-G_3}$ 2. if $x \neq \perp$, return $H_3(x) \oplus d$, else return $\perp$
4. $b' \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{NEV-PKE-CCA.Dec}}(pk, c^*, d^*)$ $b' \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{Dec}^*}(pk, c^*, d^*)$ $b' \leftarrow \mathcal{A}_1^{H_1, H_2, H_3}(pk, c^*, d^*)$	
5. return $\llbracket b' = b \rrbracket$	

Fig. 19: Games $\mathbf{G_0}$ to $\mathbf{G_3}$ in the proof of Theorem 9. The record lists L_F and L_H used here, both of which are empty sets at the beginning of the game.

$x := \text{Search}(pk, c, d, L_H)$, and then returns $H_3(x) \oplus d$ if $x \neq \perp$. The function Search used here is defined as

$$\text{Search}(pk, c, d, L_H) = \begin{cases} M & \text{if } M \text{ is the minimum value s.t. } L_H(M, H_1(pk), d) \neq \perp \\ & \wedge c = \text{NEV-PKE-OW.Enc}(pk, M; L_H(M, H_1(pk), d)), \\ \perp & \text{if such } M \text{ does not exist.} \end{cases} \quad (67)$$

By the definitions of the oracles NEV-PKE-CCA.Dec and Dec^* , their results can only differ in the following three cases:

1. The decryption query (c, d) and the record list L_H just before such query satisfy $\text{Search}(pk, c, d, L_H) \neq \perp$, but $\text{NEV-PKE-OW.Dec}(sk, c)$ equals $\perp$ or $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$ and

$$c \neq \text{NEV-PKE-OW.Enc}(pk, M; H_2(M, H_1(pk), d)).$$

In this case, $\text{NEV-PKE-CCA.Dec}(sk, c, d) = \perp$ but $\text{Dec}^*(pk, c, d) \neq \perp$.

2. The decryption query (c, d) and the record list L_H just before such query satisfy $\text{Search}(pk, c, d, L_H) = \perp$, but $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$ and

$$c = \text{NEV-PKE-OW.Enc}(pk, M; H_2(M, H_1(pk), d)).$$

In this case, $\text{NEV-PKE-CCA.Dec}(sk, c, d) \neq \perp$ but $\text{Dec}^*(pk, c, d) = \perp$.

3. The decryption query (c, d) and the record list L_H just before such query satisfy $\text{Search}(pk, c, d, L_H) \neq \perp$, $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$ and

$$c = \text{NEV-PKE-OW.Enc}(pk, M; H_2(M, H_1(pk), d)),$$

but $\text{Search}(pk, c, d, L_H) \neq M$. In this case, neither $\text{NEV-PKE-CCA.Dec}(sk, c, d)$ nor $\text{Dec}^*(pk, c, d)$ equals $\perp$ but these two values are different.

Notice that the decryption query (c, d) used above does not equal the challenge ciphertext (c^*, d^*) after (c^*, d^*) is generated. Based on the above three cases, we define two events **Error** and **Guess** that may occur in games $\mathbf{G}_0$ and $\mathbf{G}_1$ as follows:

- **Error**: $\mathcal{A}$ ever makes a decryption query that triggers the above case 1 or 3.
- **Guess**: $\mathcal{A}$ ever makes a decryption query that triggers the above case 2.

At this point, it is obvious that games $\mathbf{G}_0$ and $\mathbf{G}_1$ proceed identically if the event $\text{Error} \vee \text{Guess}$ does not occur. Hence, by the fundamental lemma, we have

$$|\Pr[1 \leftarrow \mathbf{G}_0] - \Pr[1 \leftarrow \mathbf{G}_1]| \leq \Pr[\text{Error} : \mathbf{G}_1] + \Pr[\text{Guess} : \mathbf{G}_1]. \quad (68)$$

We first consider the event **Guess**. If a decryption query (c, d) makes **Guess** occurs, we have $\text{Search}(pk, c, d, L_H) = \perp$, $\text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp$, and $c = \text{NEV-PKE-OW.Enc}(pk, M; H_2(M, H_1(pk), d))$, where L_H is the record list just before the decryption query (c, d) . Now we know that $L_H(M, H_1(pk), d) = \perp$, otherwise by the basic rules of the on-the-fly simulation we have $H_2(M, H_1(pk), d) = L_H(M, H_1(pk), d)$ and $c = \text{NEV-PKE-OW.Enc}(pk, M; L_H(M, H_1(pk), d))$, which contradicts with $\text{Search}(pk, c, d, L_H) = \perp$ by the definition of **Search** given in Eq. (67). Since $L_H(M, H_1(pk), d) = \perp$ means that $H_2(M, H_1(pk), d)$ is a uniformly random value, the probability of $c = \text{NEV-PKE-OW.Enc}(pk, M; H_2(M, H_1(pk), d))$ does not exceed

$$\max_{M, c} \Pr[c = \text{NEV-PKE-OW.Enc}(pk, M; \rho) : \rho \leftarrow \{0, 1\}^\kappa].$$

Recall that $\mathcal{A}$ makes q_D decryption queries. Therefore, by the union bound and Lemma 4, it is not hard to obtain

$$\Pr[\text{Guess} : \mathbf{G}_1] \leq q_D \cdot \epsilon, \quad (69)$$

where $\epsilon = \max_{r_0 \in R} \Pr[r = r_0 : r \leftarrow \chi_r]$.

As for the event **Error**, we first consider a fixed (pk, sk) pair. It can be observed that **Error** must imply that $\mathcal{A}$ has queried H_2 by a $(M, H_1(pk), d)$ such that $c = \text{NEV-PKE-OW.Enc}(pk, M; L_H(M, H_1(pk), d))$, but $\text{NEV-PKE-OW.Dec}(sk, c) \neq M$. Recall that H_2 is simulated on-the-fly and is queried at most q_{H_2} times by $\mathcal{A}$. Therefore, by denoting

$$\delta_{pk, sk} = \max_M \Pr_{\rho \leftarrow \{0, 1\}^\kappa} [\text{NEV-PKE-OW.Dec}(sk, \text{NEV-PKE-OW.Enc}(pk, M; \rho)) \neq M],$$

and then by the union bound, it is not hard to obtain

$$\Pr[\text{Error} : \mathbf{G}_1, (pk, sk)] \leq q_{H_2} \cdot \delta_{pk, sk}.$$

Here $\Pr[\text{Error} : \mathbf{G}_1, (pk, sk)]$ denotes the probability of **Error** in game $\mathbf{G}_1$ under a fixed (pk, sk) pair. Then by averaging over $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ and using the fact that **NEV-PKE-OW** has decryption failure rate δ , we have

$$\Pr[\text{Error} : \mathbf{G}_1] \leq q_{H_2} \cdot \delta. \quad (70)$$

Game $\mathbf{G}_2$: This game is the same as $\mathbf{G}_1$, except that the adversary $\mathcal{A}$ is replaced with a new adversary $\mathcal{A}_1$ that does not query the decryption oracle. Here $\mathcal{A}_1$ runs $\mathcal{A}$, outputs its final result, and simulates H_1 , H_2 , H_3 , and Dec^* for $\mathcal{A}$ as follows:

1. For the H_1 (resp. H_3) queries, $\mathcal{A}_1$ replies by querying its own oracle H_1 (resp. H_3).
2. For the H_2 queries, $\mathcal{A}_1$ replies by querying its own oracle H_2 . Meanwhile, $\mathcal{A}_1$ sets up a record list $L_{\mathcal{A},H}$ to record these queries and the corresponding responses.
3. For the decryption query (c, d) , $\mathcal{A}_1$ returns $\perp$ if (c^*, d^*) is generated and $(c, d) = (c^*, d^*)$. Otherwise, $\mathcal{A}_1$ computes $x := \text{Search}(pk, c, d, L_{\mathcal{A},H})$. If $x = \perp$, $\mathcal{A}_1$ returns $\perp$; if $x \neq \perp$, $\mathcal{A}_1$ returns $H_3(x) \oplus d$.

By the definition of **Search** given in Eq. (67), $\mathcal{A}_1$ queries H_1 , H_2 , and H_3 at most q_{H_1} , q_{H_2} , and $q_D + q_{H_3}$ times, respectively.

In game $\mathbf{G}_1$, since (c^*, d^*) is forbidden from being used to query the decryption oracle, the computation of the function **Search** will never involve the pair $(M^*, H_1(pk) || d^*, \rho^*)$ of the record list L_H after the generation of (c^*, d^*) . That is to say, the function **Search** will only search the list that records the H_2 queries made by $\mathcal{A}$, i.e., the list $L_{\mathcal{A},H}$ introduced above. This means that game $\mathbf{G}_2$ is actually an equivalent rewrite of game $\mathbf{G}_1$, and thus we have

$$\Pr[1 \leftarrow \mathbf{G}_1] = \Pr[1 \leftarrow \mathbf{G}_2]. \quad (71)$$

Game $\mathbf{G}_3$: This game is the same as $\mathbf{G}_2$, except that the ρ^* and d^* used to generate c^* are both replaced with fresh random values.

Denote Bad_{M^*} as the event that $\mathcal{A}_1$ ever queries H_2 with $(M^*, *, *)$ or H_3 with M^* . It is not hard to see that game $\mathbf{G}_2$ is identical to game $\mathbf{G}_3$ if Bad_{M^*} does not occur. Hence, by the fundamental difference lemma, we have

$$|\Pr[1 \leftarrow \mathbf{G}_2] - \Pr[1 \leftarrow \mathbf{G}_3]| \leq \Pr[\text{Bad}_{M^*} : \mathbf{G}_3]. \quad (72)$$

Since the bit b is independent with the view of $\mathcal{A}_1$ in game $\mathbf{G}_3$, we have

$$\Pr[1 \leftarrow \mathbf{G}_3] = \frac{1}{2}. \quad (73)$$

In the following, we define a new adversary $\mathcal{B}$ against the OW-CPA security of **NEV-PKE-OW** as follows:

- After getting the public key pk and challenge ciphertext c^* , $\mathcal{B}$ samples $d^* \leftarrow \{0, 1\}^\ell$ and runs $\mathcal{A}_1^{H_1, H_2, H_3}(pk, c^*, d^*)$ in game $\mathbf{G}_3$. $\mathcal{B}$ random chooses i from $\{1, \dots, q_D + q_{H_2} + q_{H_3}\}$ and then outputs the first κ bits of $\mathcal{A}_1$'s i -th H_2/H_3 queries.

The above definition implies that, if Bad_{M^*} occurs, $\mathcal{B}$ will output M^* with probability at least $1/(q_D + q_{H_2} + q_{H_3})$ and hence break the OW-CPA security of NEV-PKE-OW. Thus,

$$\begin{aligned} \Pr[\text{Bad}_{M^*} : \mathbf{G}_3] &\leq (q_D + q_{H_2} + q_{H_3}) \cdot \text{Adv}_{\text{NEV-PKE-OW}}^{\text{OW-CPA}}(\mathcal{B}) \\ &\stackrel{(a)}{=} (q_D + q_{H_2} + q_{H_3}) \cdot \left(\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2) \right). \end{aligned} \quad (74)$$

Here (a) follows from Theorem 2. By combining Eq. (66), Eq. (68) to Eq. (74), we finally obtain

$$\begin{aligned} \text{Adv}_{\text{NEV-PKE-CCA}}^{\text{IND-CCA}}(\mathcal{A}) &\leq (q_D + q_{H_2} + q_{H_3}) \cdot \left(\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{ssRLWE}}(\mathcal{B}_2) \right) \\ &\quad + q_D \cdot \epsilon + q_{H_2} \cdot \delta. \end{aligned}$$

This completes the proof of Theorem 9. $\square$

A.4 Proof of Theorem 10

Before giving the proof, we first introduce the following reprogramming lemma, which will be used later.

$\text{REPRO}_0(\mathcal{A})$	$\text{REPRO}_1(\mathcal{A})$
1. $H \leftarrow \text{Fcs}[m, n]$	1. $H \leftarrow \text{Fcs}[m, n]$
2. $(st, x_2^*) \leftarrow \mathcal{A}_1^H$	2. $(st, x_2^*) \leftarrow \mathcal{A}_1^H$
3. $x_1^* \leftarrow \{0, 1\}^m, y^* \leftarrow \{0, 1\}^n$	3. $x_1^* \leftarrow \{0, 1\}^m, y^* \leftarrow \{0, 1\}^n$
4. $b \leftarrow \mathcal{A}_2^H(x_1^*, st)$	$H \leftarrow H[(x_1^*, x_2^*) \rightarrow y^*]$
	4. $b \leftarrow \mathcal{A}_2^H(x_1^*, st)$

Fig. 20: Games $\text{REPRO}_0(\mathcal{A})$ and $\text{REPRO}_1(\mathcal{A})$ of Lemma 12. Here $H[(x_1^*, x_2^*) \rightarrow y^*]$ is the same as H , except that $H[(x_1^*, x_2^*) \rightarrow y^*](x_1^*, x_2^*) = y^*$.

Lemma 12. *For any two-stage adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ that quantumly queries its oracle q times, the games $\text{REPRO}_0(\mathcal{A})$ and $\text{REPRO}_1(\mathcal{A})$ shown in Fig. 20 satisfy*

$$|\Pr[1 \leftarrow \text{REPRO}_0(\mathcal{A})] - \Pr[1 \leftarrow \text{REPRO}_1(\mathcal{A})]| \leq \frac{3}{2} \sqrt{\frac{q}{2^m}}.$$

Proof (of Lemma 12). This is a direct corollary of [38, Proposition 1] by setting the reprogramming times " R " to 1. $\square$

Proof (of Theorem 10). We introduce a sequence of games as shown in Fig. 21 to prove this theorem. Similar with Fig. 19, we omit the detailed computation of NEV-PKE-OW.KeyGen and NEV-PKE-OW.Enc in Fig. 21 for notational simplicity.

GAMES $\mathbf{G_0-G_{11}}$	NEV-PKE-CCA.Dec(sk, c, d) (before (c^*, d^*)) // $\mathbf{G_0-G_5}$
1. $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ $H_1 \leftarrow \text{Fcs}[*, \kappa]$, $H_3 \leftarrow \text{Fcs}[n/k, \ell]$ $H_2 \leftarrow \text{Fcs}[n/k + \kappa + \ell, \kappa]$ $\rho^\diamond \leftarrow \{0, 1\}^\kappa$ 2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{NEV-PKE-CCA.Dec}(pk)}$ $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{H_1, H_2[M^* \rightarrow \rho^\diamond], H_3, \text{NEV-PKE-CCA.Dec}(pk)}$ $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{H_1, H_2[M^* \rightarrow \rho^\diamond], H_3, \text{Dec}(pk, \cdot)}$ $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{H_1, H_2[M^* \rightarrow \rho^\diamond], H_3, \text{Dec}^*(pk)}$ $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{Dec}^*(pk)}$ 3. $b \leftarrow \{0, 1\}$, $M^* \leftarrow \{0, 1\}^{n/k}$ $d^* := H_3(M^*) \oplus m_b^*$ $y^* \leftarrow \{0, 1\}^k$, $d^* := y^* \oplus m_b^*$ $d^* \leftarrow \{0, 1\}^k$, $y^* := d^* \oplus m_b^*$ $\rho^* := H_2(M^*, H_1(pk), d^*)$ $\rho^* \leftarrow \{0, 1\}^\kappa$ $c^* := \text{NEV-PKE-OW.Enc}(pk, M^*; \rho^*)$ 4. $b' \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{NEV-PKE-CCA.Dec}(pk, c^*, d^*)}$ $b' \leftarrow \mathcal{A}^{H_1, H_2, H_3[M^* \rightarrow y^*], \text{NEV-PKE-CCA.Dec}(pk, c^*, d^*)}$ $b' \leftarrow \mathcal{A}^{H_1, H_2[M^* \rightarrow \rho^*], H_3[M^* \rightarrow y^*], \text{NEV-PKE-CCA.Dec}(pk, c^*, d^*)}$ $b' \leftarrow \mathcal{A}^{H_1, H_2[M^* \rightarrow \rho^*], H_3[M^* \rightarrow y^*], \text{Dec}(pk, c^*, d^*)}$ $b' \leftarrow \mathcal{A}^{H_1, H_2[M^* \rightarrow \rho^*], H_3[M^* \rightarrow y^*], \text{Dec}^*(pk, c^*, d^*)}$ $b' \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{Dec}^*(pk, c^*, d^*)}$ 5. return $[b' = b]$	1. $M := \text{NEV-PKE-OW.Dec}(sk, c)$ 2. if $M = \perp$, return $\perp$ // $\mathbf{G_0-G_6}$ else $\rho := H_2(M, H_1(pk), d)$ // $\mathbf{G_4-G_{10}}$ 3. if $c = \text{NEV-PKE-OW.Enc}(pk, M; \rho)$ // $\mathbf{G_0-G_3}$ return $H_3(M) \oplus d$ // $\mathbf{G_4-G_5}$ else return $\perp$ // $\mathbf{G_6-G_8}$ NEV-PKE-CCA.Dec(sk, c, d) (after (c^*, d^*)) // $\mathbf{G_0-G_5}$ // $\mathbf{G_9-G_{10}}$ 1. if $(c, d) = (c^*, d^*)$, return $\perp$ // $\mathbf{G_{11}}$ else $M := \text{NEV-PKE-OW.Dec}(sk, c)$ // $\mathbf{G_0-G_{11}}$ 2. if $M = \perp$, return $\perp$ // $\mathbf{G_0-G_4}$ // $\mathbf{G_0}$ if $M = \perp \vee (M, d) = (M^*, d^*)$, return $\perp$ // $\mathbf{G_5}$ // $\mathbf{G_1}$ else $\rho := H_2(M, H_1(pk), d)$ // $\mathbf{G_0-G_2}$ // $\mathbf{G_2}$ else $\rho := H_2[M^* \rightarrow \rho](M, H_1(pk), d)$ // $\mathbf{G_3-G_4}$ // $\mathbf{G_0-G_2}$ else $\rho := H_2(M, H_1(pk), d)$ // $\mathbf{G_5}$ // $\mathbf{G_3-G_{11}}$ 3. if $c = \text{NEV-PKE-OW.Enc}(pk, M; \rho)$ // $\mathbf{G_0-G_{11}}$ return $H_3(M) \oplus d$ // $\mathbf{G_0}$ // $\mathbf{G_0}$ return $H_3[M^* \rightarrow y^*](M) \oplus d$ // $\mathbf{G_1-G_5}$ else return $\perp$
$H_2(x, y\rangle)$ // $\mathbf{G_0-G_{11}}$ 1. return $x, y \oplus H_2(x)\rangle$ // $\mathbf{G_0-G_6}$ 2. perform CStO // $\mathbf{G_7-G_{11}}$ $H_2[M^* \rightarrow \rho^*](x, y\rangle)$ // $\mathbf{G_3-G_{10}}$ 1. if $x \neq (M^*, H_1(pk), d^*)$, query H_2 with $x, y\rangle$ else perform U_{ρ^*} to return $x, y \oplus \rho^*\rangle$ $H_2[M^* \rightarrow \rho^\diamond](x, y\rangle)$ // $\mathbf{G_4-G_{10}}$ 1. if $x \neq (M^*, H_1(pk), d^*)$, query H_2 with $x, y\rangle$ else perform $\text{U}_{\rho^\diamond}$ to return $x, y \oplus \rho^\diamond\rangle$ $\text{Test}_H(c, M, M, H_1(pk), d, b\rangle)$ // $\mathbf{G_6}$ 1. return $c, M, M, H_1(pk), b \oplus [F(M, H_1(pk)) \in S_{c, M}]\rangle$ $\text{Test}_D(c, M, M, H_1(pk), d, b\rangle)$ // $\mathbf{G_7-G_8}$ 1. Use $\text{D}_{2q_{H_2}}$ as the work register 2. Transform $c, M, M, H_1(pk), D, b\rangle$ into $c, M, M, H_1(pk), D, b \oplus [D(M, H_1(pk)) \in S_{c, M}]\rangle$ $\text{Ext}(c, y\rangle)$ // $\mathbf{G_9}$ 1. Use $\text{D}_{2q_{H_2}}$ as the work register 2. Transform $c, D, y\rangle$ into $c, D, y \oplus \text{Extract}(c, D)\rangle$ $\text{Sea}(c, y\rangle)$ // $\mathbf{G_{10-G_{11}}}$ 1. Use $\text{D}_{2q_{H_2}}$ as the work register 2. Transform $c, D, y\rangle$ into $c, D, y \oplus \text{Search}(c, D)\rangle$	Dec(sk, c, d) (before (c^*, d^*)) // $\mathbf{G_6-G_8}$ 1. if $(c, d) = (c^*, d^*)$, return $\perp$ else $M := \text{NEV-PKE-OW.Dec}(sk, c)$ 2. if $M = \perp \vee (M, d) = (M^*, d^*)$, return $\perp$ // $\mathbf{G_6-G_7}$ if $M = \perp$, return $\perp$ // $\mathbf{G_8}$ else query Test_H with $c, M, M, H_1(pk), d, 0\rangle$ // $\mathbf{G_6}$ query Test_D with $c, M, M, H_1(pk), d, 0\rangle$ // $\mathbf{G_7-G_8}$ measure the final register to get b 3. if $b = 1$ return $H_3(M) \oplus d$ else return $\perp$ Dec(sk, c, d) (after (c^*, d^*)) // $\mathbf{G_6-G_8}$ 1. if $(c, d) = (c^*, d^*)$, return $\perp$ else $M := \text{NEV-PKE-OW.Dec}(sk, c)$ 2. if $M = \perp \vee (M, d) = (M^*, d^*)$, return $\perp$ // $\mathbf{G_6-G_7}$ if $M = \perp$, return $\perp$ // $\mathbf{G_8}$ else query Test_H with $c, M, M, H_1(pk), d, 0\rangle$ // $\mathbf{G_6}$ query Test_D with $c, M, M, H_1(pk), d, 0\rangle$ // $\mathbf{G_7-G_8}$ measure the final register to get b 3. if $b = 1$ return $H_3[M^* \rightarrow y^*](M) \oplus d$ else return $\perp$ Dec(sk, c) // $\mathbf{G_9-G_{11}}$ 1. if (c^*, d^*) is defined and $(c, d) = (c^*, d^*)$ return $\perp$ 2. query Ext with $c, d, 0^*\rangle$ // $\mathbf{G_9}$ query Sea with $c, d, 0^*\rangle$ // $\mathbf{G_{10-G_{11}}}$ measure the final register to get M 3. if $M \neq \perp$ // $\mathbf{G_9-G_{10}}$ return $H_3(M) \oplus d$ before (c^*, d^*) is defined // $\mathbf{G_9-G_{10}}$ return $H_3[M^* \rightarrow y^*](M) \oplus d$ // $\mathbf{G_9-G_{10}}$ after (c^*, d^*) is defined // $\mathbf{G_9-G_{10}}$ if $M \neq \perp$, return $H_3(M) \oplus d$ // $\mathbf{G_{11}}$ else return $\perp$

Fig. 21: Games $\mathbf{G_0}$ to $\mathbf{G_{11}}$ in the proof of Theorem 10.

Game $\mathbf{G}_0$: This game is the IND-CCA security game of NEV-PKE-CCA with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{NEV-PKE-CCA}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (75)$$

Game $\mathbf{G}_1$: Compared with game $\mathbf{G}_0$, this game has two differences: first, d^* is computed as $y^* \oplus m_b^*$, where y^* is a uniformly random value over $\{0, 1\}^\ell$; then, after (c^*, d^*) is defined, the H_3 queried by $\mathcal{A}$ and NEV-PKE-CCA.Dec is replaced by $H_3[M^* \rightarrow y^*]$, a new function identical to H_3 except that $H_3[M^* \rightarrow y^*](M^*) = y^*$.

In the following, we use Lemma 12 to analyze the difference between games $\mathbf{G}_0$ and $\mathbf{G}_1$. We first construct the following two stage adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$:

- $\mathcal{B}_1$ has quantum oracle access to H_3 , it runs the first two steps of game $\mathbf{G}_0$, replies the H_3 queries by querying its oracle H_3 , and finally return (st, x_2^*) , where $st = (m_0^*, m_1^*)$ and $x_2 = \emptyset$.
- $\mathcal{B}_2$ has quantum oracle access to H_3 and input (x_1^*, st) , where $x_1^* = M^*$ and $M^* \leftarrow \{0, 1\}^{n/k}$. $\mathcal{B}_2$ samples $b \leftarrow \{0, 1\}$, queries H_3 to compute $d^* = H_3(M^*) \oplus m_b^*$ and then computes c^* as game $\mathbf{G}_0$. Finally, $\mathcal{B}_2$ runs the fourth step of game $\mathbf{G}_0$, replies the H_3 queries by querying its oracle H_3 , and returns $\llbracket b' = b \rrbracket$.

At this point, based on the definition of $\mathcal{B}$ and the two games defined in Fig. 20, we have

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \text{REPRO}_0(\mathcal{B})], \Pr[1 \leftarrow \mathbf{G}_1] = \Pr[1 \leftarrow \text{REPRO}_1(\mathcal{B})].$$

Since $\mathcal{B}$ queries H_3 at most $q_{H_3} + 1$ times, by using Lemma 12 we have

$$|\Pr[1 \leftarrow \mathbf{G}_0] - \Pr[1 \leftarrow \mathbf{G}_1]| \leq \frac{3}{2} \sqrt{\frac{q_{H_3} + 1}{2^{n/k}}}. \quad (76)$$

Game $\mathbf{G}_2$: This game is the same as $\mathbf{G}_1$, except that the generation of y^* and d^* is changed. In this game, d^* is a uniformly random value over $\{0, 1\}^\ell$ and $y^* = d^* \oplus m_b^*$. This is only a conceptual change and we have

$$\Pr[1 \leftarrow \mathbf{G}_1] = \Pr[1 \leftarrow \mathbf{G}_2]. \quad (77)$$

Game $\mathbf{G}_3$: Compared with game $\mathbf{G}_2$, this game has two differences: first, ρ^* is replaced by a uniformly random value over $\{0, 1\}^\kappa$; then, after (c^*, d^*) is defined, the H_2 queried by $\mathcal{A}$ and NEV-PKE-CCA.Dec is replaced by $H_2[M^* \rightarrow \rho^*]$, which is identical to H_2 except that $H_2[M^* \rightarrow \rho^*](M^*, H_1(pk), d^*) = \rho^*$. The implementation of $H_2[M^* \rightarrow \rho^*]$ is similar to the $F[M^* \rightarrow \rho^*]$ given in Appendix A.2.3, which requires two queries to H_2 . Hence, H_2 is queried $2q_{H_2}$ times in this game.

By a similar analysis as the game hop $\mathbf{G}_0$ to $\mathbf{G}_1$, and set the x_1^* and x_2^* used by $\mathcal{B}$ as (M^*, d^*) and $H_1(pk)$, respectively, we have

$$|\Pr[1 \leftarrow \mathbf{G}_2] - \Pr[1 \leftarrow \mathbf{G}_3]| \leq \frac{3}{2} \sqrt{\frac{q_{H_2} + 1}{2^{n/k+\ell}}}. \quad (78)$$

Game $\mathbf{G}_4$: In this game, a fresh uniformly random $\rho^\diamond$ over $\{0, 1\}^\kappa$ is sampled, and during the phase in which $\mathcal{A}$ generates (m_0^*, m_1^*) , the H_2 queried by $\mathcal{A}$ is replaced by $H_2[M^* \rightarrow \rho^\diamond]$, which is identical to H_2 except $H_2[M^* \rightarrow \rho^\diamond](M^*, H_1(pk), d^*) = \rho^\diamond$.

We stress that game $\mathbf{G}_4$ is well defined. In fact, since M^* and d^* are sampled uniformly at random in game $\mathbf{G}_3$, we can equivalently regard them as being sampled at the beginning of the game, which further enables us to adjust the view of $\mathcal{A}$ while generating (m_0^*, m_1^*) . As for the difference between game $\mathbf{G}_3$ and game $\mathbf{G}_4$, since M^* and d^* are sampled uniformly random, $\mathcal{A}$ can hardly detect the modifications we made in game $\mathbf{G}_4$. The detailed proof for this claim follows a similar argument to that of Lemma 9: we first use semi-classical oracle $\mathcal{O}_{\{(M^*, H_1(pk), d^*)\}}^{SC}$ to puncture $\mathcal{A}$'s quantum queries to H_2 that are performed in the procedure of generating (m_0^*, m_1^*) , then equivalently replace H_2 with its reprogrammed version $H_2[M^* \rightarrow \rho^\diamond]$, proceed to estimate the probability that event Find occurs, and finally obtain

$$|\Pr[1 \leftarrow \mathbf{G}_3] - \Pr[1 \leftarrow \mathbf{G}_4]| \leq \frac{8(q_{H_2} + 1)}{\sqrt{2^{n/k+\ell}}}. \quad (79)$$

For the sake of simplicity, we omit the complete proof here.

Game $\mathbf{G}_5$: In this game, after the challenge ciphertext (c^*, d^*) is generated, the decryption oracle NEV-PKE-CCA.Dec additionally checks whether $(M, d) = (M^*, d^*)$ and directly queries H_2 to compute ρ . If $(M, d) = (M^*, d^*)$, NEV-PKE-CCA.Dec outputs $\perp$.

In game $\mathbf{G}_4$ and after the (c^*, d^*) is generated, during the computation of $\text{NEV-KEM-CCA.Dec}(sk, c, d)$ for $(c, d) \neq (c^*, d^*)$, if $M^* = \text{NEV-PKE-OW.Dec}(sk, c)$ and $d = d^*$, then the next re-encryption check cannot succeed since this makes $c = \text{NEV-PKE-OW.Enc}(pk, M^*; \rho^*) = c^*$, which contradicts with $(c, d) \neq (c^*, d^*)$. This means that, the additional check added in game $\mathbf{G}_5$ has long been implicitly present in game $\mathbf{G}_4$. As for the function $H_2[M^* \rightarrow \rho^*]$ used by NEV-PKE-CCA.Dec , after we explicitly write out that additional check, $H_2[M^* \rightarrow \rho^*]$ will never be queried by $(M^*, H_1(pk), d^*)$ and hence is equivalent to H_2 . Thus,

$$\Pr[1 \leftarrow \mathbf{G}_4] = \Pr[1 \leftarrow \mathbf{G}_5]. \quad (80)$$

Game $\mathbf{G}_6$: In this game, a new quantum oracle Test_H is introduced, which maps $|c, M, M, H_1(pk), d, b\rangle$ into $|c, M, M, H_1(pk), d, b \oplus \llbracket H_2(M, H_1(pk), d) \in S_{c,M} \rrbracket\rangle$, where

$$S_{c,M} := \{\rho \in \{0, 1\}^\kappa : \text{NEV-PKE-OW.Enc}(pk, M; \rho) = c\}. \quad (81)$$

Then, the NEV-PKE-CCA.Dec is replaced with Dec , a decryption oracle identical to NEV-PKE-CCA.Dec except that the re-encryption check is replaced with $b = 1$, where b is generated as follows:

- After getting M , query Test_H with the state $|c, M, M, H_1(pk), d, 0\rangle$, then measure the last qubit and denote the measurement outcome as b .

Since the Dec can only be classically queried, the Test_H query performed by Dec in game $\mathbf{G}_6$ is essentially a classical query to a quantum oracle. At this point, based on the definition of Test_H , it is not hard to verify that NEV-PKE-CCA.Dec is equivalent to Dec , and thus

$$\Pr[1 \leftarrow \mathbf{G}_5] = \Pr[1 \leftarrow \mathbf{G}_6]. \quad (82)$$

Game $\mathbf{G}_7$: Compared with game $\mathbf{G}_6$, this game has the following two changes:

1. The random oracle H_2 queried by $H_2[M^* \rightarrow \rho^*]/H_2[M^* \rightarrow \rho^\diamond]$ is implemented as a compressed standard oracle based on the database register $D_{2q_{H_2}}$.
2. The quantum oracle Test_H queried by Dec is replaced with Test_D , a new oracle uses $D_{2q_{H_2}}$ as the work register and transforms $|c, M, M, H_1(pk), d, D, b\rangle$ into $|c, M, M, H_1(pk), d, D, b \oplus \llbracket D(M, H_1(pk), d) \in S_{c,M} \rrbracket\rangle$.

In fact, Test_D here performs a purified projective measurement related to c and M on the database register $D_{2q_{H_2}}$.

To analyze the game hop $\mathbf{G}_6$ to $\mathbf{G}_7$, we first consider a fixed (pk, sk) pair and define a quantum oracle algorithm $\mathcal{B}$ as follows:

- $\mathcal{B}(pk, sk)$ has quantum oracle access to H_2 and Test_H , where H_2 is a random oracle. Then, $\mathcal{B}$ runs game $\mathbf{G}_6$ and outputs its final result. In particular, $\mathcal{B}$ replies to the H_2 (resp. Test_H) queries performed in game $\mathbf{G}_6$ by querying its own oracle H_2 (resp. Test_H).

In fact, some of the H_2 queries performed by $\mathcal{B}$ is used to implement the oracle $H_2[M^* \rightarrow \rho^*]/H_2[M^* \rightarrow \rho^\diamond]$. The implementation of $H_2[M^* \rightarrow \rho^*]/H_2[M^* \rightarrow \rho^\diamond]$ is similar to that of $F[M^* \rightarrow \rho^*]$ given in Appendix A.2.3, which is valid whether H_2 is a random oracle or a compressed standard oracle, and it requires two queries to H_2 in either case. In addition, notice that simulating one query to Dec requires one query to Test_H in game $\mathbf{G}_6$. Hence, $\mathcal{B}$ queries H_2 (resp. Test_H) at most $2q_{H_2}$ (resp. q_D) times, and we have

$$\Pr[1 \leftarrow \mathbf{G}_6 : (pk, sk)] = \Pr[1 \leftarrow \mathcal{B}^{H_2, \text{Test}_H}(pk, sk)].$$

Here $\Pr[1 \leftarrow \mathbf{G} : (pk, sk)]$ denotes the probability of game $\mathbf{G}$ outputs 1 under a fixed (pk, sk) pair.

Next, according to the definition of game $\mathbf{G}_7$, if H_2 is implemented as a compressed standard oracle based on the database register $D_{2q_{H_2}}$ and Test_H is replaced with Test_D , $\mathcal{B}$ will run game $\mathbf{G}_7$. That is,

$$\Pr[1 \leftarrow \mathbf{G}_7 : (pk, sk)] = \Pr[1 \leftarrow \mathcal{B}^{H_2^c, \text{Test}_D}(pk, sk)].$$

In the above equation, we write " H_2^c " to stress that it is a compressed standard oracle. Now by using Lemma 7 with $\mathcal{B}$, we can compute

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_6 : (pk, sk)] - \Pr[1 \leftarrow \mathbf{G}_7 : (pk, sk)]| \\ & \leq 6q_D \cdot \sqrt{\max_{c,M} |S_{c,M}| / 2^\kappa} \\ & \stackrel{(a)}{=} 6q_D \cdot \sqrt{\max_{c,M} \Pr[c = \text{NEV-PKE-OW.Enc}(pk, M; r) : r \leftarrow \{0, 1\}^\kappa]}. \end{aligned}$$

Here (a) follows from the definition of $S_{c,M}$ as given in Eq. (81). Then, by averaging over $(pk, sk) \leftarrow \text{NEV-PKE-OW.KeyGen}$ and using Lemma 4, we get

$$|\Pr[1 \leftarrow \mathbf{G}_6] - \Pr[1 \leftarrow \mathbf{G}_7]| \leq 6q_D \cdot \sqrt{\epsilon}. \quad (83)$$

Game $\mathbf{G}_8$: This game is the same as $\mathbf{G}_7$, except that the decryption oracle Dec no longer checks whether $(M, d) = (M^*, d^*)$ after the (c^*, d^*) is defined.

Although the oracle Test_D involves the database register, it does not change the database state (in the computational basis) and only extract some information from it. Thus, only the implementation of $H_2[M^* \rightarrow \rho^*]/H_2[M^* \rightarrow \rho^\diamond]$ may change the database state. For the query state $|x, y\rangle$, $H_2[M^* \rightarrow \rho^*]/H_2[M^* \rightarrow \rho^\diamond]$ queries the compressed standard oracle H_2 if $x \neq (M^*, H_1(pk), d^*)$, while directly returning $|x, y \oplus \rho^*\rangle/|x, y \oplus \rho^\diamond\rangle$ otherwise. This implies that during the execution of games $\mathbf{G}_7$ and $\mathbf{G}_8$, the database D superposed before and after every implementation of $H_2[M^* \rightarrow \rho^*]/H_2[M^* \rightarrow \rho^\diamond]$ must satisfy $D(M^*, H_1(pk), d^*) = \perp$, and thus $D(M^*, H_1(pk), d^*) \notin S_{c,M}$ for any c and M . Therefore, in Dec , if $(M, d) = (M^*, d^*)$, the next single bit b obtained by first querying Test_D and then performing a measurement will always be 0, which further makes Dec output $\perp$. That is, game $\mathbf{G}_7$ is equivalent to game $\mathbf{G}_8$ and

$$\Pr[1 \leftarrow \mathbf{G}_7] = \Pr[1 \leftarrow \mathbf{G}_8]. \quad (84)$$

Game $\mathbf{G}_9$: In this game, the Dec is replaced with Dec^* , a new decryption oracle does not compute M and extracts it instead from the database register $D_{2q_{H_2}}$ by querying the quantum oracle Ext . Here Ext uses $D_{2q_{H_2}}$ as the work register and transforms $|c, d, D, y\rangle$ into $|c, d, D, y \oplus \text{Extract}(c, d, D)\rangle$. The function Extract used by Ext is defined as

$$\text{Extract}(c, d, D) = \begin{cases} M & \text{NEV-PKE-OW.Dec}(sk, c) = M \neq \perp \\ & \wedge D(M, H_1(pk), d) \in S_{c,M}, \\ \perp & \text{otherwise.} \end{cases} \quad (85)$$

Since the computation of Extract requires the secret key sk , the simulation of the decryption oracle Dec^* in game $\mathbf{G}_9$ still requires sk .

Compared with the Dec in game $\mathbf{G}_8$, the Dec^* in game $\mathbf{G}_9$ merely encapsulates the computation of M in Ext via the function Extract , while Extract also performs the checks of whether M equals $\perp$ and whether $D(M, H_1(pk), d)$ belongs to $S_{c,M}$. Hence, Dec^* is essentially a rewrite of Dec , and thus

$$\Pr[1 \leftarrow \mathbf{G}_8] = \Pr[1 \leftarrow \mathbf{G}_9]. \quad (86)$$

Game $\mathbf{G}_{10}$: In this game, the quantum oracle Ext queried by Dec^* is replaced with Sea , which is identical to Ext except that the internally computed function

Extract is replaced with

$$\text{Search}(c, d, D) = \begin{cases} M & M \text{ is the minimum value s.t. } D(M, H_1(pk), d) \neq \perp \\ & \wedge c = \text{NEV-PKE-OW.Enc}(pk, M; D(M, H_1(pk), d)), \\ \perp & \text{if such } M \text{ does not exist.} \end{cases} \quad (87)$$

Since the computation of **Search** does not require the secret key sk , the decapsulation oracle Dec^* in game $\mathbf{G}_{10}$ no longer uses sk and only uses the public key pk .

As for the difference between the success probabilities of game $\mathbf{G}_9$ and $\mathbf{G}_{10}$, we can prove the following lemma, which demonstrates that the decryption failure rate δ of the **NEV-PKE-OW** can be used to bound this difference.

Lemma 13. $|\Pr[1 \leftarrow \mathbf{G}_9] - \Pr[1 \leftarrow \mathbf{G}_{10}]| \leq (32q_{H_2} + 16) \cdot \sqrt{\delta} + 128q_{H_2} \cdot \delta.$

Proof. The proof is similar to Lemma 8, and we omit the detailed proof. $\square$

Game $\mathbf{G}_{11}$: This game is the same as $\mathbf{G}_8$, except that the oracles $H_2[M^* \rightarrow \rho^*]$ and $H_2[M^* \rightarrow \rho^\diamond]$ queried by $\mathcal{A}$ are both changed back to H_2 , and the oracle $H_3[M^* \rightarrow y^*]$ queried by $\mathcal{A}$ and Dec^* is changed back to H_3 . Since the bit b is independent of $\mathcal{A}_1$'s view in game $\mathbf{G}_{11}$, we have

$$\Pr[1 \leftarrow \mathbf{G}_{11}] = \frac{1}{2}. \quad (88)$$

In fact, game $\mathbf{G}_{10}$ and game $\mathbf{G}_{11}$ are equivalent if $H_2[M^* \rightarrow \rho^*]$ and $H_2[M^* \rightarrow \rho^\diamond]$ have not been queried by $(M^*, H_1(pk), d^*)$ and $H_3[M^* \rightarrow y^*]$ has not been queried by M^* , but what makes this somewhat complex is that $\mathcal{A}$ can quantumly query these oracles. Based on this, we use the Semi-Classical One-Way to Hiding theorem [6] to prove the following lemma.

Lemma 14. *There exists a DNTRU adversary $\mathcal{B}_1$ and a sspRLWE adversary $\mathcal{B}_2$ whose running times are roughly the same as those of $\mathcal{A}$ and $|\Pr[1 \leftarrow \mathbf{G}_{10}] - \Pr[1 \leftarrow \mathbf{G}_{11}]| \leq (24q_{H_2} + 8q_D) \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2)}.$*

Proof. The proof is similar to Lemma 9, and we omit the detailed proof. $\square$

Finally, by combining Eq. (75) to Eq. (80), Eq. (82) to Eq. (84), Eq. (86), Eq. (88), Lemma 13, and Lemma 14, we obtain

$$\begin{aligned} \text{Adv}_{\text{NEV-PKE-CCA}}^{\text{IND-CCA}}(\mathcal{A}) &\leq (24q_{H_2} + 8q_D) \cdot \sqrt{\text{Adv}_{n,q,\chi_f,\chi_g,v}^{\text{DNTRU}}(\mathcal{B}_1) + \text{Adv}_{n,q,\chi_r,B_\eta,\bar{v}}^{\text{sspRLWE}}(\mathcal{B}_2)} \\ &\quad + (32q_{H_2} + 16) \cdot \sqrt{\delta} + 128q_{H_2} \cdot \delta + 6q_D \cdot \sqrt{\epsilon} \\ &\quad + \frac{3}{2} \sqrt{\frac{q_{H_3} + 1}{2^{n/k}}} + \frac{10(q_{H_2} + 1)}{\sqrt{2^{n/k+\ell}}}. \end{aligned}$$

This completes the proof of Theorem 10. $\square$